

Lab2_FFNN_RNN_GNN

May 17, 2026

1 Laboratory 2 — Model Engineering

text

1.1 Setup

```
[1]: # --- Check Python and pip versions ---  
!python --version  
!pip install --upgrade pip
```

Python 3.12.13

Requirement already satisfied: pip in /usr/local/lib/python3.12/dist-packages (24.1.2)

Collecting pip

Downloading pip-26.1.1-py3-none-any.whl.metadata (4.6 kB)

Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)

1.8/1.8 MB

30.0 MB/s eta 0:00:00

Installing collected packages: pip

Attempting uninstall: pip

Found existing installation: pip 24.1.2

Uninstalling pip-24.1.2:

Successfully uninstalled pip-24.1.2

Successfully installed pip-26.1.1

```
[2]: # --- Install required libraries ---  
!pip install torch  
!pip install numpy pandas scikit-learn matplotlib seaborn  
!pip install tqdm  
!pip install torch_geometric  
# torch_sparse torch_scatter # TODO: check
```

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-

packages (from torch) (75.2.0)
 Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
 Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
 Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
 Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
 Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
 Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
 Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
 Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
 Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
 Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
 Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
 Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
 Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
 Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)
 Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
 Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
 Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
 Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
 Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
 Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
 Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
 Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch) (1.5.3)
 Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)

Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)

Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)

Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)

Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)

Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.1)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)

Collecting torch_geometric

Downloading torch_geometric-2.7.0-py3-none-any.whl.metadata (63 kB)

Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-packages (from torch_geometric) (3.13.5)

Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch_geometric) (2025.3.0)

Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch_geometric) (3.1.6)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torch_geometric) (2.0.2)

Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from torch_geometric) (5.9.5)

Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dist-packages (from torch_geometric) (3.3.2)

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from torch_geometric) (2.32.4)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from torch_geometric) (4.67.3)

Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from torch_geometric) (3.6.0)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (2.6.1)

Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.4.0)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (26.1.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.8.0)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (6.7.1)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (0.4.1)

Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.23.0)

Requirement already satisfied: idna>=2.0 in /usr/local/lib/python3.12/dist-packages (from yarl<2.0,>=1.17.0->aiohttp->torch_geometric) (3.13)

Requirement already satisfied: typing-extensions>=4.2 in /usr/local/lib/python3.12/dist-packages (from aiosignal>=1.4.0->aiohttp->torch_geometric) (4.15.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch_geometric) (3.0.3)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->torch_geometric) (3.4.7)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->torch_geometric) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->torch_geometric) (2026.4.22)

Downloading torch_geometric-2.7.0-py3-none-any.whl (1.3 MB)

1.3/1.3 MB

8.5 MB/s 0:00:00

Installing collected packages: torch_geometric

Successfully installed torch_geometric-2.7.0

```
[3]: # --- Import libraries ---
import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json
import math

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler,
↳LabelEncoder
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import classification_report, confusion_matrix,
↳accuracy_score, f1_score, log_loss
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, TensorDataset
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence,
↳pad_packed_sequence
import torch_geometric
from torch_geometric.nn import GCNConv, SAGEConv, GATConv, global_mean_pool,
↳global_max_pool
from torch_geometric.loader import DataLoader as PyGDataLoader
from torch_geometric.utils import to_networkx
from torch_geometric.data import Data

import networkx as nx

from tqdm import tqdm
```

1.1.1 Colab Pro

```
[4]: # --- Check GPU availability ---
gpu_info = !nvidia-smi
gpu_info = '\n'.join(gpu_info)
if gpu_info.find('failed') >= 0:
    print('Not connected to a GPU')
else:
```

```
print(gpu_info)
```

Sun May 17 09:49:26 2026

```
+-----+
+-----+
| NVIDIA-SMI 580.82.07                  Driver Version: 580.82.07          CUDA Version:
13.0   |
+-----+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile
Uncorr. ECC |
| Fan   Temp   Perf           Pwr:Usage/Cap |      Memory-Usage | GPU-Util
Compute M. |
|               |              |
MIG M. |
|=====+=====+=====
=====|
|  0  Tesla T4                               Off |  00000000:00:04.0 Off |
0 |
| N/A   40C    P8              10W /  70W |      3MiB /  15360MiB |      0%
Default |
|               |              |
N/A |
+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name
GPU Memory |
|       ID    ID
Usage      |
|=====+=====+=====
=====|
| No running processes found
|
+-----+-----+-----+
+-----+
```

```
[5]: # --- Check RAM availability ---
from psutil import virtual_memory
ram_gb = virtual_memory().total / 1e9
print('Your runtime has {:.1f} gigabytes of available RAM\n'.format(ram_gb))

if ram_gb < 20:
```

```

    print('Not using a high-RAM runtime')
else:
    print('You are using a high-RAM runtime!')

```

Your runtime has 13.6 gigabytes of available RAM

Not using a high-RAM runtime

1.1.2 Paths setup

```

[6]: # --- Mount Google Drive (for Google Colab users) ---
from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

```

[7]: # --- Define Paths ---
group = 'AImSecure'
laboratory = 'Laboratory2'

base_path = '/content/drive/MyDrive/'
project_path = base_path + f'Projects/{group}/{laboratory}/'
data_path = project_path + 'data/'
results_path = project_path + 'results/'

# Ensure directories exist
os.makedirs(project_path, exist_ok=True)
os.makedirs(data_path, exist_ok=True)
os.makedirs(results_path, exist_ok=True)

print(f"Project path: {project_path}")
print(f>Data path: {data_path}")
print(f"Results path: {results_path}")

```

Project path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/
 Data path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/data/
 Results path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/

```

[8]: # --- Set visual style ---
sns.set(style="whitegrid", palette="muted", font_scale=1.1)

def save_plot(fig: plt.Figure, filename: str, path: str = "./plots/", fmt: str =
    ↪ "png", dpi: int = 300, close_fig: bool = False) -> None:
    """
    Save a Matplotlib figure in a specific to a specified directory.

    Args:
        fig (plt.Figure): Matplotlib figure object to save.
    """

```

```

    filename (str): Name of the file to save (e.g., 'plot.png').
    path (str, optional): Directory path to save the figure. Defaults to './
↳ plots/'.
    fmt (str, optional): File format for the saved figure. Defaults to
↳ 'png'.
    dpi (int, optional): Dots per inch for the saved figure. Defaults to
↳ 300.

Returns:
    None
"""
# Ensure the directory exists
os.makedirs(path, exist_ok=True)
save_path = os.path.join(path, f"{filename}.{fmt}")

# Save the figure
fig.savefig(save_path, bbox_inches='tight', pad_inches=0.1, dpi=dpi,
↳ format=fmt)
# plt.close(fig) # Removed to display plots in notebook

if close_fig:
    plt.close(fig)

print(f"Saved plot: {save_path}")

```

```

[9]: def _internal_plot_cm(y_true, y_pred, model_name, save_dir):
    """Internal helper to draw the confusion matrix plot."""
    cm = confusion_matrix(y_true, y_pred)

    # Create figure explicitly
    fig, ax = plt.subplots(figsize=(5, 4))

    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        cbar=False,
        xticklabels=['Benign (0)', 'Malware (1)'],
        yticklabels=['Benign (0)', 'Malware (1)'],
        ax=ax
    )

    ax.set_xlabel('Predicted Label', fontsize=12)
    ax.set_ylabel('True Label', fontsize=12)
    ax.set_title(f'Confusion Matrix: {model_name}', fontsize=14)

```

```

if save_dir:
    clean_name = model_name.lower().replace(" ", "_").replace("-", "_")
    save_plot(
        fig=fig,
        filename=f"cm_{clean_name}",
        path=save_dir,
        fmt="png",
        dpi=300,
        close_fig=False
    )

plt.show()

```

1.2 Task 1 — Frequency-based baseline

We implement a simple frequency-based baseline.

- Transform sequences into feature vectors counting API call occurrences.
- Helps evaluate whether simple approaches already perform well before using complex models.

```

[10]: # Create directory for plots
save_dir = results_path + 'images/' + 'task1_plots/'
os.makedirs(save_dir, exist_ok=True)

```

1.2.1 Frequency-Based Approach

- Extract vocabulary from train and test datasets.
- Use vocabulary to create feature vectors (frequency counts per API call).
- Output dataframe: one row per sequence, one column per API call.

```

[11]: # --- Load dataset and perform initial inspection ---

# Train set
file_path_train = data_path + 'train.json'
df_train = pd.read_json(file_path_train)

# Basic info
print("Train set:")
print("  Shape (raw):", df_train.shape)
print("  Columns:", list(df_train.columns))

# Test set
file_path_test = data_path + 'test.json'
df_test = pd.read_json(file_path_test)

# Basic info
print("\nTest set:")
print("  Shape (raw):", df_test.shape)
print("  Columns:", list(df_test.columns))

```

```
Train set:
  Shape (raw): (16325, 2)
  Columns: ['api_call_sequence', 'is_malware']
```

```
Test set:
  Shape (raw): (6505, 2)
  Columns: ['api_call_sequence', 'is_malware']
```

```
[12]: df_train
```

```
[12]:
```

	api_call_sequence	is_malware
0	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	1
1	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1
2	[FindResourceExW, LoadResource, FindResourceEx...	1
3	[FindResourceExW, LoadResource, FindResourceEx...	1
4	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	1
...	...	...
16320	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	1
16321	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1
16322	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16323	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16324	[SetFilePointer, NtReadFile, SetFilePointer, N...	1

```
[16325 rows x 2 columns]
```

```
[13]: malware_counts = df_train['is_malware'].value_counts()
num_total_samples = len(df_train)

print("Malware vs. Benign Samples in Training Set:")
print(malware_counts)

print("\nPercentages:")
for is_malware, count in malware_counts.items():
    percentage = (count / num_total_samples) * 100
    if is_malware == 1:
        print(f"  Malware (1): {count} samples ({percentage:.2f}%) - {(count / num_total_samples)*100:.2f}% of total")
    else:
        print(f"  Benign (0): {count} samples ({percentage:.2f}%) - {(count / num_total_samples)*100:.2f}% of total")
```

```
Malware vs. Benign Samples in Training Set:
```

```
is_malware
```

```
1    15714
```

```
0      611
```

```
Name: count, dtype: int64
```

```
Percentages:
```

Malware (1): 15714 samples (96.26%) - 96.26% of total
 Benign (0): 611 samples (3.74%) - 3.74% of total

```
[14]: df_test
```

```
[14]:
```

	api_call_sequence	is_malware
0	[NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...	1
1	[LdrGetProcedureAddress, NtClose, NtOpenKey, N...	1
2	[NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...	1
3	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1
4	[NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...	1
...	...	...
6500	[SetErrorMode, NtOpenFile, NtClose, SHGetFolde...	0
6501	[NtProtectVirtualMemory, RegOpenKeyExW, RegQue...	1
6502	[RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...	1
6503	[NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...	1
6504	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1

[6505 rows x 2 columns]

```
[15]: malware_counts = df_test['is_malware'].value_counts()
num_total_samples = len(df_test)

print("Malware vs. Benign Samples in Test Set:")
print(malware_counts)

print("\nPercentages:")
for is_malware, count in malware_counts.items():
    percentage = (count / num_total_samples) * 100
    if is_malware == 1:
        print(f" Malware (1): {count} samples ({percentage:.2f}%) - {(count /
↳ num_total_samples)*100:.2f}% of total")
    else:
        print(f" Benign (0): {count} samples ({percentage:.2f}%) - {(count /
↳ num_total_samples)*100:.2f}% of total")
```

Malware vs. Benign Samples in Test Set:

is_malware

1 6262

0 243

Name: count, dtype: int64

Percentages:

Malware (1): 6262 samples (96.26%) - 96.26% of total

Benign (0): 243 samples (3.74%) - 3.74% of total

1.2.2 Extract the vocabularies

```
[16]: # Extract vocabulary from training set
train_vocab = set(api_call for seq in df_train['api_call_sequence'] for
    ↪ api_call in seq)

# Count unique API calls
print(f"Train: {len(train_vocab)}")
```

Train: 258

```
[17]: # Extract vocabulary from test set
test_vocab = set(api_call for seq in df_test['api_call_sequence'] for api_call
    ↪ in seq)

# Count unique API calls
print(f"Test: {len(test_vocab)}")
```

Test: 232

Q: How many unique API calls does the training set contain? How many in the test set?

- Training set: there are **258** unique API calls.
- Test set: there are **232** unique API calls.

```
[18]: # --- Find API calls only in the training set ---

only_in_train = train_vocab - test_vocab

# Count and print them
print(f"Number of API calls only in training set: {len(only_in_train)}")
print("API calls only in training set:", only_in_train)
```

Number of API calls only in training set: 29

API calls only in training set: {'NtWriteVirtualMemory', 'FindFirstFileExA', 'NtLoadKey', 'RtlRemoveVectoredExceptionHandler', 'InternetOpenUrlW', 'InternetSetStatusCallback', 'SendNotifyMessageW', 'WSASocketW', 'CertOpenSystemStoreW', 'RegDeleteKeyA', 'InternetReadFile', 'ObtainUserAgentString', 'NtDeleteFile', 'NetUserGetLocalGroups', 'listen', 'GetUserNameExA', 'CreateRemoteThread', 'NtReadVirtualMemory', 'GetAddrInfoW', 'GetUserNameExW', 'NetUserGetInfo', 'CryptProtectMemory', 'CryptDecodeObjectEx', 'IWbemServices_ExecMethod', 'recvfrom', 'GetKeyboardState', 'sendto', 'CertOpenStore', 'GetFileInformationByHandleEx'}

```
[19]: # --- Find API calls only in the test set ---

only_in_test = test_vocab - train_vocab
```

```
# Count and print them
print(f"Number of API calls only in test set: {len(only_in_test)}")
print("API calls only in test set:", only_in_test)
```

Number of API calls only in test set: 3

API calls only in test set: {'WSASocketA', 'NtDeleteKey', 'ControlService'}

```
[20]: # Create a boolean series to identify samples in df_test that contain any of
      ↪ these API calls
has_test_only_apis = df_test['api_call_sequence'].apply(
    lambda seq: any(api in only_in_test for api in seq)
)

# Count the samples
samples_with_test_only_apis = has_test_only_apis.sum()

print(f"Number of test set samples with API calls unique to the test set:
      ↪ {samples_with_test_only_apis}")

# Display the actual rows
print("\nTest samples containing API calls unique to the test set:")
display(df_test[has_test_only_apis])
```

Number of test set samples with API calls unique to the test set: 3

Test samples containing API calls unique to the test set:

	api_call_sequence	is_malware
2873	[LdrGetDllHandle, LdrGetProcedureAddress, LdrG...	1
4105	[NtProtectVirtualMemory, CreateThread, NtAlloc...	1
5948	[NtDelayExecution, gethostbyname, NtDelayExecu...	1

Q: Are there any API calls that appear only in the test set (but not in the training set)? If yes, how many? Which ones? Yes — there are API calls that appear only in the test set and not in the training set.

- Number of such API calls: 3
- API calls only in the test set: NtDeleteKey, ControlService, WSASocketA

1.2.3 New frequency-based dataframes

- Create dataframe using training vocabulary as features.
- Count occurrences of each API call per sequence.

```
[21]: # Create frequency-based dataframe for training data
train_df_freq = pd.DataFrame([{api: seq.count(api) for api in train_vocab} for
      ↪ seq in df_train['api_call_sequence']])
```

```
[22]: # Dataframe characteristics
print("Shape (row):", train_df_freq.shape)
print("Columns:", list(train_df_freq.columns))
```

```
Shape (row): (16325, 258)
Columns: ['NtSetContextThread', 'OleInitialize', 'CreateProcessInternalW',
'connect', 'NtEnumerateKey', 'DrawTextExW', 'RtlAddVectoredExceptionHandler',
'getsockname', 'FindFirstFileExA', 'DeleteUrlCacheEntryW', 'InternetOpenUrlW',
'InternetSetStatusCallback', 'CryptDecrypt', 'CryptEncrypt', 'NtOpenMutant',
'GetFileInformationByHandle', 'LdrGetDllHandle', 'WriteConsoleW',
'GetFileVersionInfoExW', 'GetFileAttributesExW', 'RegQueryInfoKeyW',
'ObtainUserAgentString', 'GetSystemTimeAsFileTime', 'GetCursorPos',
'GetFileSizeEx', 'UnhookWindowsHookEx', 'NtOpenThread', 'HttpOpenRequestA',
'HttpSendRequestA', 'FindWindowExA', 'DrawTextExA', 'CopyFileExW',
'FindResourceA', 'MessageBoxTimeoutW', 'InternetCloseHandle',
'GlobalMemoryStatus', 'CreateToolhelp32Snapshot', 'CreateRemoteThread',
'NtEnumerateValueKey', 'NtWriteFile', 'GetFileVersionInfoSizeExW',
'RegCloseKey', 'GetTempPathW', 'gethostbyname', 'Module32FirstW',
'GetAddrInfoW', 'NtCreateFile', 'SetWindowsHookExW', 'InternetCrackUrlW',
'CreateServiceA', 'CryptExportKey', 'NetUserGetInfo', 'setsockopt',
'NtDeviceIoControlFile', 'NtQueryAttributesFile', 'InternetOpenA',
'LdrGetProcedureAddress', 'Module32NextW', 'RegEnumValueW', 'send',
'DnsQuery_A', 'NtQueryDirectoryFile', 'CreateThread', 'LoadStringA',
'Thread32Next', 'GetFileInformationByHandleEx', 'StartServiceW',
'NtTerminateThread', 'CryptCreateHash', 'NtWriteVirtualMemory', 'bind',
'timeGetTime', 'NtLoadKey', 'StartServiceA', 'CoInitializeSecurity',
'NtOpenSection', 'InternetSetOptionA', 'CoInitializeEx', 'DeleteUrlCacheEntryA',
'SendNotifyMessageW', 'GetKeyState', 'WSARecv', 'socket', 'SetErrorMode',
'FindFirstFileExW', 'CryptAcquireContextA', 'WSASocketW', 'InternetCrackUrlA',
'RegDeleteKeyA', 'NtGetContextThread', 'FindWindowA', 'NtDuplicateObject',
'GetAdaptersInfo', 'NetUserGetLocalGroups', 'RegSetValueExW', 'recv',
'CreateJobObjectW', 'RegisterHotKey', 'GetVolumePathNameW', 'GetSystemMetrics',
'GetAdaptersAddresses', 'Thread32First', 'GetUserNameExA',
'GetFileVersionInfoSizeW', 'RegQueryValueExA', 'RemoveDirectoryA',
'NtOpenDirectoryObject', 'RegEnumKeyW', 'NtOpenKeyEx',
'InternetGetConnectedState', 'MoveFileWithProgressW', 'RemoveDirectoryW',
'WriteConsoleA', 'Process32FirstW', 'NtQuerySystemInformation',
'SetFilePointer', '__exception__', 'CreateServiceW', 'NtUnmapViewOfSection',
'recvfrom', 'NtFreeVirtualMemory', 'RegEnumKeyExW', 'NtOpenProcess',
'GetSystemDirectoryW', 'NtOpenFile', 'RegCreateKeyExW',
'IWbemServices_ExecQuery', 'RegEnumKeyExA', 'SetFileTime', 'RegDeleteKeyW',
'SetWindowsHookExA', 'OpenSCManagerA', 'FindResourceExA', 'RegCreateKeyExA',
'NtQueueApcThread', 'SetFilePointerEx', 'InternetQueryOptionA',
'GetComputerNameW', 'SetStdHandle', 'InternetOpenUrlA', 'InternetConnectW',
'RegEnumValueA', 'GetSystemInfo', 'NtProtectVirtualMemory', 'FindResourceW',
'NtSuspendThread', 'RtlRemoveVectoredExceptionHandler', 'SHGetFolderPathW',
'SetEndOfFile', 'RegDeleteValueW', 'InternetConnectA', 'CryptHashData',
'EnumServicesStatusA', 'InternetReadFile', 'RegQueryInfoKeyA',
```

```

'RegDeleteValueA', 'NtCreateThreadEx', 'NtDeleteFile', 'LoadStringW',
'GetDiskFreeSpaceExW', 'RegOpenKeyExW', 'LdrLoadDll',
'GetVolumeNameForVolumeMountPointW', 'CoGetClassObject',
'GetTimeZoneInformation', 'NtQueryInformationFile', 'NtResumeThread',
'NtCreateSection', 'GetFileAttributesW', 'NtTerminateProcess', 'GetFileSize',
'CoCreateInstanceEx', 'listen', 'CreateActCtxW', 'GetSystemDirectoryA',
'closesocket', 'SetUnhandledExceptionFilter', 'GetFileVersionInfoW',
'CreateDirectoryW', 'GetUserNameExW', 'LookupPrivilegeValueW', 'NtSetValueKey',
'OutputDebugStringA', 'IsDebuggerPresent', 'IWbemServices_ExecMethod',
'ShellExecuteExW', 'GetKeyboardState', 'GetForegroundWindow', 'SizeofResource',
'MessageBoxTimeoutA', 'SearchPathW', 'sendto', 'RegSetValueExA', 'DeleteFileW',
'CertOpenStore', 'GetUserNameA', 'OpenSCManagerW', 'NtDelayExecution',
'CopyFileA', 'SetFileAttributesW', 'OpenServiceW', 'NtAllocateVirtualMemory',
'CoCreateInstance', 'LoadResource', 'CryptGenKey', 'InternetOpenW',
'ReadProcessMemory', 'FindWindowExW', 'WriteProcessMemory', 'NtOpenKey',
'EnumWindows', 'LdrUnloadDll', 'select', 'CryptProtectData', 'NtQueryValueKey',
'CertOpenSystemStoreW', 'GetFileType', 'NtMapViewOfSection', 'NtReadFile',
'FindWindowW', 'RegOpenKeyExA', 'HttpQueryInfoA', 'NtClose', 'shutdown',
'SHGetSpecialFolderLocation', 'LookupAccountSidW', 'RegQueryValueExW',
'GlobalMemoryStatusEx', 'Process32NextW', 'GetAsyncKeyState',
'CryptAcquireContextW', 'FindResourceExW', 'CoUninitialize', 'NtCreateKey',
'GetUserNameW', 'GetBestInterfaceEx', 'GetSystemWindowsDirectoryW',
'NtReadVirtualMemory', 'DeviceIoControl', 'OpenServiceA', 'UuidCreate',
'NtSetInformationFile', 'GetComputerNameA', 'GetDiskFreeSpaceW',
'GetNativeSystemInfo', 'CryptProtectMemory', 'ioctlsocket', 'HttpOpenRequestW',
'CryptDecodeObjectEx', 'WSAStartup', 'NtCreateMutant', 'getaddrinfo',
'RtlDecompressBuffer', 'NtQueryKey', 'GetVolumePathNamesForVolumeNameW',
'CopyFileW', 'GetShortPathNameW', 'GetSystemWindowsDirectoryA']

```

```
[23]: train_df_freq.head(10)
```

```

[23]:      NtSetContextThread  OleInitialize  CreateProcessInternalW  connect  \
0                0                1                0                0
1                0                0                0                0
2                0                0                0                0
3                0                0                0                0
4                0                1                0                0
5                0                0                0                0
6                0                0                0                0
7                0                0                0                0
8                0                0                0                0
9                0                0                0                0

      NtEnumerateKey  DrawTextExW  RtlAddVectoredExceptionHandler  getsockname  \
0                0                2                0                0
1                2                0                0                0
2                0                2                0                0

```

3	0	2	0	0
4	0	1	0	0
5	0	2	0	0
6	0	0	0	0
7	0	0	0	0
8	0	2	0	0
9	0	0	0	0

	FindFirstFileExA	DeleteUrlCacheEntryW	...	CryptDecodeObjectEx	\
0	0	0	...	0	
1	0	0	...	0	
2	0	0	...	0	
3	0	0	...	0	
4	0	0	...	0	
5	0	0	...	0	
6	0	0	...	0	
7	0	0	...	0	
8	0	0	...	0	
9	0	0	...	0	

	WSAStartup	NtCreateMutant	getaddrinfo	RtlDecompressBuffer	NtQueryKey	\
0	0	0	0	0	0	
1	0	1	0	0	0	
2	0	0	0	0	0	
3	0	0	0	0	0	
4	0	0	0	0	0	
5	0	0	0	0	0	
6	0	0	0	0	0	
7	0	0	0	0	0	
8	0	0	0	0	0	
9	0	0	0	0	0	

	GetVolumePathNamesForVolumeNameW	CopyFileW	GetShortPathNameW	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	
5	0	0	0	
6	0	0	0	
7	0	0	0	
8	0	0	0	
9	0	0	0	

	GetSystemWindowsDirectoryA
0	0
1	0

2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

[10 rows x 258 columns]

```
[24]: # Create frequency-based dataframe for test data
test_df_freq = pd.DataFrame([{api: seq.count(api) for api in test_vocab} for
    ↪seq in df_test['api_call_sequence']])
```

```
[25]: # Dataframe characteristics
print("Shape (row):", test_df_freq.shape)
print("Columns:", list(test_df_freq.columns))
```

Shape (row): (6505, 232)

Columns: ['NtSetContextThread', 'OleInitialize', 'RtlAddVectoredExceptionHandler', 'CreateProcessInternalW', 'NtEnumerateKey', 'DrawTextExW', 'connect', 'getsockname', 'DeleteUrlCacheEntryW', 'CryptDecrypt', 'CryptEncrypt', 'NtOpenMutant', 'GetFileInformationByHandle', 'LdrGetDllHandle', 'WriteConsoleW', 'GetFileVersionInfoExW', 'GetFileAttributesExW', 'RegQueryInfoKeyW', 'GetSystemTimeAsFileTime', 'GetCursorPos', 'UnhookWindowsHookEx', 'GetFileSizeEx', 'NtOpenThread', 'HttpOpenRequestA', 'HttpSendRequestA', 'FindWindowExA', 'DrawTextExA', 'CopyFileExW', 'FindResourceA', 'MessageBoxTimeoutW', 'InternetCloseHandle', 'GlobalMemoryStatus', 'CreateToolhelp32Snapshot', 'NtEnumerateValueKey', 'NtWriteFile', 'GetFileVersionInfoSizeExW', 'RegCloseKey', 'GetTempPathW', 'gethostbyname', 'Module32FirstW', 'NtCreateFile', 'SetWindowsHookExW', 'CryptExportKey', 'InternetCrackUrlW', 'CreateServiceA', 'setsockopt', 'NtDeviceIoControlFile', 'NtQueryAttributesFile', 'InternetOpenA', 'LdrGetProcedureAddress', 'Module32NextW', 'RegEnumValueW', 'send', 'DnsQuery_A', 'NtQueryDirectoryFile', 'CreateThread', 'LoadStringA', 'Thread32Next', 'StartServiceW', 'NtTerminateThread', 'CryptCreateHash', 'bind', 'timeGetTime', 'StartServiceA', 'CoInitializeSecurity', 'NtOpenSection', 'InternetSetOptionA', 'CoInitializeEx', 'DeleteUrlCacheEntryA', 'GetKeyState', 'WSARecv', 'socket', 'SetErrorMode', 'FindFirstFileExW', 'CryptAcquireContextA', 'InternetCrackUrlA', 'NtGetContextThread', 'FindWindowA', 'NtDuplicateObject', 'GetAdaptersInfo', 'recv', 'RegSetValueExW', 'CreateJobObjectW', 'RegisterHotKey', 'GetVolumePathNameW', 'GetSystemMetrics', 'GetAdaptersAddresses', 'Thread32First', 'GetFileVersionInfoSizeW', 'RegQueryValueExA', 'RemoveDirectoryA', 'NtOpenDirectoryObject', 'RegEnumKeyW', 'NtOpenKeyEx', 'InternetGetConnectedState', 'MoveFileWithProgressW', 'RemoveDirectoryW', 'WriteConsoleA', 'Process32FirstW', 'SetFilePointer', 'NtQuerySystemInformation', '__exception__', 'CreateServiceW',

```

'NtUnmapViewOfSection', 'NtFreeVirtualMemory', 'RegEnumKeyExW', 'NtOpenProcess',
'GetSystemDirectoryW', 'NtOpenFile', 'RegCreateKeyExW', 'RegEnumKeyExA',
'IWbemServices_ExecQuery', 'SetFileTime', 'RegDeleteKeyW', 'SetWindowsHookExA',
'OpenSCManagerA', 'FindResourceExA', 'RegCreateKeyExA', 'NtQueueApcThread',
'SetFilePointerEx', 'InternetQueryOptionA', 'GetComputerNameW', 'SetStdHandle',
'InternetOpenUrlA', 'InternetConnectW', 'RegEnumValueA', 'GetSystemInfo',
'NtProtectVirtualMemory', 'FindResourceW', 'NtSuspendThread',
'SHGetFolderPathW', 'SetEndOfFile', 'RegDeleteValueW', 'InternetConnectA',
'CryptHashData', 'EnumServicesStatusA', 'RegQueryInfoKeyA', 'RegDeleteValueA',
'NtCreateThreadEx', 'LoadStringW', 'GetDiskFreeSpaceExW', 'RegOpenKeyExW',
'LdrLoadDll', 'GetVolumeNameForVolumeMountPointW', 'CoGetClassObject',
'GetTimeZoneInformation', 'NtQueryInformationFile', 'NtResumeThread',
'NtCreateSection', 'GetFileAttributesW', 'CoCreateInstanceEx', 'GetFileSize',
'NtTerminateProcess', 'CreateActCtxW', 'GetSystemDirectoryA', 'closesocket',
'SetUnhandledExceptionFilter', 'GetFileVersionInfoW', 'CreateDirectoryW',
'LookupPrivilegeValueW', 'NtSetValueKey', 'OutputDebugStringA',
'IsDebuggerPresent', 'ShellExecuteExW', 'GetForegroundWindow', 'SizeofResource',
'MessageBoxTimeoutA', 'SearchPathW', 'RegSetValueExA', 'DeleteFileW',
'GetUserNameA', 'OpenSCManagerW', 'NtDelayExecution', 'CopyFileA',
'NtDeleteKey', 'SetFileAttributesW', 'OpenServiceW', 'NtAllocateVirtualMemory',
'ControlService', 'CoCreateInstance', 'LoadResource', 'CryptGenKey',
'InternetOpenW', 'ReadProcessMemory', 'WriteProcessMemory', 'FindWindowExW',
'NtOpenKey', 'EnumWindows', 'LdrUnloadDll', 'select', 'CryptProtectData',
'NtQueryValueKey', 'GetFileType', 'NtMapViewOfSection', 'NtReadFile',
'FindWindowW', 'RegOpenKeyExA', 'HttpQueryInfoA', 'NtClose', 'WSASocketA',
'shutdown', 'SHGetSpecialFolderLocation', 'LookupAccountSidW',
'RegQueryValueExW', 'Process32NextW', 'GlobalMemoryStatusEx',
'GetAsyncKeyState', 'CryptAcquireContextW', 'FindResourceExW',
'GetBestInterfaceEx', 'NtCreateKey', 'GetUserNameW', 'CoUninitialize',
'GetSystemWindowsDirectoryW', 'DeviceIoControl', 'OpenServiceA', 'UuidCreate',
'NtSetInformationFile', 'GetComputerNameA', 'GetDiskFreeSpaceW',
'GetNativeSystemInfo', 'HttpOpenRequestW', 'ioctlsocket', 'WSAStartup',
'NtCreateMutant', 'getaddrinfo', 'RtlDecompressBuffer',
'GetVolumePathNamesForVolumeNameW', 'NtQueryKey', 'CopyFileW',
'GetShortPathNameW', 'GetSystemWindowsDirectoryA']

```

```
[26]: test_df_freq.head(10)
```

```

[26]:   NtSetContextThread  OleInitialize  RtlAddVectoredExceptionHandler  \
0                0                0                0
1                0                0                0
2                0                1                0
3                0                0                0
4                0                0                0
5                0                0                0
6                0                0                0
7                0                0                0

```

8	0	0	0
9	0	0	0

	CreateProcessInternalW	NtEnumerateKey	DrawTextExW	connect	getsockname	\
0	0	0	2	0	0	
1	0	0	1	0	0	
2	0	0	2	0	0	
3	0	0	4	0	0	
4	0	0	0	0	0	
5	0	0	0	0	0	
6	0	0	0	0	0	
7	0	0	0	0	0	
8	0	0	0	0	0	
9	0	0	0	0	0	

	DeleteUrlCacheEntryW	CryptDecrypt	...	ioctlsocket	WSAStartup	\
0	0	0	...	0	0	
1	0	0	...	0	0	
2	0	0	...	0	0	
3	0	1	...	0	1	
4	0	0	...	0	0	
5	0	0	...	0	0	
6	0	0	...	0	0	
7	0	0	...	0	0	
8	0	1	...	0	1	
9	0	0	...	0	0	

	NtCreateMutant	getaddrinfo	RtlDecompressBuffer	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	
5	1	0	0	
6	0	0	0	
7	0	0	0	
8	0	0	0	
9	0	0	0	

	GetVolumePathNamesForVolumeNameW	NtQueryKey	CopyFileW	GetShortPathNameW	\
0		0	0	0	
1		0	0	0	
2		0	0	0	
3		0	0	0	
4		0	0	0	
5		0	0	0	
6		0	0	0	

7	0	0	0	0
8	0	0	0	0
9	0	0	0	0

	GetSystemWindowsDirectoryA
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

[10 rows x 232 columns]

Q: Can you use the test vocabulary to build the new test dataframe? If not, how do you handle API calls in the test set that do not exist in the training vocabulary? No, you cannot use the test vocabulary to build the test dataframe.

Machine learning models require the feature space (columns) during inference (testing) to be identical to the feature space used during training.

Common strategies for handling API calls in the test set that did not exist in the training vocabulary include:

1. **Add an “Unknown” Column:** Map all unseen API calls to a single generic token (e.g., <UNK>) or a specific “is_unknown” feature. This preserves the information that *an* action occurred, even if the specific action is unrecognized.
2. **Ignore Them:** Simply discard or zero-out any API calls that were not seen during training.

Decision: In this specific case, since there are **very few unseen API calls** (i.e. *only 3* samples) in the test set, we decided to **ignore them**. This approach simplifies the pipeline without significantly impacting the model’s performance.

Sparse Vectors Analysis

```
[27]: # --- Non-zero elements analysis for training data ---

# Calculate average non-zero elements per row
avg_nonzero_train = (train_df_freq != 0).sum(axis=1).mean()
print(f"Average non-zero elements per row in train_df_freq: {avg_nonzero_train:.2f}")

# Calculate ratio of non-zero elements
ratio_nonzero_train = avg_nonzero_train / train_df_freq.shape[1]
```

```
print(f"Ratio of non-zero elements per row in train_df_freq:␣  
↪{ratio_nonzero_train:.4f}")
```

Average non-zero elements per row in train_df_freq: 21.95

Ratio of non-zero elements per row in train_df_freq: 0.0851

```
[28]: # --- Non-zero elements analysis for test data ---  
  
# Calculate average non-zero elements per row  
avg_nonzero_test = (test_df_freq != 0).sum(axis=1).mean()  
print(f"Average non-zero elements per row in test_df_freq: {avg_nonzero_test:.  
↪2f}")  
  
# Calculate ratio of non-zero elements  
ratio_nonzero_test = avg_nonzero_test / test_df_freq.shape[1]  
print(f"Ratio of non-zero elements per row in test_df_freq: {ratio_nonzero_test:  
↪.4f}")
```

Average non-zero elements per row in test_df_freq: 24.28

Ratio of non-zero elements per row in test_df_freq: 0.1046

Q: How many non-zero elements per row do you have on average in the training set? How many in the test set? What is the ratio with respect to the number of elements per row? On average:

- **Training set:** ~22 non-zero elements per row, which is **8.5%** of all elements.
- **Test set:** ~24 non-zero elements per row, which is **10.5%** of all elements.

Q: The original API sequences were ordered. Is it still the case now in the frequency-based dataframe? Why? No, the original order of API calls is **not preserved** in the frequency-based dataframe.

This is because the frequency-based representation **counts how many times each API call occurs** in a sequence, without keeping track of their positions. Each row now simply contains **one column per API call in the vocabulary**, with the value representing its count, so the sequential order is lost.

This approach converts sequences into fixed-dimensional vectors, simplifying the computation but discarding temporal information. As a result, complex behavioral patterns based on the chronology of calls cannot be captured by this representation.

1.2.4 Feed the frequency-based datasets to a classifier

- Any classifier can be used (shallow/deep neural or non-neural).
- Goal: evaluate baseline performance on sparse vectors without sequence information.

Training

```
[29]: # --- Training with a classifier ---
```

```

# Split data into features and target
X = train_df_freq
y = df_train['is_malware']

# Create a validation set from training set
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)

# Construct test set with features aligned to train_vocab
# TODO: justify this strategy in the report (too few samples)
X_test = pd.DataFrame([{'api': seq.count(api) for api in train_vocab} for seq in
    ↪df_test['api_call_sequence']])
y_test = df_test['is_malware']

# Train a simple classifier (using default hyperparameters for a simple
    ↪baseline)
clf = LogisticRegression(random_state=42, solver='liblinear') # 'liblinear'
    ↪solver is good for small datasets and sparse data
clf.fit(X_train, y_train)

# Compute training and validation loss
y_train_proba = clf.predict_proba(X_train)
y_val_proba = clf.predict_proba(X_val)

print(f"Training log-loss: {log_loss(y_train, y_train_proba):.4f}")
print(f"Validation log-loss: {log_loss(y_val, y_val_proba):.4f}")

```

Training log-loss: 0.0806
Validation log-loss: 0.1224

Evaluation

```

[30]: def evaluate_model(model, X, y_true, model_name: str = "Unnamed model"):
    """
    Evaluate a trained scikit-learn model on a given dataset and return the
    ↪classification report.
    Handles missing predicted classes gracefully (zero_division=0).
    """
    # Get predictions
    y_pred = model.predict(X)

    # Identify missing classes (not predicted at all)
    missing_classes = set(np.unique(y_true)) - set(np.unique(y_pred))
    if missing_classes:
        missing_classes = [int(x) for x in sorted(missing_classes)]
        print(f"Warning: {model_name} made no predictions for classes:
            ↪{missing_classes}")

```

```

# Generate classification report
report = classification_report(y_true, y_pred, digits=4, zero_division=0)

return report

```

```

[31]: def plot_cm_task1(model, X_test, y_test, model_name, save_dir=None):
    """
    Specific for Task 1 (Scikit-Learn models).
    Generates predictions using .predict() and then plots.
    """
    # 1. Get predictions (Scikit-learn models work directly on X_test)
    y_pred = model.predict(X_test)

    # 2. Use the core helper to plot
    # Note: y_test might need to be converted to a simple list/array if it's a
    ↪ pandas Series
    _internal_plot_cm(y_test, y_pred, model_name, save_dir)

```

```

[32]: # --- Evaluate validation and test set and print classification reports ---

print(f"\nValidation classification report:\n")
report = evaluate_model(clf, X_val, y_val, "Logistic Regression (Validation)")
print(report)

print(f"\nTest classification report:\n")
report = evaluate_model(clf, X_test, y_test, "Logistic Regression (Test)")
print(report)

```

Validation classification report:

	precision	recall	f1-score	support
0	0.6750	0.3320	0.4451	244
1	0.9746	0.9938	0.9841	6286
accuracy			0.9691	6530
macro avg	0.8248	0.6629	0.7146	6530
weighted avg	0.9634	0.9691	0.9639	6530

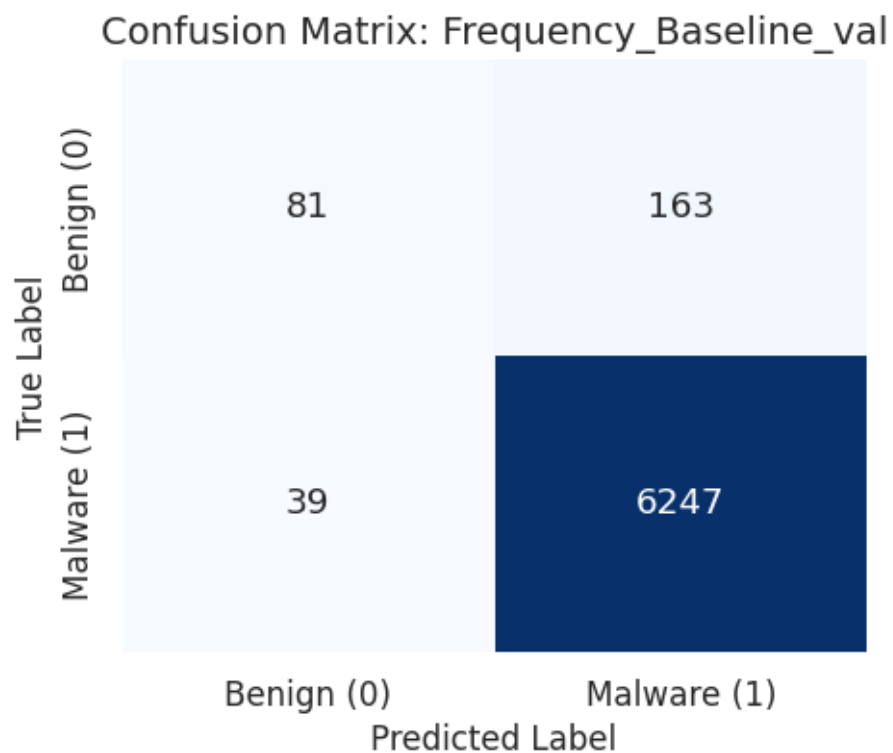
Test classification report:

	precision	recall	f1-score	support
0	0.6885	0.3457	0.4603	243
1	0.9751	0.9939	0.9844	6262

accuracy			0.9697	6505
macro avg	0.8318	0.6698	0.7223	6505
weighted avg	0.9644	0.9697	0.9648	6505

```
[33]: plot_cm_task1(
    model=clf,
    X_test=X_val,
    y_test=y_val,
    model_name="Frequency_Baseline_val",
    save_dir=results_path + 'images/task1_plots/'
)
```

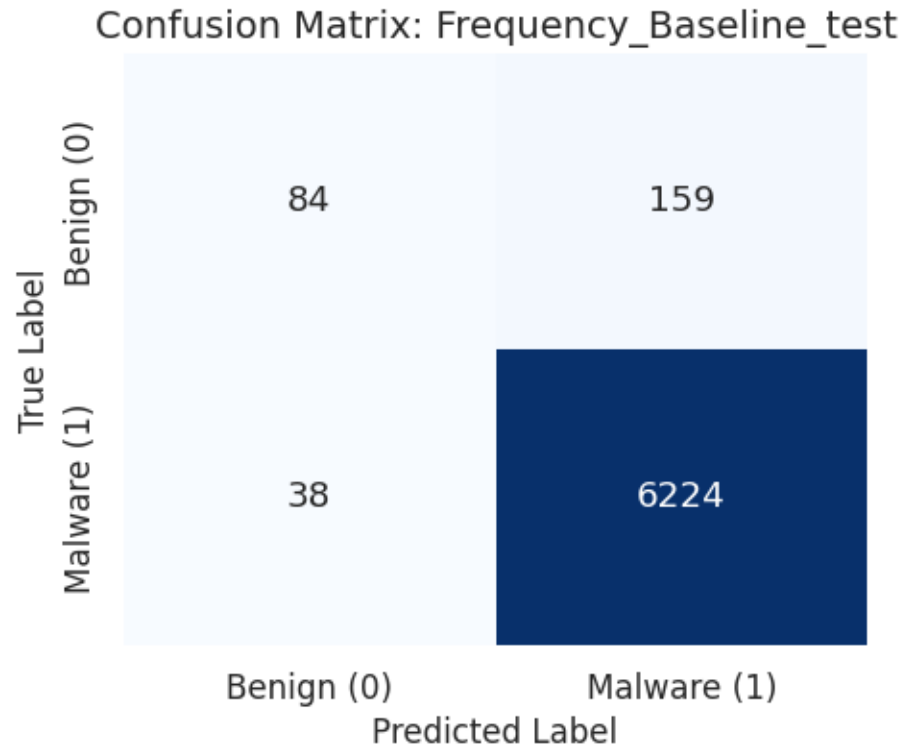
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task1_plots/cm_frequency_baseline_val.png



```
[34]: plot_cm_task1(
    model=clf,
    X_test=X_test,
    y_test=y_test,
    model_name="Frequency_Baseline_test",
    save_dir=results_path + 'images/task1_plots/'
)
```

```
)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task1_plots/cm_frequency_baseline_test.png



Q: Report how you chose the hyperparameters of your classifier, and the final performance on the test set. In Machine Learning problems, it is always good practice to compare against baseline solutions.

We used a simple logistic regression as a baseline classifier. The hyperparameters were mostly left at their defaults. We specifically chose:

- `solver='liblinear'` → suitable for small and sparse datasets
- `random_state=42` → ensures reproducibility.

No additional hyperparameter tuning (like regularization strength C) was performed, since this was intended as a simple baseline.

Final performance on the test set: - the classifier strongly predicts the malware class, but is weaker on the minority class (benign samples).

Q: Is the final performance good, even ignoring the order of API calls and handling very sparse vectors? Yes, the performance is **good for a baseline frequency-based model**, given that: - The **order of API calls is ignored**, so any sequential patterns are lost. - The feature vectors are **very sparse** (only ~8-10% non-zero elements per row).

Despite these limitations, the model achieves **high accuracy and excellent detection of malware**, which indicates that the presence/frequency of certain API calls alone is highly informative for this classification task.

Although the model performs well (especially in detecting malware), an analysis of the metrics by class reveals significant difficulty in correctly identifying benign samples. The low recall for class 0 suggests that, although API frequency is informative, data imbalance and loss of order prevent the model from generalizing well to the minority class.

1.3 Task 2 - Feed Forward Neural Network (FFNN)

text

```
[35]: # Create directory for plots
save_dir = results_path + 'images/' + 'task2_plots/'
os.makedirs(save_dir, exist_ok=True)
```

1.3.1 API Call Statistics

```
[36]: # --- Compute the length (number of API calls) per sequence ---
df_train['seq_length'] = df_train['api_call_sequence'].apply(len)
df_test['seq_length'] = df_test['api_call_sequence'].apply(len)

# --- Basic descriptive statistics ---
print("Train set sequence length stats:")
print(df_train['seq_length'].describe(), "\n")

print("Test set sequence length stats:")
print(df_test['seq_length'].describe(), "\n")

# --- Check if all sequences have the same length ---
same_length_train = df_train['seq_length'].nunique() == 1
same_length_test = df_test['seq_length'].nunique() == 1
print(f"All train sequences same length? {same_length_train}")
print(f"All test sequences same length? {same_length_test}\n")
```

Train set sequence length stats:

count	16325.000000
mean	75.030812
std	8.951411
min	60.000000
25%	67.000000
50%	75.000000
75%	83.000000
max	90.000000

Name: seq_length, dtype: float64

Test set sequence length stats:

count	6505.000000
-------	-------------

```

mean      86.331130
std       9.113511
min       70.000000
25%      79.000000
50%      87.000000
75%      94.000000
max      100.000000
Name: seq_length, dtype: float64

```

```

All train sequences same length? False
All test sequences same length? False

```

```
[37]: df_train
```

```

[37]:          api_call_sequence  is_malware  \
0      [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...      1
1      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
2      [FindResourceExW, LoadResource, FindResourceEx...      1
3      [FindResourceExW, LoadResource, FindResourceEx...      1
4      [LdrGetProcedureAddress, SetErrorMode, LdrLoad...      1
...      ...      ...
16320  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...      1
16321  [NtClose, LdrGetProcedureAddress, CryptCreateH...      1
16322  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16323  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16324  [SetFilePointer, NtReadFile, SetFilePointer, N...      1

      seq_length
0              73
1              88
2              79
3              71
4              63
...      ...
16320          64
16321          78
16322          84
16323          62
16324          77

```

```
[16325 rows x 3 columns]
```

```

[38]: # --- Compare distributions visually ---

plt.figure(figsize=(6, 4))
plt.hist(df_train['seq_length'], bins=50, alpha=0.6, label='Train',
        density=True)

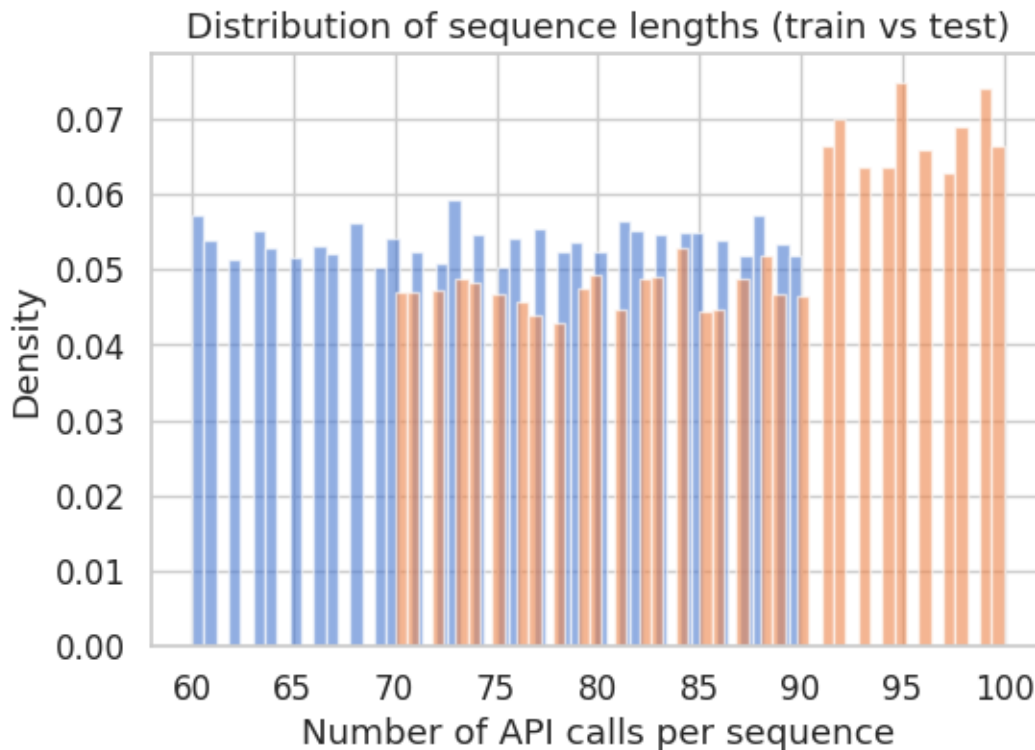
```

```
plt.hist(df_test['seq_length'], bins=50, alpha=0.6, label='Test', density=True)
plt.xlabel("Number of API calls per sequence")
plt.ylabel("Density")
plt.title("Distribution of sequence lengths (train vs test)")

# Save the plot to the specified path
save_plot(plt.gcf(), f"distribution_sequence_lengths_train_test", save_dir)

plt.show()
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/distribution_sequence_lengths_train_test.png



Q: Do you have the same number of API calls per sequence? If not, is the distribution of API calls per sequence the same for training and test sets? No, the sequences do not all have the same length in either the training or test sets.

- **Training set:** sequence lengths range from 60 to 90 API calls (mean = 75).
- **Test set:** sequence lengths range from 70 to 100 API calls (mean = 86).

The **sequences in the test set** are not only longer on average, but also fall within a range (90–100) that is completely absent from the training set. This discrepancy means that the model will have to generalize to sequence lengths it has never encountered during training, posing an additional challenge for sequential architectures.

Q: Can a FFNN handle a variable number of elements? If not, why? No, a feedforward neural network (FFNN) **cannot directly handle a variable number** of elements per input.

- FFNNs require a **fixed-size input vector**, because each input neuron corresponds to a specific feature, and the weight matrix has a fixed shape.
- If the number of elements (features) changes between samples, the network **cannot align inputs with the learned weights**, so it won't work properly.

Due to this architectural constraint, in order to process variable-length API sequences using a FFNN, they must be converted into fixed-length vectors using padding (to extend short sequences) or truncation (to shorten longer ones).

1.3.2 Fixed-Size Sequences

Q: How to estimate a fixed-size candidate? Which partition do you use to estimate it? To estimate a fixed-size input for an FFNN, it's possible to choose a representative sequence length from the training set only, since the model should not use information from the test set.

Common strategies include: 1. **Maximum length**: ensures all training sequences fit, but can be too large and waste memory. 2. **Median length**: gives a typical sequence size, but some longer sequences will need truncation. 3. **75th percentile length**: a compromise that covers most sequences without being overly large.

Among these strategies, we opted for the maximum training set length (90). This choice is justified by the fact that the distribution of lengths (as seen earlier) is fairly uniform and does not contain any extreme outliers; therefore, the computational “cost” of using 90 as a fixed size is offset by the advantage of not having to truncate (and thus lose) information in any training sequence.

```
[39]: # Compute sequence lengths in training set
seq_lengths = df_train['api_call_sequence'].apply(len)

# Option 1: maximum length
max_len = seq_lengths.max()
print(f"Max length: {max_len}")

# Option 2: median
median_len = int(seq_lengths.median())
print(f"Median length: {median_len}")

# Option 3: 75th percentile
p75_len = int(np.percentile(seq_lengths, 75))
print(f"75th percentile length: {p75_len}")
```

Max length: 90

Median length: 75

75th percentile length: 83

Q: Given the estimate, what technique could you use to obtain the same number of API calls per sequence? To obtain the same number of API calls per sequence, the main strategies are:

1. **Padding:** Add “dummy” or special tokens to sequences **shorter than the fixed length** so they reach the target size.
2. **Truncation:** Cut sequences **longer than the fixed length** to fit the target size.

By combining padding and truncation based on the estimated fixed length, you can ensure that **all sequences have the same number of elements**, which is required for an FFNN.

```
[40]: def pad_truncate_sequence(seq, fixed_len, pad_value=0):
    """
    Convert a variable-length sequence into a fixed-length sequence.

    Args:
        seq (list): original sequence (list of API calls)
        fixed_len (int): target length L
        pad_value: value to use for padding (0 is common for integers)

    Returns:
        list of length fixed_len
    """
    if len(seq) < fixed_len:
        # Pad at the end
        seq_padded = seq + [pad_value] * (fixed_len - len(seq))
    else:
        # Truncate at the end
        seq_padded = seq[:fixed_len]
    return seq_padded
```

```
[41]: # --- Perform padding or truncating on the training data ---

L = max_len # chosen fixed size

# Apply the padding or truncating to all the dataframe
df_train['seq_fixed'] = df_train['api_call_sequence'].apply(lambda x: pad_truncate_sequence(x, L))
```

```
[42]: # --- Inspect the train set (e.g. the first row) ---

print("First row of the train set before padding:")
print(df_train['api_call_sequence'].iloc[0])
print(f"Lenght of the sequence: {len(df_train['api_call_sequence'].iloc[0])}")
    ↪ # should be less than L

print("\nFirst row of the train set after padding:")
print(df_train['seq_fixed'].iloc[0])
print(f"Lenght of the sequence: {len(df_train['seq_fixed'].iloc[0])}") # should
    ↪ be exactly L
```

First row of the train set before padding:
 ['LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrLoadDll',

```

'LdrGetProcedureAddress', 'LdrGetDllHandle', 'LdrGetProcedureAddress',
'GetTimeZoneInformation', 'LoadStringW', 'RegOpenKeyExW', 'RegQueryValueExW',
'RegCloseKey', 'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 'SetErrorMode', 'LdrLoadDll', 'SetErrorMode',
'LdrLoadDll', 'SetErrorMode', 'GetSystemMetrics', 'FindResourceExW',
'OleInitialize', 'FindResourceExW', 'LoadResource', 'FindResourceExW',
'LoadResource', 'NtClose', 'NtAllocateVirtualMemory', 'GetSystemMetrics',
'NtAllocateVirtualMemory', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LookupAccountSidW', 'LdrGetProcedureAddress', 'LookupAccountSidW',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrGetDllHandle', 'FindResourceExW',
'LoadResource', 'FindResourceExW', 'LoadResource', 'DrawTextExW',
'GetSystemMetrics', 'FindResourceExW', 'LoadResource', 'GetSystemMetrics',
'DrawTextExW', 'LdrGetDllHandle', 'FindResourceExW', 'LoadResource',
'FindResourceExW', 'LoadResource', 'FindResourceExW', 'LoadResource',
'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LoadStringW', 'LdrGetDllHandle',
'LdrGetProcedureAddress']

```

Lenght of the sequence: 73

First row of the train set after padding:

```

['LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrGetDllHandle', 'LdrGetProcedureAddress',
'GetTimeZoneInformation', 'LoadStringW', 'RegOpenKeyExW', 'RegQueryValueExW',
'RegCloseKey', 'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 'SetErrorMode', 'LdrLoadDll', 'SetErrorMode',
'LdrLoadDll', 'SetErrorMode', 'GetSystemMetrics', 'FindResourceExW',
'OleInitialize', 'FindResourceExW', 'LoadResource', 'FindResourceExW',
'LoadResource', 'NtClose', 'NtAllocateVirtualMemory', 'GetSystemMetrics',
'NtAllocateVirtualMemory', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LookupAccountSidW', 'LdrGetProcedureAddress', 'LookupAccountSidW',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrGetDllHandle', 'FindResourceExW',
'LoadResource', 'FindResourceExW', 'LoadResource', 'DrawTextExW',
'GetSystemMetrics', 'FindResourceExW', 'LoadResource', 'GetSystemMetrics',
'DrawTextExW', 'LdrGetDllHandle', 'FindResourceExW', 'LoadResource',
'FindResourceExW', 'LoadResource', 'FindResourceExW', 'LoadResource',
'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LoadStringW', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

```

Lenght of the sequence: 90

Q: If at test time you have more API calls than the fixed-size, what do you do with the exceeding API calls? At test time, if a sequence has **more API calls than the fixed size**, the exceeding API calls should be **truncated** to fit the fixed length. Sequences shorter

than the fixed size are **padded** with dummy tokens.

We have decided to perform the same pad or truncate action on the test set, so that all sequences have a consistent length compatible with the FFNN.

```
[43]: # To ensure consistency, we apply the padding or truncating to also on the test
      ↪df
      df_test['seq_fixed'] = df_test['api_call_sequence'].apply(lambda x:
      ↪pad_truncate_sequence(x, L))
```

```
[44]: df_train
```

```
[44]:
```

		api_call_sequence	is_malware	\
0		[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	1	
1		[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1	
2		[FindResourceExW, LoadResource, FindResourceEx...	1	
3		[FindResourceExW, LoadResource, FindResourceEx...	1	
4		[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	1	
...		...	...	
16320		[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	1	
16321		[NtClose, LdrGetProcedureAddress, CryptCreateH...	1	
16322		[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1	
16323		[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1	
16324		[SetFilePointer, NtReadFile, SetFilePointer, N...	1	

	seq_length		seq_fixed
0	73	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	
1	88	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	
2	79	[FindResourceExW, LoadResource, FindResourceEx...	
3	71	[FindResourceExW, LoadResource, FindResourceEx...	
4	63	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	
...	...	...	...
16320	64	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	
16321	78	[NtClose, LdrGetProcedureAddress, CryptCreateH...	
16322	84	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	
16323	62	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	
16324	77	[SetFilePointer, NtReadFile, SetFilePointer, N...	

[16325 rows x 4 columns]

```
[45]: df_test
```

```
[45]:
```

		api_call_sequence	is_malware	\
0		[NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...	1	
1		[LdrGetProcedureAddress, NtClose, NtOpenKey, N...	1	
2		[NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...	1	
3		[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1	
4		[NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...	1	

		...	...	
6500	[SetErrorMode, NtOpenFile, NtClose, SHGetFolde...			0
6501	[NtProtectVirtualMemory, RegOpenKeyExW, RegQue...			1
6502	[RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...			1
6503	[NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...			1
6504	[NtClose, LdrGetProcedureAddress, CryptCreateH...			1

	seq_length		seq_fixed
0	98	[NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...	
1	98	[LdrGetProcedureAddress, NtClose, NtOpenKey, N...	
2	99	[NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...	
3	85	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	
4	80	[NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...	

	...		...
6500	75	[SetErrorMode, NtOpenFile, NtClose, SHGetFolde...	
6501	83	[NtProtectVirtualMemory, RegOpenKeyExW, RegQue...	
6502	70	[RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...	
6503	96	[NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...	
6504	78	[NtClose, LdrGetProcedureAddress, CryptCreateH...	

[6505 rows x 4 columns]

1.3.3 Handling Categorical Features

```
[46]: # --- Build a unified vocabulary ---
```

```
all_sequences = df_train['seq_fixed']

# Flatten all sequences into a single list of API calls
all_api_calls = [api for seq in all_sequences for api in seq]
```

```
[47]: # --- Build vocabulary with padding handling ---
```

```
# 1. Filter strings only. We ignore 0 here because we will manually assign it
↳ later.
unique_api_calls = sorted(list(set(api for api in all_api_calls if
↳ isinstance(api, str))))

# 2. Define Special Tokens
PAD_IDX = 0 # <PAD>
UNK_IDX = 1 # <UNK>

# 3. Map API call -> sequential ID (Start from 2 to save space for PAD and UNK)
api_to_id = {api: i + 2 for i, api in enumerate(unique_api_calls)}

# 4. Add the special tokens to the dictionary
# (Optional: helpful if you need to reverse look up '<PAD>' later)
```

```

api_to_id['<PAD>'] = PAD_IDX
api_to_id['<UNK>'] = UNK_IDX

# Map sequential ID -> API call (for verification/debugging)
id_to_api = {i: api for api, i in api_to_id.items()}

print(f"Number of unique API calls (including special tokens): {len(api_to_id)}")
print("\nExample mapping (API -> ID):", list(api_to_id.items())[:5])
print("\nExample mapping (ID -> API):", list(id_to_api.items())[:5])

```

Number of unique API calls (including special tokens): 260

Example mapping (API -> ID): [('CertOpenStore', 2), ('CertOpenSystemStoreW', 3), ('CoCreateInstance', 4), ('CoCreateInstanceEx', 5), ('CoGetClassObject', 6)]

Example mapping (ID -> API): [(2, 'CertOpenStore'), (3, 'CertOpenSystemStoreW'), (4, 'CoCreateInstance'), (5, 'CoCreateInstanceEx'), (6, 'CoGetClassObject')]

Sequential Identifiers Approach

```

[48]: def sequence_to_ids(seq, mapping, pad_idx, unk_idx):
    """
    Convert a sequence of API calls to their corresponding IDs.
    - If item is 0 (integer padding), map to pad_idx.
    - If item is string but not in mapping, map to unk_idx.
    """
    ids = []
    for item in seq:
        # Case A: It is existing padding (integer 0)
        if item == 0:
            ids.append(pad_idx)
        # Case B: It is a known string API
        elif item in mapping:
            ids.append(mapping[item])
        # Case C: It is an unknown string API (Test set OOV)
        else:
            ids.append(unk_idx)
    return ids

```

```

[49]: # --- Convert sequences to sequential IDs ---

# Apply conversion to train and test sets
# Note: We pass UNK_IDX (1) so unknown calls in Test set become 1.
df_train['seq_sequential_ids'] = df_train['seq_fixed'].apply(
    lambda seq: sequence_to_ids(seq, api_to_id, PAD_IDX, UNK_IDX)
)

```

```

df_test['seq_sequential_ids'] = df_test['seq_fixed'].apply(
    lambda seq: sequence_to_ids(seq, api_to_id, PAD_IDX, UNK_IDX)
)

# Quick check
print("\nSequential IDs - first row of train set:")
print(df_train['seq_sequential_ids'].iloc[0])
print("Length of the sequence:", len(df_train['seq_sequential_ids'].iloc[0]))

print("\nSequential IDs - first row of test set:")
print(df_test['seq_sequential_ids'].iloc[0])
print("Length of the sequence:", len(df_test['seq_sequential_ids'].iloc[0]))

# Verify that the logic works (check for any None values)
# If this prints nothing, your data is clean!
for seq in df_test['seq_sequential_ids']:
    if None in seq:
        print("Error: Found None in sequence! Check mapping.")
        break

```

Sequential IDs - first row of train set:

```

[117, 118, 119, 118, 117, 118, 86, 123, 201, 205, 188, 117, 118, 117, 118, 219,
119, 219, 119, 219, 81, 46, 179, 46, 121, 46, 121, 134, 133, 81, 133, 119, 118,
119, 118, 119, 118, 119, 118, 119, 118, 124, 118, 124, 119, 118, 119, 118, 119,
118, 117, 46, 121, 46, 121, 39, 81, 46, 121, 81, 39, 117, 46, 121, 46, 121, 46,
121, 117, 118, 123, 117, 118, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Length of the sequence: 90

```

Sequential IDs - first row of test set:

```

[164, 134, 152, 164, 134, 159, 122, 133, 122, 117, 118, 117, 118, 117, 122, 118,
81, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46,
121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 118, 46, 121, 46, 121, 133,
81, 134, 81, 133, 119, 118, 117, 46, 121, 46, 121, 39, 81, 46, 121, 81, 39, 117,
46, 121, 46, 121, 46, 121, 117, 118, 122, 117, 118, 117, 118, 81, 44, 121, 229,
133, 119, 118, 84]
Length of the sequence: 90

```

[50]: df_train

```

[50]:
      api_call_sequence  is_malware  \
0  [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...      1
1  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
2  [FindResourceExW, LoadResource, FindResourceEx...      1
3  [FindResourceExW, LoadResource, FindResourceEx...      1
4  [LdrGetProcedureAddress, SetErrorMode, LdrLoad...      1
...
16320 [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...      1

```

16321	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1
16322	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16323	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16324	[SetFilePointer, NtReadFile, SetFilePointer, N...	1

	seq_length	seq_fixed \
0	73	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...
1	88	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...
2	79	[FindResourceExW, LoadResource, FindResourceEx...
3	71	[FindResourceExW, LoadResource, FindResourceEx...
4	63	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...
...	...	...
16320	64	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...
16321	78	[NtClose, LdrGetProcedureAddress, CryptCreateH...
16322	84	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...
16323	62	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...
16324	77	[SetFilePointer, NtReadFile, SetFilePointer, N...

	seq_sequential_ids
0	[117, 118, 119, 118, 117, 118, 86, 123, 201, 2...
1	[133, 119, 118, 82, 80, 137, 82, 152, 153, 163...
2	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
3	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
4	[118, 219, 119, 219, 81, 46, 121, 46, 121, 46,...
...	...
16320	[118, 119, 118, 119, 118, 119, 118, 119, 118, ...
16321	[134, 118, 24, 118, 30, 118, 135, 66, 221, 166...
16322	[118, 117, 118, 81, 216, 119, 118, 46, 121, 46...
16323	[118, 117, 118, 117, 118, 117, 118, 117, 118, ...
16324	[221, 166, 221, 166, 221, 45, 121, 133, 146, 1...

[16325 rows x 5 columns]

[51]: df_test

	api_call_sequence	is_malware \
0	[NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...	1
1	[LdrGetProcedureAddress, NtClose, NtOpenKey, N...	1
2	[NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...	1
3	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1
4	[NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...	1
...	...	...
6500	[SetErrorMode, NtOpenFile, NtClose, SHGetFolde...	0
6501	[NtProtectVirtualMemory, RegOpenKeyExW, RegQue...	1
6502	[RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...	1
6503	[NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...	1
6504	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1

```

seq_length                                seq_fixed \
0      98 [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...
1      98 [LdrGetProcedureAddress, NtClose, NtOpenKey, N...
2      99 [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...
3      85 [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...
4      80 [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...
...      ...
6500    75 [SetErrorMode, NtOpenFile, NtClose, SHGetFolde...
6501    83 [NtProtectVirtualMemory, RegOpenKeyExW, RegQue...
6502    70 [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...
6503    96 [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...
6504    78 [NtClose, LdrGetProcedureAddress, CryptCreateH...

                                seq_sequential_ids
0      [164, 134, 152, 164, 134, 159, 122, 133, 122, ...
1      [118, 134, 152, 164, 134, 152, 164, 134, 119, ...
2      [152, 164, 134, 152, 164, 134, 117, 118, 117, ...
3      [133, 119, 118, 119, 118, 82, 200, 204, 188, 1...
4      [152, 164, 134, 118, 24, 118, 30, 118, 135, 66...
...      ...
6500    [219, 151, 134, 214, 219, 63, 219, 151, 134, 1...
6501    [158, 201, 205, 188, 140, 133, 158, 133, 146, ...
6502    [201, 205, 188, 201, 205, 188, 80, 117, 118, 2...
6503    [164, 134, 200, 204, 188, 153, 164, 134, 119, ...
6504    [134, 118, 24, 118, 30, 118, 135, 66, 221, 166...

[6505 rows x 5 columns]

```

Learnable Embeddings Approach

```

[52]: # The learnable embeddings approach is implemented in the first layer of the
      ↪neural network model.
      # The nn.Embedding layer in the MalwareFFNN class handles this process.

```

1.3.4 FFNN Training

```

[53]: class MalwareFFNN(nn.Module):
      def __init__(self, seq_len, hidden_dims=[128,64,32],
                    dropout_rates=None, batchnorm_flags=None,
                    use_embedding=True, vocab_size=None, embedding_dim=16,
                    dropout_embedding_rate=0.1):
          """
          Flexible FFNN for malware classification.

          Parameters:
          - seq_len: length of each API call sequence

```

```

        - vocab_size: number of unique API calls (required if
↪use_embedding=True)
        - hidden_dims: list of hidden layer sizes
        - embedding_dim: size of embedding vectors (only used if
↪use_embedding=True)
        - use_embedding: whether to use an embedding layer (True) or raw
↪sequential IDs / one-hot (False)
    """
    super(MalwareFFNN, self).__init__()
    self.use_embedding = use_embedding

    if self.use_embedding:
        if vocab_size is None:
            raise ValueError("vocab_size must be provided if
↪use_embedding=True")

        # Embedding layer for sequential IDs
        self.embedding = nn.Embedding(num_embeddings=vocab_size,
↪embedding_dim=embedding_dim, padding_idx=0)
        input_dim = seq_len * embedding_dim

        self.dropout_embed = nn.Dropout(dropout_embedding_rate)
    else:
        # Input will be one-hot encoded or numeric vector
        input_dim = seq_len # assumes IDs are already numeric

    # Ensure defaults if none are provided
    if dropout_rates is None:
        dropout_rates = [0.0] * len(hidden_dims) # no dropout by default
    if batchnorm_flags is None:
        batchnorm_flags = [False] * len(hidden_dims) # no batchnorm by
↪default

    # Safety check for lengths
    assert len(dropout_rates) == len(hidden_dims), \
        "dropout_rates length must match hidden_dims length"
    assert len(batchnorm_flags) == len(hidden_dims), \
        "batchnorm_flags length must match hidden_dims length"

    # Build hidden layers
    layers = []
    prev_dim = input_dim
    for h_dim, dropout_rate, use_bn in zip(hidden_dims, dropout_rates,
↪batchnorm_flags):
        layers.append(nn.Linear(prev_dim, h_dim))
        if use_bn:

```

```

        layers.append(nn.BatchNorm1d(h_dim))
    layers.append(nn.ReLU())
    if dropout_rate > 0:
        layers.append(nn.Dropout(dropout_rate))
    prev_dim = h_dim
    self.ffnn = nn.Sequential(*layers)

    # Output layer: binary classification
    self.output_layer = nn.Linear(prev_dim, 1)

def forward(self, x):
    if self.use_embedding:
        # x: [batch_size, seq_len] of IDs
        x = self.embedding(x)      # [batch_size, seq_len, embedding_dim]
        x = self.dropout_embed(x)
        x = x.view(x.size(0), -1) # flatten: [batch_size, ↵
↵seq_len*embedding_dim]
    else:
        # x: [batch_size, seq_len] or [batch_size, seq_len*vocab_size] if ↵
↵one-hot flattened
        x = x.float()

    x = self.ffnn(x)
    out = self.output_layer(x) # [batch_size, 1]
    return out

```

```

[54]: # --- Training function with early stopping ---

def train_model(model, train_loader, val_loader, epochs, optimizer, criterion, ↵
↵min_delta=None, patience=None, scheduler=None):
    """
    Train a PyTorch model with early stopping and metric tracking.
    Returns: model, history dictionary (losses + metrics), and elapsed time.
    """

    # --- DEVICE DETECTION ---
    device = next(model.parameters()).device

    # --- HISTORY STORAGE ---
    history = {
        'train_loss': [],
        'val_loss': [],
        'val_f1_macro': [],
        'val_f1_weighted': [],
    }

    best_val_loss = float('inf')

```

```

best_model_state = None
trigger_times = 0

start_time = time.time()

for epoch in range(epochs):
    # --- TRAINING ---
    model.train()
    batch_losses = []
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        batch_losses.append(loss.item())
    train_loss = np.mean(batch_losses) if batch_losses else 0.0

    # --- VALIDATION & METRICS ---
    model.eval()
    val_batch_losses = []
    all_preds = []
    all_targets = []

    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            val_batch_losses.append(loss.item())

        # Convert logits to probabilities and then binary predictions
        ↪ (0 or 1)

        # Assumes Binary Classification (BCEWithLogitsLoss)
        probs = torch.sigmoid(outputs)
        preds = (probs > 0.5).float()

        # Store for epoch-level metric calculation
        all_preds.extend(preds.cpu().numpy())
        all_targets.extend(y_batch.cpu().numpy())

    val_loss = np.mean(val_batch_losses) if val_batch_losses else 0.0

    # Calculate Metrics for this Epoch
    epoch_f1_macro = f1_score(all_targets, all_preds, average='macro', ↪
    ↪ zero_division=0)

```

```

        epoch_f1_weighted = f1_score(all_targets, all_preds,
↪average='weighted', zero_division=0)

        # Update History
        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['val_f1_macro'].append(epoch_f1_macro)
        history['val_f1_weighted'].append(epoch_f1_weighted)

        # --- SCHEDULER STEP ---
        if scheduler is not None:
            last_lr = optimizer.param_groups[0]['lr']
            if isinstance(scheduler, torch.optim.lr_scheduler.
↪ReduceLROnPlateau):
                scheduler.step(val_loss)
            else:
                scheduler.step()
            curr_lr = optimizer.param_groups[0]['lr']
            if curr_lr != last_lr:
                print(f"Epoch {epoch}: LR reduced from {last_lr:.6f} to
↪{curr_lr:.6f}")

        # --- EARLY STOPPING ---
        if min_delta is not None:
            if val_loss < best_val_loss - min_delta:
                best_val_loss = val_loss
                # Save best model state
                best_model_state = {k: v.cpu().clone() for k, v in model.
↪state_dict().items()}
                trigger_times = 0
            else:
                trigger_times += 1
                if trigger_times >= patience:
                    print(f"Early stopping at epoch {epoch+1} (best val loss:
↪{best_val_loss:.6f})")
                    break

        if (epoch+1) % 5 == 0 or epoch == 0 or epoch == epochs:
            print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_loss:.4f},
↪Val Loss: {val_loss:.4f}, Val F1(W): {epoch_f1_weighted:.4f}")

        # Restore best model after training loop completes
        if best_model_state is not None:
            model.load_state_dict(best_model_state)
            model.to(device)

```

```

elapsed = time.time() - start_time
elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
print(f'\nTraining completed in {elapsed_str} (hh:mm:ss) - {elapsed:.2f}␣
↪seconds')

return model, history, elapsed

```

```

[55]: def evaluate_model(model, X_tensor, y_true, model_name: str = "Unnamed model"):
    """
    Evaluate a trained model on a given dataset and return the classification␣
    ↪report.

    Handles missing predicted classes gracefully (zero_division=0) and
    reports which classes were not predicted, along with the model/config name.
    """
    model.eval()
    with torch.no_grad():
        outputs = model(X_tensor)
        probs = torch.sigmoid(outputs)
        y_pred = (probs > 0.5).long().cpu().numpy().flatten() # Binary␣
        ↪prediction (0 or 1)

    # Convert y_true to numpy if it's a tensor
    if isinstance(y_true, torch.Tensor):
        y_true = y_true.cpu().numpy().flatten()

    # Identify missing classes (not predicted at all)
    missing_classes = set(np.unique(y_true)) - set(np.unique(y_pred))
    if missing_classes:
        missing_classes = [int(x) for x in sorted(missing_classes)]
        print(f"Warning: {model_name} made no predictions for classes:␣
        ↪{missing_classes}")

    # Generate classification report without raising warnings
    report = classification_report(y_true, y_pred, digits=4, zero_division=0)

    return report

```

```

[56]: def plot_cm_task2(model, X_tensor, y_tensor, model_name, save_dir=None):
    """
    Specific for FFNN. Takes tensors, moves to device, predicts, and plots.
    """
    # Ensure model is in eval mode
    model.eval()

    # Detect device (cuda or cpu) from the model parameters
    device = next(model.parameters()).device

```

```

# Move inputs to the same device as model
X_input = X_tensor.to(device)

with torch.no_grad():
    # Get logits
    outputs = model(X_input)
    # Apply sigmoid to get probabilities
    probs = torch.sigmoid(outputs)
    # Convert to binary predictions (0 or 1)
    y_pred = (probs > 0.5).long().cpu().numpy().flatten()

# Ensure y_true is on CPU and numpy
y_true = y_tensor.cpu().numpy().flatten()

# Plot
_internal_plot_cm(y_true, y_pred, model_name, save_dir)

```

```

[57]: def plot_training_metrics(histories, model_names=None, save_dir=None,
    ↪ plot_name="validation_f1_macro_comparison"):
    """
    Compare multiple models' validation F1 Macro across epochs.
    Only shows validation F1 macro (no weighted, no markers for cleaner
    ↪ plots).
    """
    # Normalize input
    if isinstance(histories, dict):
        hist_map = histories
        names = [f"lr_{k:.0e}" if isinstance(k, float) else str(k) for k in
    ↪ hist_map.keys()]
        hist_map = dict(zip(names, hist_map.values()))
    else:
        if model_names is None:
            names = [f"Model_{i+1}" for i in range(len(histories))]
        else:
            names = list(model_names)
            hist_map = dict(zip(names, histories))

    # Validate
    for name, h in hist_map.items():
        if 'val_f1_macro' not in h:
            raise KeyError(f"History for '{name}' must contain 'val_f1_macro'."
    ↪)

    colors = ['#E74C3C', '#F39C12', '#27AE60', '#3498DB', '#9B59B6',
    ↪ '#34495E']

```

```

fig, ax = plt.subplots(figsize=(10, 5))

for idx, (name, h) in enumerate(hist_map.items()):
    epochs = range(1, len(h['val_f1_macro']) + 1)
    color = colors[idx % len(colors)]
    ax.plot(epochs, h['val_f1_macro'], label=name, linewidth=2,
    ↪color=color)

ax.set_title("Validation F1 Macro Comparison", fontsize=13)
ax.set_xlabel("Epochs", fontsize=11)
ax.set_ylabel("F1 Macro", fontsize=11)
ax.grid(True, alpha=0.3)
ax.legend(fontsize=9)
plt.tight_layout()

if save_dir:
    save_plot(fig, filename=plot_name, path=save_dir, fmt="png",
    ↪dpi=300, close_fig=False)
plt.show()

```

```

[58]: # --- Plotting function for LR comparison ---
def plot_lr_comparison(histories_dict, model_name, save_dir=None):
    """
    Plot ONLY validation losses for different learning rates.
    (When multiple LRs are present, we show only val curves for readability.)
    """
    plt.style.use('seaborn-v0_8-paper')

    fig, ax = plt.subplots(figsize=(8, 5))

    colors = ['#E74C3C', '#F39C12', '#27AE60', '#3498DB', '#9B59B6',
    ↪'#34495E']

    for idx, (lr, history) in enumerate(sorted(histories_dict.items(),
    ↪reverse=True)):
        color = colors[idx % len(colors)]
        label = f'lr_{lr:.0e}'
        ax.plot(history['val_loss'], color=color, linewidth=2, label=label,
        ↪alpha=0.8)

    ax.set_title(f'{model_name} - Validation Loss (LR Comparison)',
    ↪fontsize=13, pad=10)
    ax.set_ylabel('Validation Loss', fontsize=10)
    ax.set_xlabel('Epochs', fontsize=10)
    ax.grid(True, linestyle='--', alpha=0.3)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

```

```

ax.legend(fontsize=9, loc='best')

plt.tight_layout()

if save_dir:
    save_plot(fig, f'{model_name.lower().replace(" ", "_")}_lr_comparison', save_dir)

plt.show()

```

[59]: # --- Plot Train vs Val ---

```

def plot_train_val_grid(histories_dict, model_name, save_dir=None):
    """
    Plots a grid (1 row, N columns) showing Train vs Val loss for EACH learning
    ↪rate.
    This helps diagnose overfitting/underfitting for each specific setting.
    Works for both FFNN (Task 2) and RNN (Task 3) models.

    Args:
        histories_dict: Dictionary with LR values as keys and history dicts as
        ↪values
        model_name: Name of the model architecture
        save_dir: Directory to save the plot
    """
    lrs = sorted(histories_dict.keys(), reverse=True) # Highest LR first
    num_lrs = len(lrs)

    # Set style
    plt.style.use('seaborn-v0_8-paper')

    # Create figure with 1 row and N columns
    fig, axes = plt.subplots(1, num_lrs, figsize=(5 * num_lrs, 4), sharey=True)

    # Handle case if there is only 1 LR (axes is not a list)
    if num_lrs == 1:
        axes = [axes]

    # Common colors
    col_train = '#2980B9' # Blue
    col_val = '#E74C3C' # Red

    for idx, lr in enumerate(lrs):
        ax = axes[idx]
        history = histories_dict[lr]
        epochs = range(1, len(history['train_loss']) + 1)

```

```

    # Plot Lines
    ax.plot(epochs, history['train_loss'], label='Train Loss',
    ↪color=col_train, linewidth=2)
    ax.plot(epochs, history['val_loss'], label='Val Loss', color=col_val,
    ↪linewidth=2)

    # Find best epoch (min val loss)
    min_val_loss = min(history['val_loss'])
    best_epoch = history['val_loss'].index(min_val_loss) + 1

    # Mark best epoch
    ax.axvline(x=best_epoch, color='green', linestyle=':', alpha=0.6,
    ↪label=f'Best Ep: {best_epoch}')
    ax.scatter(best_epoch, min_val_loss, color='green', zorder=5)

    # Styling
    ax.set_title(f'LR: {lr:.0e}', fontsize=12, fontweight='bold')
    ax.set_xlabel('Epochs')
    if idx == 0:
        ax.set_ylabel('Loss')

    ax.legend(fontsize=9)
    ax.grid(True, linestyle='-', alpha=0.3)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

    fig.suptitle(f'{model_name} - Overfitting Diagnosis (Train vs Val)',
    ↪fontsize=14, y=1.05)
    plt.tight_layout()

    if save_dir:
        save_plot(fig, f'{model_name.lower().replace(" ", "_")}_diagnosis',
    ↪save_dir)

    plt.show()

```

Training with Sequential Identifiers Approach

```

[60]: # --- Create a validation set ---

# Convert sequences to a numpy array (shape: [samples, seq_len])
X_train = np.array(df_train['seq_sequential_ids'].tolist())
X_test = np.array(df_test['seq_sequential_ids'].tolist())

# Labels
y_train = df_train['is_malware'].values
y_test = df_test['is_malware'].values

```

```

# Split train into train + val
X_train_split, X_val, y_train_split, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42, stratify=y_train
)

print("Train set:", X_train_split.shape, y_train_split.shape)
print("Validation set:", X_val.shape, y_val.shape)

```

Train set: (13060, 90) (13060,)
Validation set: (3265, 90) (3265,)

```

[61]: # --- Prepare PyTorch datasets and loaders ---

# Convert to tensors
X_train_tensor = torch.tensor(X_train_split.astype(np.int64), dtype=torch.long)
y_train_tensor = torch.tensor(y_train_split, dtype=torch.float32).unsqueeze(1)

X_val_tensor = torch.tensor(X_val.astype(np.int64), dtype=torch.long)
y_val_tensor = torch.tensor(y_val, dtype=torch.float32).unsqueeze(1)

X_test_tensor = torch.tensor(X_test.astype(np.int64), dtype=torch.long)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

# Create datasets and loaders
batch_size = 64

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=True)

```

```

[62]: unique, counts = np.unique(y_train_split, return_counts=True)
print(dict(zip(unique, counts)))

```

{np.int64(0): np.int64(489), np.int64(1): np.int64(12571)}

```

[63]: def compute_pos_weight(y):
    """
    Compute positive class weight for BCEWithLogitsLoss.
    Returns a tensor > 1 if positive class is minority.
    """
    y_np = np.array(y, dtype=int)
    class_counts = np.bincount(y_np)

```

```

if len(class_counts) < 2:
    raise ValueError("Dataset must contain both classes 0 and 1.")

count_neg, count_pos = class_counts[0], class_counts[1]

# if 1 = malware (majority class)
ratio = count_neg / count_pos

# Optionally stabilize the weight
ratio = max(ratio, 0.2)

print(f"Negatives: {count_neg}, Positives: {count_pos}")
print(f"pos_weight (applied to y=1 samples): {ratio:.4f}")

# return torch.tensor([ratio], dtype=torch.float32)
return torch.tensor([ratio], dtype=torch.float32)

```

```

[64]: # --- Configuration Constants ---
seq_len = X_train.shape[1]
hidden_dims = [256, 128, 64]
dropout_rates = [0.4, 0.35, 0.3]
batchnorm_flags = [True, True, True]
min_delta = 0.00001
patience_early_stop = 15
epochs = 100

# --- Device Setup ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Compute weights and move to device
pos_weight = compute_pos_weight(y_train_split).to(device)

# --- Define LR Candidates ---
learning_rates_seq = [0.0001, 0.00005, 0.00001]
histories_seq = {} # Store results here
models_seq = {} # Store models here
elapsed_seq_times = {} # Store elapsed times here

print(f"\n{'='*80}")
print("Training FFNN (Sequential IDs) with different learning rates")
print(f"{'='*80}")

for lr in learning_rates_seq:
    print(f"\n--- Training FFNN (Sequential IDs) with lr={lr:.0e} ---")

    # 1. Re-Initialize Model and move to device

```

```

model_seq = MalwareFFNN(
    seq_len,
    hidden_dims,
    dropout_rates,
    batchnorm_flags,
    use_embedding=False,
    dropout_embedding_rate=0.1
).to(device)

# 2. Re-Initialize Criterion
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

# 3. Initialize Optimizer with current LR
optimizer = optim.AdamW(model_seq.parameters(), lr=lr, weight_decay=1e-5)

# 4. Initialize Scheduler
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.7,
    patience=10,
    min_lr=1e-6
)

# 5. Train
model_seq, history, elapsed_seq = train_model(
    model_seq,
    train_loader,
    val_loader,
    epochs,
    optimizer,
    criterion,
    min_delta,
    patience=patience_early_stop,
    scheduler=scheduler
)

# Store model and history with numeric LR key
models_seq[lr] = model_seq
histories_seq[lr] = history
elapsed_seq_times[lr] = elapsed_seq

elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_seq))
print(f"Completed. Training time: {elapsed_str} ({elapsed_seq:.2f}s) -_
↪Final val loss: {history['val_loss'][-1]:.4f}")

```

Using device: cuda

Negatives: 489, Positives: 12571

pos_weight (applied to y=1 samples): 0.2000

```
=====
Training FFNN (Sequential IDs) with different learning rates
=====
```

--- Training FFNN (Sequential IDs) with lr=1e-04 ---

Epoch 1/100 - Train Loss: 0.1253, Val Loss: 0.1273, Val F1(W): 0.9425
Epoch 5/100 - Train Loss: 0.0996, Val Loss: 0.1037, Val F1(W): 0.9431
Epoch 10/100 - Train Loss: 0.0907, Val Loss: 0.0999, Val F1(W): 0.9375
Epoch 15/100 - Train Loss: 0.0854, Val Loss: 0.0961, Val F1(W): 0.9294
Epoch 20/100 - Train Loss: 0.0828, Val Loss: 0.0929, Val F1(W): 0.9234
Epoch 25/100 - Train Loss: 0.0845, Val Loss: 0.0927, Val F1(W): 0.9232
Epoch 30/100 - Train Loss: 0.0772, Val Loss: 0.0922, Val F1(W): 0.9218
Epoch 35/100 - Train Loss: 0.0733, Val Loss: 0.0928, Val F1(W): 0.9193
Epoch 37: LR reduced from 0.000100 to 0.000070
Epoch 40/100 - Train Loss: 0.0741, Val Loss: 0.0940, Val F1(W): 0.9148
Early stopping at epoch 42 (best val loss: 0.091365)

Training completed in 00:00:33 (hh:mm:ss) - 33.04 seconds

Completed. Training time: 00:00:33 (33.04s) - Final val loss: 0.0959

--- Training FFNN (Sequential IDs) with lr=5e-05 ---

Epoch 1/100 - Train Loss: 0.1570, Val Loss: 0.1495, Val F1(W): 0.8276
Epoch 5/100 - Train Loss: 0.1114, Val Loss: 0.1209, Val F1(W): 0.9414
Epoch 10/100 - Train Loss: 0.1005, Val Loss: 0.1055, Val F1(W): 0.9450
Epoch 15/100 - Train Loss: 0.0930, Val Loss: 0.0990, Val F1(W): 0.9416
Epoch 20/100 - Train Loss: 0.0906, Val Loss: 0.0973, Val F1(W): 0.9361
Epoch 25/100 - Train Loss: 0.0897, Val Loss: 0.0957, Val F1(W): 0.9308
Epoch 30/100 - Train Loss: 0.0854, Val Loss: 0.1101, Val F1(W): 0.9283
Epoch 35/100 - Train Loss: 0.0858, Val Loss: 0.0953, Val F1(W): 0.9244
Epoch 40/100 - Train Loss: 0.0832, Val Loss: 0.0935, Val F1(W): 0.9260
Epoch 45/100 - Train Loss: 0.0810, Val Loss: 0.0947, Val F1(W): 0.9177
Epoch 50/100 - Train Loss: 0.0778, Val Loss: 0.0940, Val F1(W): 0.9200
Epoch 52: LR reduced from 0.000050 to 0.000035
Epoch 55/100 - Train Loss: 0.0780, Val Loss: 0.0934, Val F1(W): 0.9163
Epoch 60/100 - Train Loss: 0.0783, Val Loss: 0.0967, Val F1(W): 0.9195
Epoch 64: LR reduced from 0.000035 to 0.000024
Epoch 65/100 - Train Loss: 0.0769, Val Loss: 0.0933, Val F1(W): 0.9175
Early stopping at epoch 69 (best val loss: 0.091826)

Training completed in 00:00:52 (hh:mm:ss) - 52.92 seconds

Completed. Training time: 00:00:52 (52.92s) - Final val loss: 0.0950

--- Training FFNN (Sequential IDs) with lr=1e-05 ---

Epoch 1/100 - Train Loss: 0.1529, Val Loss: 0.1552, Val F1(W): 0.7490
Epoch 5/100 - Train Loss: 0.1408, Val Loss: 0.1428, Val F1(W): 0.8888
Epoch 10/100 - Train Loss: 0.1270, Val Loss: 0.1348, Val F1(W): 0.9326

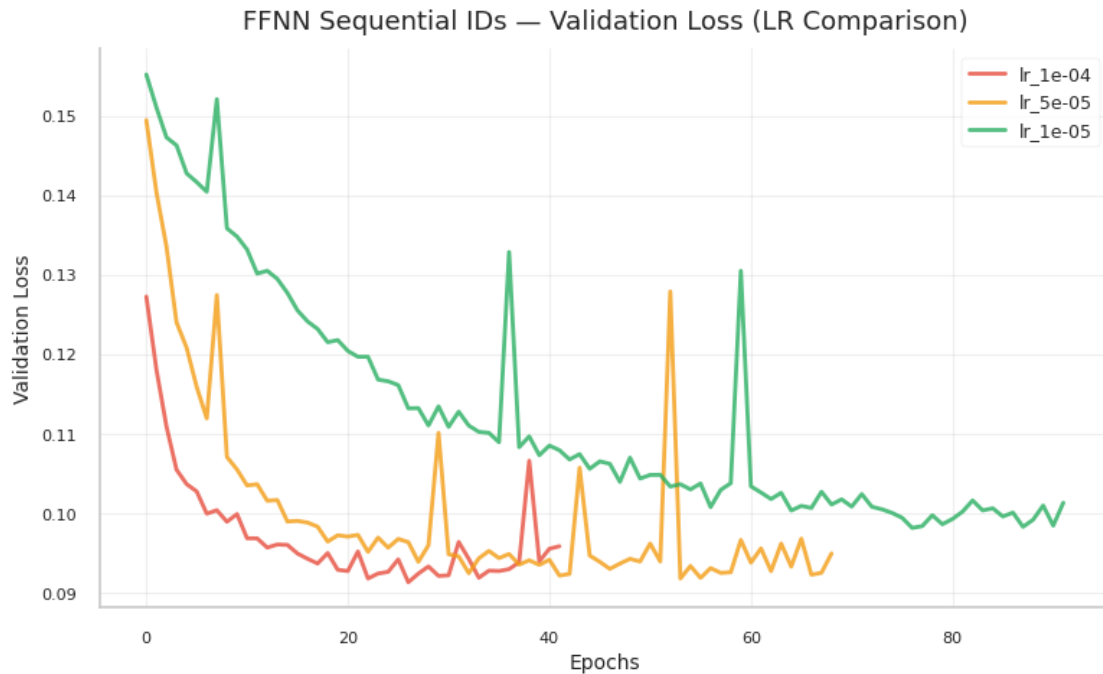
Epoch 15/100 - Train Loss: 0.1168, Val Loss: 0.1278, Val F1(W): 0.9408
Epoch 20/100 - Train Loss: 0.1097, Val Loss: 0.1218, Val F1(W): 0.9432
Epoch 25/100 - Train Loss: 0.1055, Val Loss: 0.1166, Val F1(W): 0.9440
Epoch 30/100 - Train Loss: 0.1023, Val Loss: 0.1135, Val F1(W): 0.9448
Epoch 35/100 - Train Loss: 0.1009, Val Loss: 0.1102, Val F1(W): 0.9447
Epoch 40/100 - Train Loss: 0.0993, Val Loss: 0.1073, Val F1(W): 0.9447
Epoch 45/100 - Train Loss: 0.0972, Val Loss: 0.1056, Val F1(W): 0.9450
Epoch 50/100 - Train Loss: 0.0970, Val Loss: 0.1044, Val F1(W): 0.9440
Epoch 55/100 - Train Loss: 0.0967, Val Loss: 0.1030, Val F1(W): 0.9435
Epoch 60/100 - Train Loss: 0.0943, Val Loss: 0.1305, Val F1(W): 0.9448
Epoch 65/100 - Train Loss: 0.0940, Val Loss: 0.1004, Val F1(W): 0.9441
Epoch 70/100 - Train Loss: 0.0927, Val Loss: 0.1018, Val F1(W): 0.9401
Epoch 75/100 - Train Loss: 0.0929, Val Loss: 0.1001, Val F1(W): 0.9416
Epoch 80/100 - Train Loss: 0.0921, Val Loss: 0.0987, Val F1(W): 0.9418
Epoch 85/100 - Train Loss: 0.0912, Val Loss: 0.1007, Val F1(W): 0.9395
Epoch 87: LR reduced from 0.000010 to 0.000007
Epoch 90/100 - Train Loss: 0.0912, Val Loss: 0.1010, Val F1(W): 0.9378
Early stopping at epoch 92 (best val loss: 0.098200)

Training completed in 00:01:11 (hh:mm:ss) - 71.12 seconds

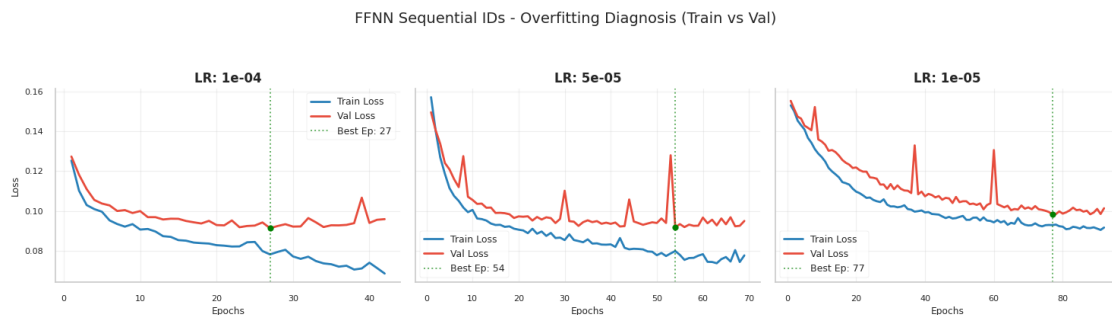
Completed. Training time: 00:01:11 (71.12s) - Final val loss: 0.1014

```
[65]: # --- Plot loss curves ---  
  
# Plot comparison  
plot_lr_comparison(histories_seq, "FFNN Sequential IDs", save_dir)  
  
# Plot Train vs Validation  
plot_train_val_grid(histories_seq, "FFNN Sequential IDs", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_sequential_ids_lr_comparison.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_sequential_ids_diagnosis.png



```
[66]: # --- Evaluate validation and test sets for ALL learning rates ---
```

```
print("\n" + "=" * 80)
print("Evaluation for ALL FFNN (Sequential IDs) learning rates")
print("=" * 80)

for lr in learning_rates_seq:
    model = models_seq[lr]

    print(f"\n{'='*60}")
```

```

print(f"FFNN (Sequential IDs) with lr={lr:.0e}")
print(f"{'='*60}")

# Validation evaluation
print(f'\nValidation report (FFNN Sequential IDs - lr={lr:.0e}):')
report = evaluate_model(model, X_val_tensor.to(device), y_val) # Moved
↪X_val_tensor to device
print(report)

# Test evaluation
print(f'Test report (FFNN Sequential IDs - lr={lr:.0e}):')
report_test = evaluate_model(model, X_test_tensor.to(device), y_test) #
↪Moved X_test_tensor to device
print(report_test)

# Find and highlight best model
best_lr_seq = min(histories_seq.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f" Best FFNN (Sequential IDs) - LR: {best_lr_seq[0]:.0e} (Min Val Loss:
↪{min(best_lr_seq[1]['val_loss']):.4f})")
print(f"{'='*60}")

```

```

=====
Evaluation for ALL FFNN (Sequential IDs) learning rates
=====

```

```

=====
FFNN (Sequential IDs) with lr=1e-04
=====

```

Validation report (FFNN Sequential IDs - lr=1e-04):

	precision	recall	f1-score	support
0	0.1158	0.1803	0.1410	122
1	0.9675	0.9465	0.9569	3143
accuracy			0.9179	3265
macro avg	0.5416	0.5634	0.5490	3265
weighted avg	0.9357	0.9179	0.9264	3265

Test report (FFNN Sequential IDs - lr=1e-04):

	precision	recall	f1-score	support
0	0.1199	0.1317	0.1255	243
1	0.9662	0.9625	0.9643	6262

accuracy			0.9314	6505
macro avg	0.5430	0.5471	0.5449	6505
weighted avg	0.9346	0.9314	0.9330	6505

```
=====
FFNN (Sequential IDs) with lr=5e-05
=====
```

Validation report (FFNN Sequential IDs - lr=5e-05):

	precision	recall	f1-score	support
0	0.1082	0.2049	0.1416	122
1	0.9680	0.9345	0.9509	3143

accuracy			0.9072	3265
macro avg	0.5381	0.5697	0.5463	3265
weighted avg	0.9359	0.9072	0.9207	3265

Test report (FFNN Sequential IDs - lr=5e-05):

	precision	recall	f1-score	support
0	0.1184	0.1852	0.1445	243
1	0.9677	0.9465	0.9570	6262

accuracy			0.9181	6505
macro avg	0.5430	0.5658	0.5507	6505
weighted avg	0.9359	0.9181	0.9266	6505

```
=====
FFNN (Sequential IDs) with lr=1e-05
=====
```

Validation report (FFNN Sequential IDs - lr=1e-05):

	precision	recall	f1-score	support
0	0.1754	0.0820	0.1117	122
1	0.9651	0.9850	0.9750	3143

accuracy			0.9513	3265
macro avg	0.5703	0.5335	0.5433	3265
weighted avg	0.9356	0.9513	0.9427	3265

Test report (FFNN Sequential IDs - lr=1e-05):

	precision	recall	f1-score	support
0	0.2807	0.0658	0.1067	243

1	0.9648	0.9935	0.9789	6262
accuracy			0.9588	6505
macro avg	0.6227	0.5296	0.5428	6505
weighted avg	0.9392	0.9588	0.9463	6505

```
=====
Best FFNN (Sequential IDs) - LR: 1e-04 (Min Val Loss: 0.0914)
=====
```

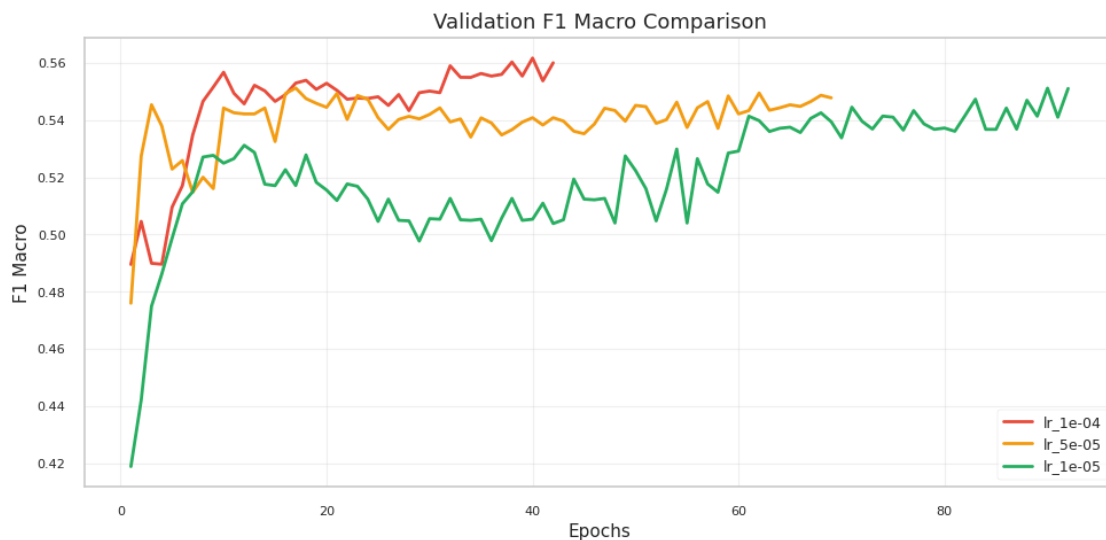
```
[67]: # --- Plot metric curves for ALL learning rates (Sequential IDs) ---
```

```
print("\n" + "=" * 80)
print("Metric Curves for ALL FFNN (Sequential IDs) learning rates")
print("=" * 80)

plot_training_metrics(histories_seq, save_dir=save_dir,
    ↪plot_name="ffnn_sequential_ids_f1_comparison")
```

```
=====
Metric Curves for ALL FFNN (Sequential IDs) learning rates
=====
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_sequential_ids_f1_comparison.png
```



```
[68]: # --- Plot confusion matrices for ALL learning rates (Sequential IDs) -
    ↪VALIDATION ---
```

```

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Sequential IDs) learning rates -_
↳VALIDATION")
print("=" * 80)

for lr in learning_rates_seq:
    model = models_seq[lr]
    print(f"\nGenerating confusion matrix for FFNN (Sequential IDs) with lr={lr:
↳.0e}...")

    plot_cm_task2(model=model, X_tensor=X_val_tensor, y_tensor=y_val_tensor,
↳model_name=f"FFNN Sequential IDs (lr={lr:.0e})", save_dir=save_dir)

```

```

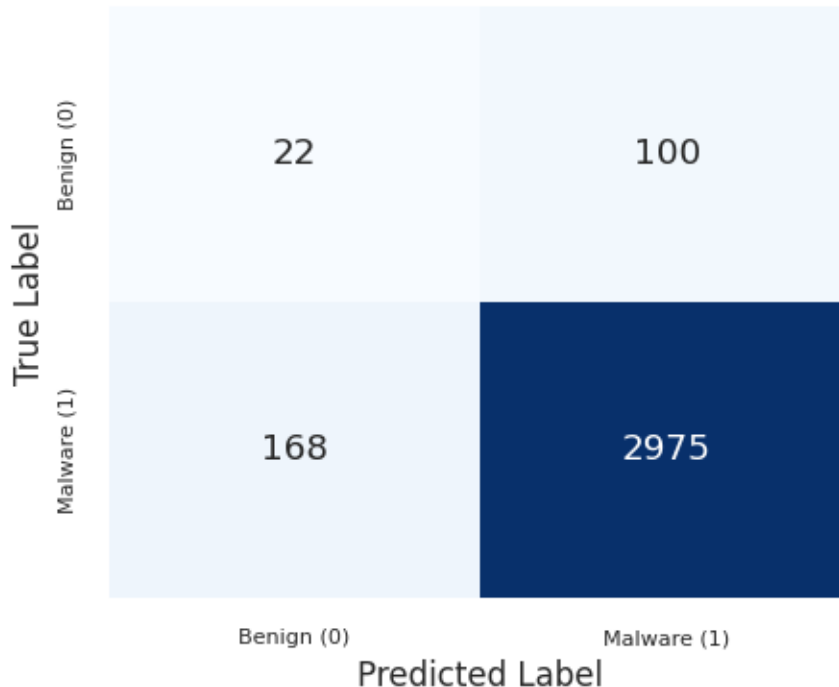
=====
Confusion matrices for ALL FFNN (Sequential IDs) learning rates - VALIDATION
=====

```

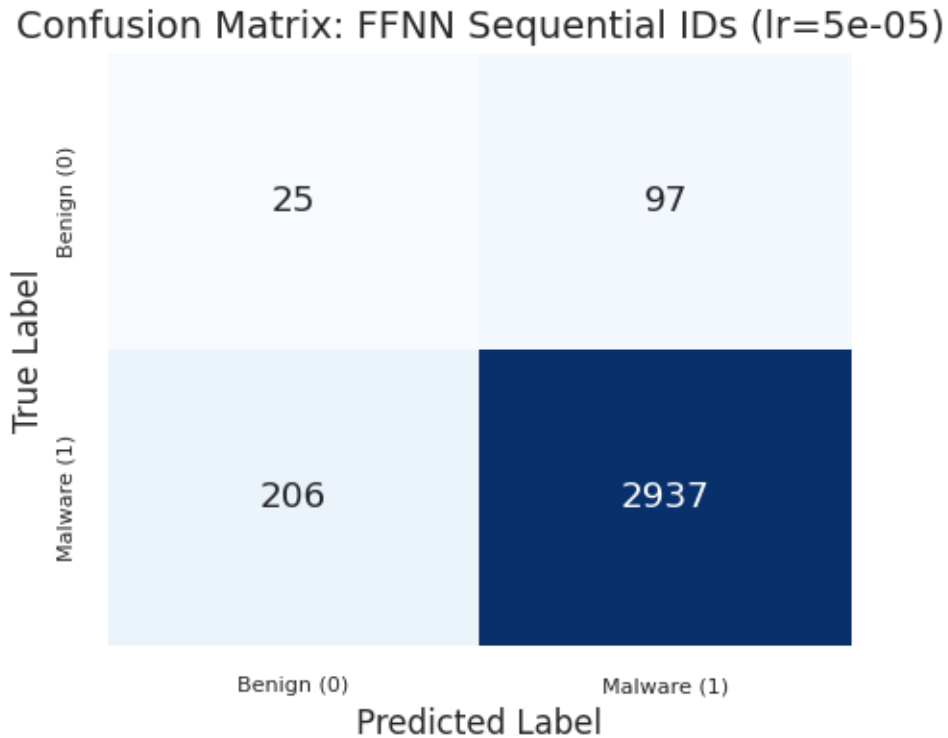
Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/cm_ffnn_sequential_ids_(lr=1e_04).png

Confusion Matrix: FFNN Sequential IDs (lr=1e-04)

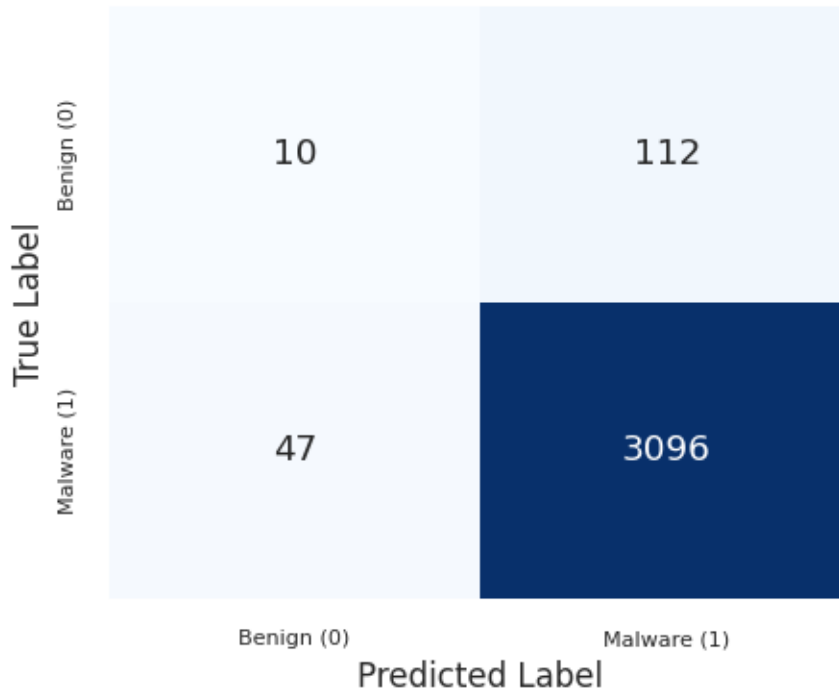


Generating confusion matrix for FFNN (Sequential IDs) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=5e_05).png



Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_05).png

Confusion Matrix: FFNN Sequential IDs (lr=1e-05)



```
[69]: # --- Plot confusion matrices for ALL learning rates (Sequential IDs) - TEST ---
```

```
print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Sequential IDs) learning rates - TEST")
print("=" * 80)

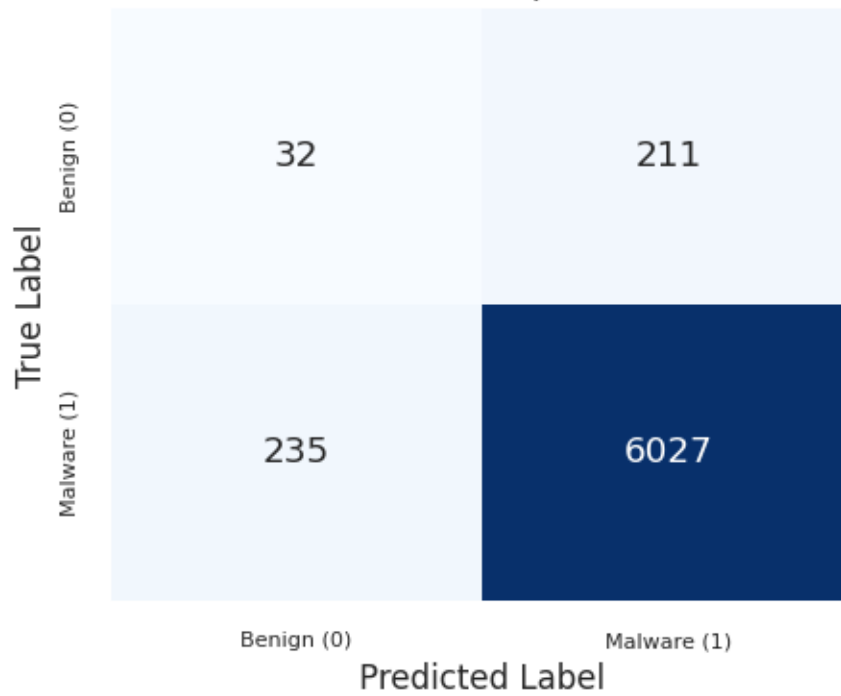
for lr in learning_rates_seq:
    model = models_seq[lr]
    print(f"\nGenerating confusion matrix for FFNN (Sequential IDs) with lr={lr:
↪.0e}...")

    plot_cm_task2(model=model, X_tensor=X_test_tensor, y_tensor=y_test_tensor,
↪model_name=f"FFNN Sequential IDs (lr={lr:.0e})", save_dir=save_dir)
```

```
=====
Confusion matrices for ALL FFNN (Sequential IDs) learning rates - TEST
=====
```

```
Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_04).png
```

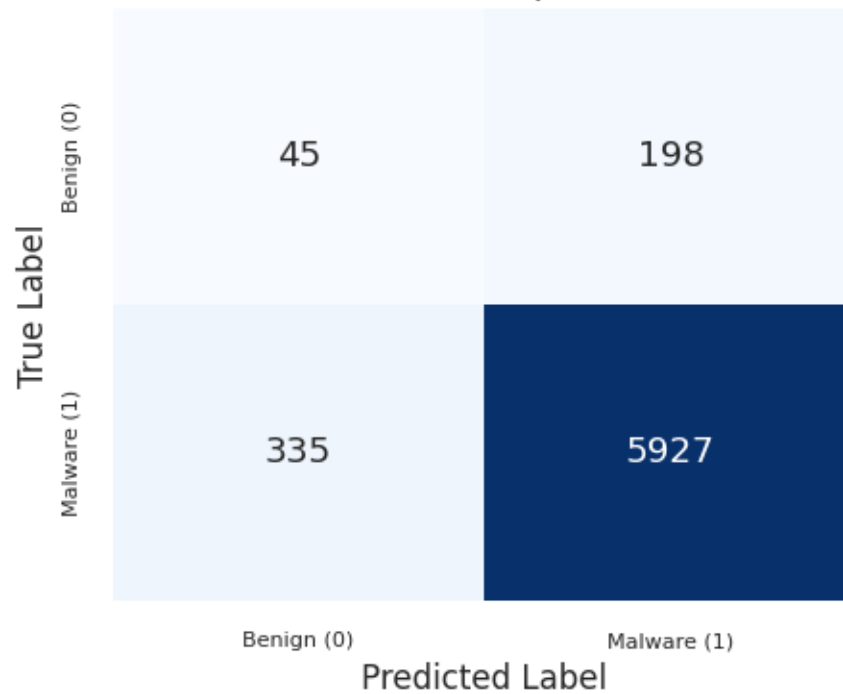
Confusion Matrix: FFNN Sequential IDs (lr=1e-04)



Generating confusion matrix for FFNN (Sequential IDs) with lr=5e-05...

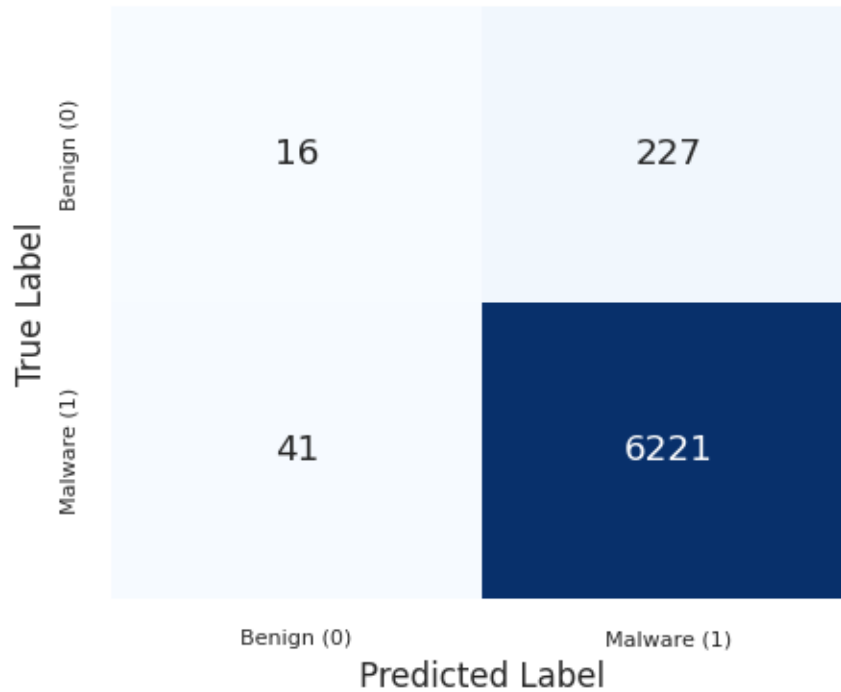
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/cm_ffnn_sequential_ids_(lr=5e_05).png

Confusion Matrix: FFNN Sequential IDs (lr=5e-05)



Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_05).png

Confusion Matrix: FFNN Sequential IDs (lr=1e-05)



Training with Learnable Embeddings Approach

```
[70]: # --- Configuration Constants ---
seq_len = X_train.shape[1]
hidden_dims = [128, 64, 32]
dropout_rates = [0.4, 0.4, 0.4]
batchnorm_flags = [True, True, True]
min_delta = 0.00001
patience_early_stop = 15
epochs = 100

vocab_size = len(api_to_id)
embedding_dim = 128

# --- Device Setup (reuse from previous section if available) ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Compute weights and move to device
pos_weight = compute_pos_weight(y_train_split).to(device)

# --- Define LR Candidates ---
learning_rates_emb = [0.00005, 0.00001, 0.000005]
```

```

histories_emb = {}      # Store results here
models_emb = {}         # Store models here
elapsed_emb_times = {} # Store elapsed times here

print(f"\n{'='*80}")
print("Training FFNN (Learnable Embeddings) with different learning rates")
print(f"{'='*80}")

for lr in learning_rates_emb:
    print(f"\n--- Training FFNN (Learnable Embeddings) with lr={lr:.0e} ---")

    # Initialize model with embedding layer and move to device
    model_emb = MalwareFFNN(
        seq_len,
        hidden_dims,
        dropout_rates,
        batchnorm_flags,
        use_embedding=True,
        vocab_size=vocab_size,
        embedding_dim=embedding_dim,
    ).to(device)

    # Criterion (pos_weight already on device)
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

    # Initialize Optimizer
    optimizer = optim.AdamW(model_emb.parameters(), lr=lr, weight_decay=1e-4)

    # Setup Scheduler
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer,
        mode='min',
        factor=0.5,
        patience=5,
        min_lr=1e-6
    )

    # Train the model
    model_emb, history, elapsed_emb = train_model(
        model_emb,
        train_loader,
        val_loader,
        epochs,
        optimizer,
        criterion,
        min_delta,
        patience_early_stop,

```

```

        scheduler=None
    )

    # Store model and history with numeric LR key
    models_emb[lr] = model_emb
    histories_emb[lr] = history
    elapsed_emb_times[lr] = elapsed_emb

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_emb))
    print(f"Completed. Training time: {elapsed_str} ({elapsed_emb:.2f}s) -  

    ↪Final val loss: {history['val_loss'][-1]:.4f}")

```

Using device: cuda
 Negatives: 489, Positives: 12571
 pos_weight (applied to y=1 samples): 0.2000

=====

Training FFNN (Learnable Embeddings) with different learning rates

=====

--- Training FFNN (Learnable Embeddings) with lr=5e-05 ---

Epoch 1/100 - Train Loss: 0.1499, Val Loss: 0.1436, Val F1(W): 0.8602
 Epoch 5/100 - Train Loss: 0.1136, Val Loss: 0.1153, Val F1(W): 0.9210
 Epoch 10/100 - Train Loss: 0.0874, Val Loss: 0.0942, Val F1(W): 0.9443
 Epoch 15/100 - Train Loss: 0.0641, Val Loss: 0.0818, Val F1(W): 0.9421
 Epoch 20/100 - Train Loss: 0.0463, Val Loss: 0.0768, Val F1(W): 0.9464
 Epoch 25/100 - Train Loss: 0.0375, Val Loss: 0.0741, Val F1(W): 0.9516
 Epoch 30/100 - Train Loss: 0.0301, Val Loss: 0.0800, Val F1(W): 0.9519
 Epoch 35/100 - Train Loss: 0.0235, Val Loss: 0.0816, Val F1(W): 0.9540
 Epoch 40/100 - Train Loss: 0.0211, Val Loss: 0.0890, Val F1(W): 0.9560
 Early stopping at epoch 41 (best val loss: 0.073075)

Training completed in 00:00:36 (hh:mm:ss) - 36.94 seconds

Completed. Training time: 00:00:36 (36.94s) - Final val loss: 0.0934

--- Training FFNN (Learnable Embeddings) with lr=1e-05 ---

Epoch 1/100 - Train Loss: 0.1583, Val Loss: 0.1577, Val F1(W): 0.6542
 Epoch 5/100 - Train Loss: 0.1457, Val Loss: 0.1499, Val F1(W): 0.7580
 Epoch 10/100 - Train Loss: 0.1340, Val Loss: 0.1405, Val F1(W): 0.8339
 Epoch 15/100 - Train Loss: 0.1233, Val Loss: 0.1339, Val F1(W): 0.8669
 Epoch 20/100 - Train Loss: 0.1147, Val Loss: 0.1269, Val F1(W): 0.8940
 Epoch 25/100 - Train Loss: 0.1063, Val Loss: 0.1216, Val F1(W): 0.8962
 Epoch 30/100 - Train Loss: 0.1010, Val Loss: 0.1116, Val F1(W): 0.9160
 Epoch 35/100 - Train Loss: 0.0923, Val Loss: 0.1078, Val F1(W): 0.9174
 Epoch 40/100 - Train Loss: 0.0853, Val Loss: 0.1046, Val F1(W): 0.9226
 Epoch 45/100 - Train Loss: 0.0795, Val Loss: 0.0989, Val F1(W): 0.9288
 Epoch 50/100 - Train Loss: 0.0741, Val Loss: 0.0941, Val F1(W): 0.9351
 Epoch 55/100 - Train Loss: 0.0672, Val Loss: 0.0883, Val F1(W): 0.9420

```
Epoch 60/100 - Train Loss: 0.0616, Val Loss: 0.0854, Val F1(W): 0.9431
Epoch 65/100 - Train Loss: 0.0565, Val Loss: 0.0813, Val F1(W): 0.9428
Epoch 70/100 - Train Loss: 0.0518, Val Loss: 0.0788, Val F1(W): 0.9432
Epoch 75/100 - Train Loss: 0.0482, Val Loss: 0.0782, Val F1(W): 0.9404
Epoch 80/100 - Train Loss: 0.0444, Val Loss: 0.0767, Val F1(W): 0.9387
Epoch 85/100 - Train Loss: 0.0409, Val Loss: 0.0745, Val F1(W): 0.9452
Epoch 90/100 - Train Loss: 0.0369, Val Loss: 0.0731, Val F1(W): 0.9485
Epoch 95/100 - Train Loss: 0.0343, Val Loss: 0.0761, Val F1(W): 0.9462
Epoch 100/100 - Train Loss: 0.0331, Val Loss: 0.0740, Val F1(W): 0.9476
```

```
Training completed in 00:01:31 (hh:mm:ss) - 91.64 seconds
Completed. Training time: 00:01:31 (91.64s) - Final val loss: 0.0740
```

```
--- Training FFNN (Learnable Embeddings) with lr=5e-06 ---
```

```
Epoch 1/100 - Train Loss: 0.1709, Val Loss: 0.1473, Val F1(W): 0.8396
Epoch 5/100 - Train Loss: 0.1610, Val Loss: 0.1452, Val F1(W): 0.8399
Epoch 10/100 - Train Loss: 0.1519, Val Loss: 0.1390, Val F1(W): 0.8704
Epoch 15/100 - Train Loss: 0.1435, Val Loss: 0.1361, Val F1(W): 0.8713
Epoch 20/100 - Train Loss: 0.1371, Val Loss: 0.1333, Val F1(W): 0.8738
Epoch 25/100 - Train Loss: 0.1308, Val Loss: 0.1372, Val F1(W): 0.8936
Epoch 30/100 - Train Loss: 0.1243, Val Loss: 0.1237, Val F1(W): 0.9008
Epoch 35/100 - Train Loss: 0.1195, Val Loss: 0.1183, Val F1(W): 0.9098
Epoch 40/100 - Train Loss: 0.1140, Val Loss: 0.1177, Val F1(W): 0.9071
Epoch 45/100 - Train Loss: 0.1089, Val Loss: 0.1118, Val F1(W): 0.9217
Epoch 50/100 - Train Loss: 0.1052, Val Loss: 0.1118, Val F1(W): 0.9173
Epoch 55/100 - Train Loss: 0.1006, Val Loss: 0.1090, Val F1(W): 0.9180
Epoch 60/100 - Train Loss: 0.0961, Val Loss: 0.1068, Val F1(W): 0.9218
Epoch 65/100 - Train Loss: 0.0922, Val Loss: 0.1031, Val F1(W): 0.9269
Epoch 70/100 - Train Loss: 0.0884, Val Loss: 0.0981, Val F1(W): 0.9355
Epoch 75/100 - Train Loss: 0.0843, Val Loss: 0.1008, Val F1(W): 0.9298
Epoch 80/100 - Train Loss: 0.0810, Val Loss: 0.0958, Val F1(W): 0.9340
Epoch 85/100 - Train Loss: 0.0775, Val Loss: 0.0910, Val F1(W): 0.9382
Epoch 90/100 - Train Loss: 0.0728, Val Loss: 0.0933, Val F1(W): 0.9334
Epoch 95/100 - Train Loss: 0.0711, Val Loss: 0.0889, Val F1(W): 0.9339
Epoch 100/100 - Train Loss: 0.0692, Val Loss: 0.0845, Val F1(W): 0.9490
```

```
Training completed in 00:01:30 (hh:mm:ss) - 90.03 seconds
Completed. Training time: 00:01:30 (90.03s) - Final val loss: 0.0845
```

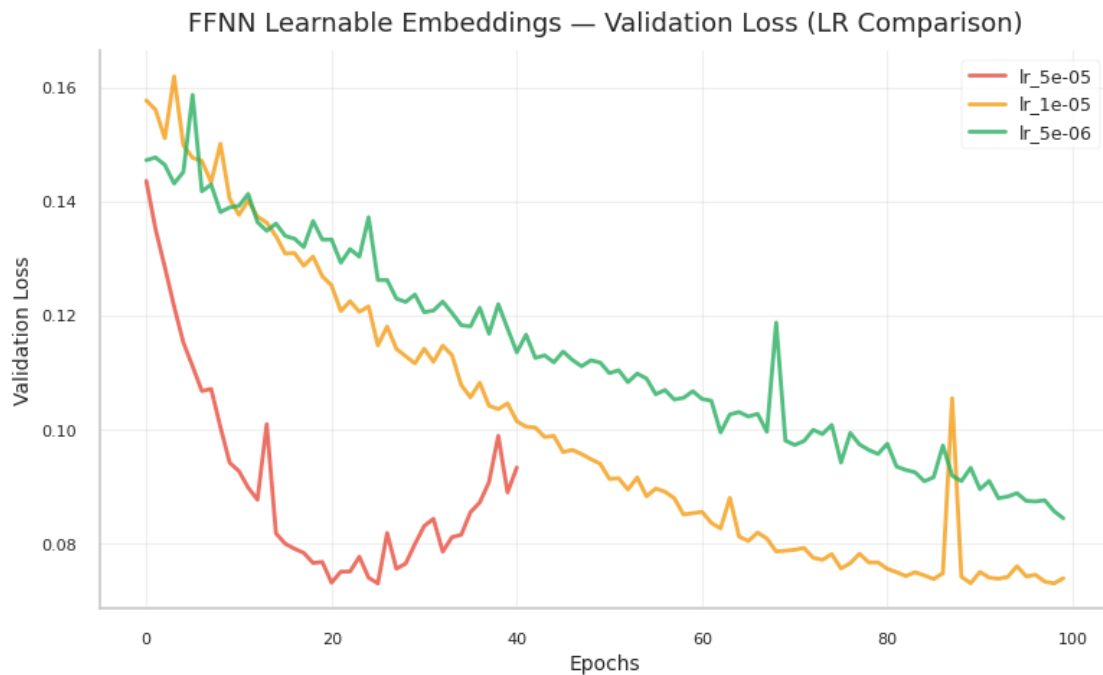
```
[71]: # --- Plot loss curves ---
```

```
# Plot comparison
plot_lr_comparison(histories_emb, "FFNN Learnable Embeddings", save_dir)

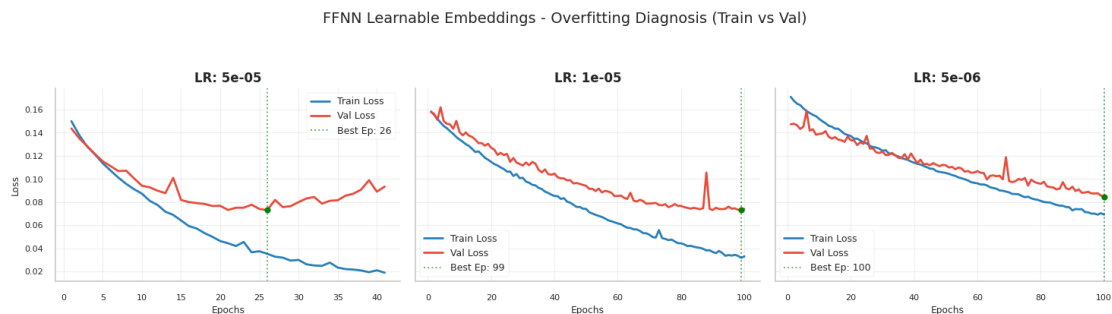
# Plot Train vs Validation
plot_train_val_grid(histories_emb, "FFNN Learnable Embeddings", save_dir)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
```

/task2_plots/ffnn_learnable_embeddings_lr_comparison.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_learnable_embeddings_diagnosis.png



```
[72]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL FFNN (Learnable Embeddings) learning rates")
print("=" * 80)

for lr in learning_rates_emb:
    model = models_emb[lr]
```

```

print(f"\n{'='*60}")
print(f"FFNN (Learnable Embeddings) with lr={lr:.0e}")
print(f"{'='*60}")

# Validation evaluation
print(f'\nValidation report (FFNN Learnable Embeddings - lr={lr:.0e}):')
report = evaluate_model(model, X_val_tensor.to(device), y_val) # Moved
↳X_val_tensor to device
print(report)

# Test evaluation
print(f'Test report (FFNN Learnable Embeddings - lr={lr:.0e}):')
report_test = evaluate_model(model, X_test_tensor.to(device), y_test) #
↳Moved X_test_tensor to device
print(report_test)

# Find and highlight best model
best_lr_emb = min(histories_emb.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f" Best FFNN (Learnable Embeddings) - LR: {best_lr_emb[0]:.0e} (Min Val_
↳Loss: {min(best_lr_emb[1]['val_loss']):.4f})")
print(f"{'='*60}")

```

```

=====
Evaluation for ALL FFNN (Learnable Embeddings) learning rates
=====

```

```

=====
FFNN (Learnable Embeddings) with lr=5e-05
=====

```

Validation report (FFNN Learnable Embeddings - lr=5e-05):

	precision	recall	f1-score	support
0	0.3194	0.3770	0.3459	122
1	0.9756	0.9688	0.9722	3143
accuracy			0.9467	3265
macro avg	0.6475	0.6729	0.6590	3265
weighted avg	0.9511	0.9467	0.9488	3265

Test report (FFNN Learnable Embeddings - lr=5e-05):

	precision	recall	f1-score	support
0	0.1726	0.2593	0.2072	243

1	0.9707	0.9518	0.9611	6262
accuracy			0.9259	6505
macro avg	0.5716	0.6055	0.5842	6505
weighted avg	0.9409	0.9259	0.9330	6505

```
=====
FFNN (Learnable Embeddings) with lr=1e-05
=====
```

Validation report (FFNN Learnable Embeddings - lr=1e-05):

	precision	recall	f1-score	support
0	0.2956	0.3852	0.3345	122
1	0.9759	0.9644	0.9701	3143
accuracy			0.9427	3265
macro avg	0.6357	0.6748	0.6523	3265
weighted avg	0.9504	0.9427	0.9463	3265

Test report (FFNN Learnable Embeddings - lr=1e-05):

	precision	recall	f1-score	support
0	0.1578	0.2798	0.2018	243
1	0.9712	0.9420	0.9564	6262
accuracy			0.9173	6505
macro avg	0.5645	0.6109	0.5791	6505
weighted avg	0.9408	0.9173	0.9282	6505

```
=====
FFNN (Learnable Embeddings) with lr=5e-06
=====
```

Validation report (FFNN Learnable Embeddings - lr=5e-06):

	precision	recall	f1-score	support
0	0.3221	0.3934	0.3542	122
1	0.9763	0.9679	0.9720	3143
accuracy			0.9464	3265
macro avg	0.6492	0.6807	0.6631	3265
weighted avg	0.9518	0.9464	0.9490	3265

Test report (FFNN Learnable Embeddings - lr=5e-06):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.1685	0.2510	0.2017	243
1	0.9704	0.9519	0.9611	6262
accuracy			0.9257	6505
macro avg	0.5694	0.6015	0.5814	6505
weighted avg	0.9404	0.9257	0.9327	6505

```
=====
Best FFNN (Learnable Embeddings) - LR: 5e-05 (Min Val Loss: 0.0731)
=====
```

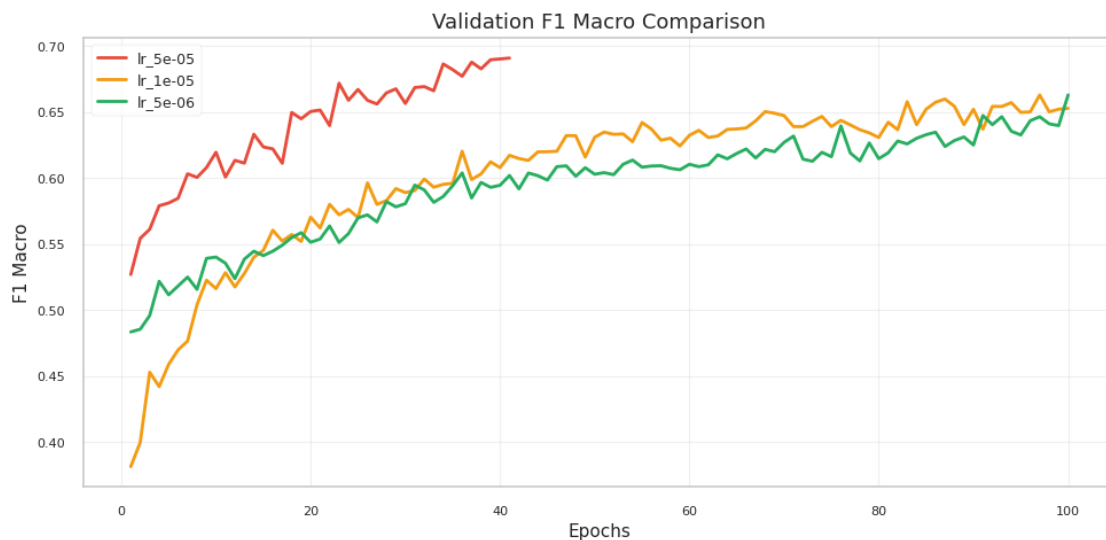
```
[73]: # --- Plot metric curves for ALL learning rates (Learnable Embeddings) ---

print("\n" + "=" * 80)
print("Metric Curves for ALL FFNN (Learnable Embeddings) learning rates")
print("=" * 80)

# The plot_training_metrics function is designed to compare multiple histories.
plot_training_metrics(histories_emb, save_dir=save_dir,
    plot_name="ffnn_learnable_embeddings_f1_comparison")
```

```
=====
Metric Curves for ALL FFNN (Learnable Embeddings) learning rates
=====
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_learnable_embeddings_f1_comparison.png
```



```
[74]: # --- Plot confusion matrices for ALL learning rates (Learnable Embeddings) - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates - VALIDATION")
print("=" * 80)

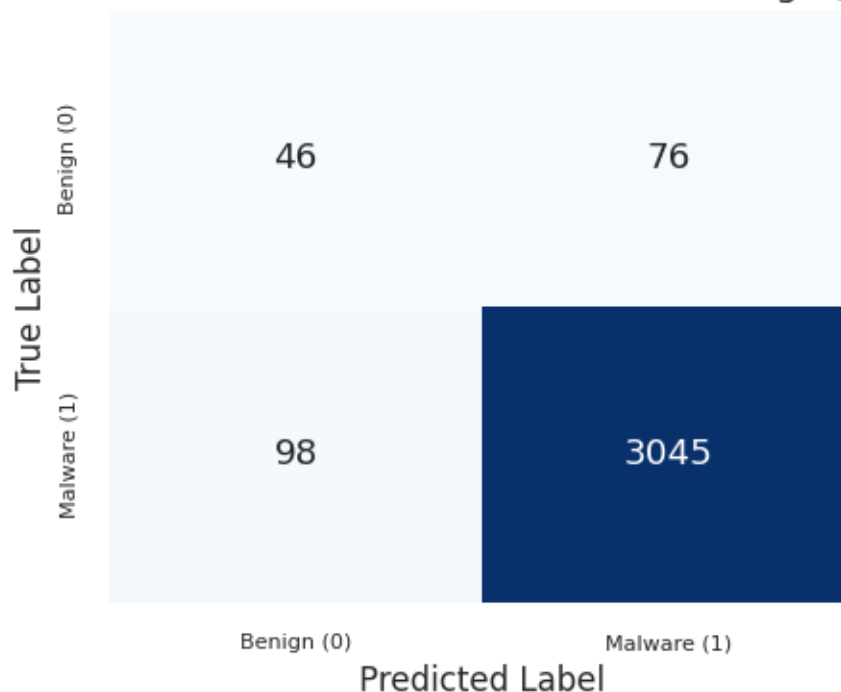
for lr in learning_rates_emb:
    model = models_emb[lr]
    print(f"\nGenerating confusion matrix for FFNN (Learnable Embeddings) with lr={lr:.0e}...")

    plot_cm_task2(model=model, X_tensor=X_val_tensor, y_tensor=y_val_tensor, model_name=f"FFNN Learnable Embeddings (lr={lr:.0e})", save_dir=save_dir)
```

```
=====
Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates -
VALIDATION
=====
```

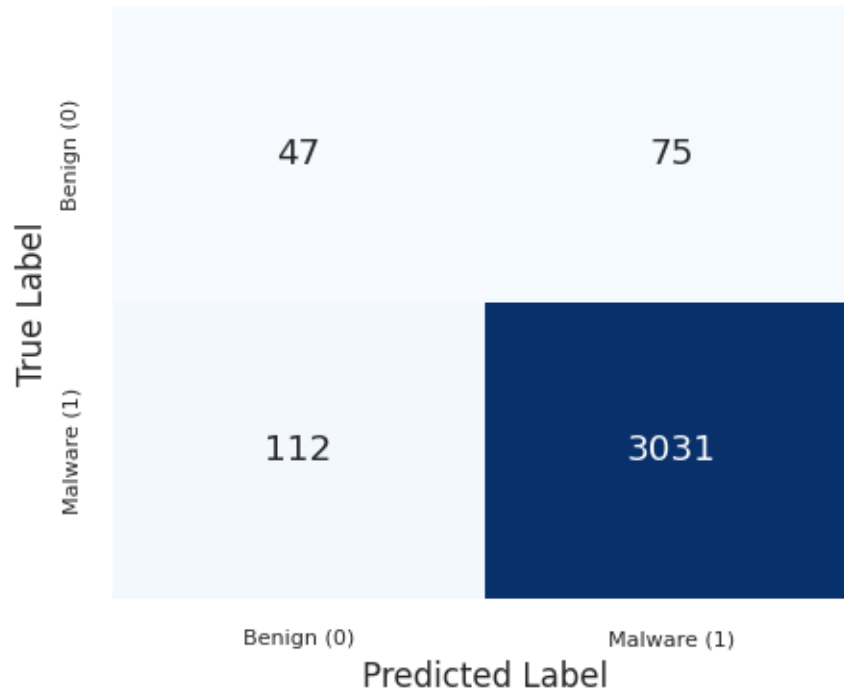
```
Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_05).png
```

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-05)



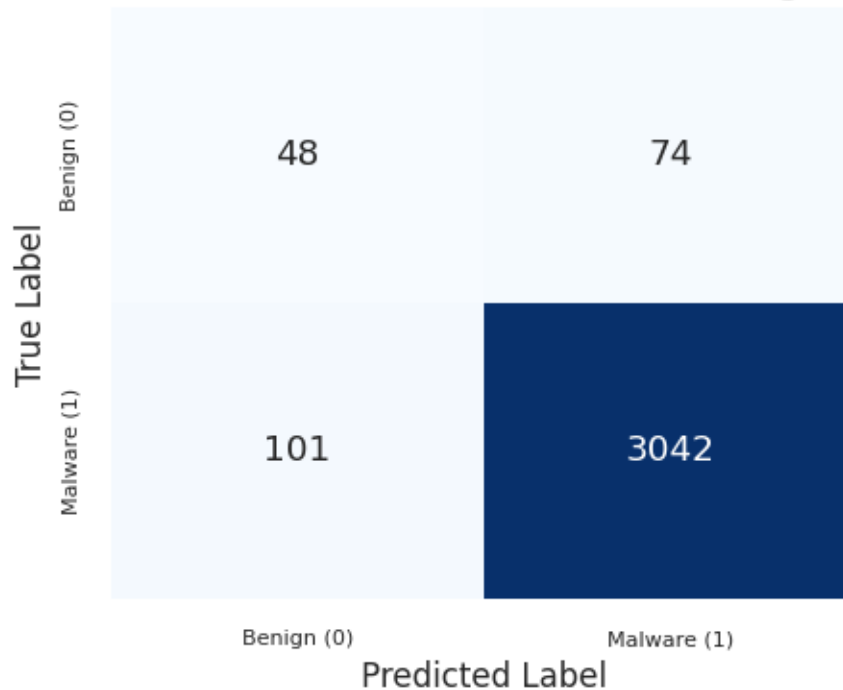
Generating confusion matrix for FFNN (Learnable Embeddings) with $lr=1e-05$..
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=1e_05).png

Confusion Matrix: FFNN Learnable Embeddings ($lr=1e-05$)



Generating confusion matrix for FFNN (Learnable Embeddings) with $lr=5e-06$..
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_06).png

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-06)



```
[75]: # --- Plot confusion matrices for ALL learning rates (Learnable Embeddings) - TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates - TEST")
print("=" * 80)

for lr in learning_rates_emb:
    model = models_emb[lr]
    print(f"\nGenerating confusion matrix for FFNN (Learnable Embeddings) with lr={lr:.0e}...")

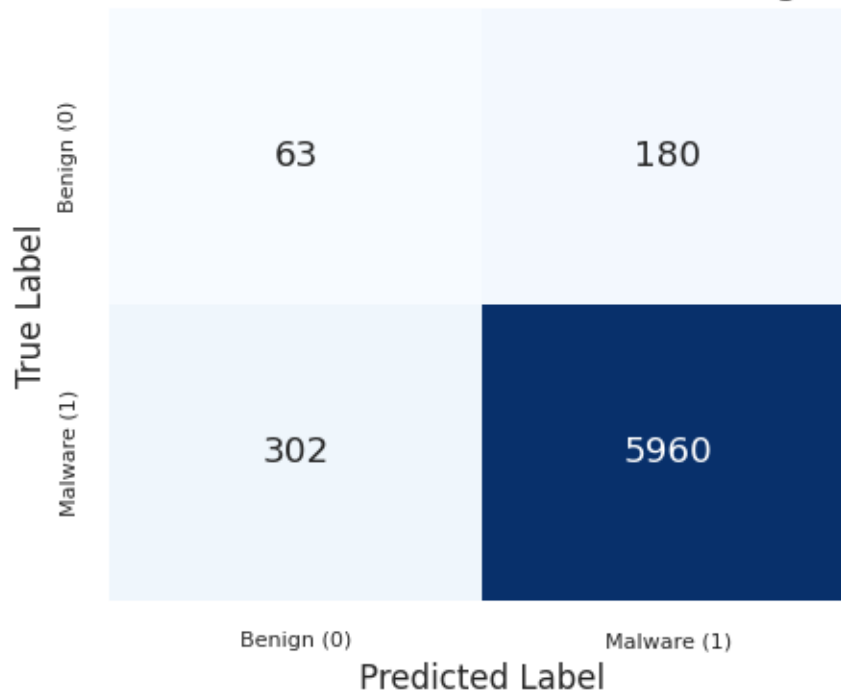
    plot_cm_task2(model=model, X_tensor=X_test_tensor, y_tensor=y_test_tensor, model_name=f"FFNN Learnable Embeddings (lr={lr:.0e})", save_dir=save_dir)
```

=====
Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates - TEST
=====

Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images

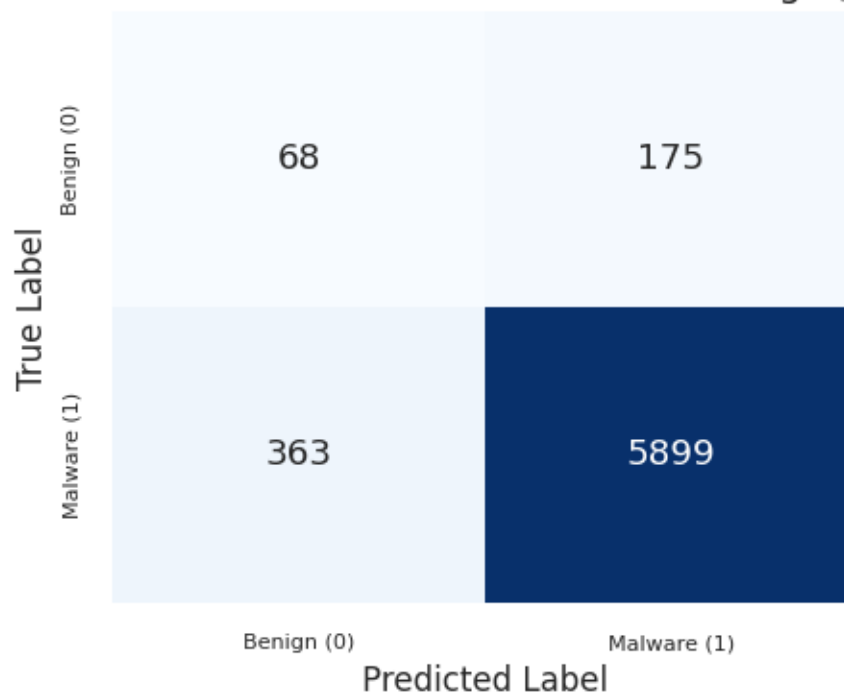
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_05).png

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-05)



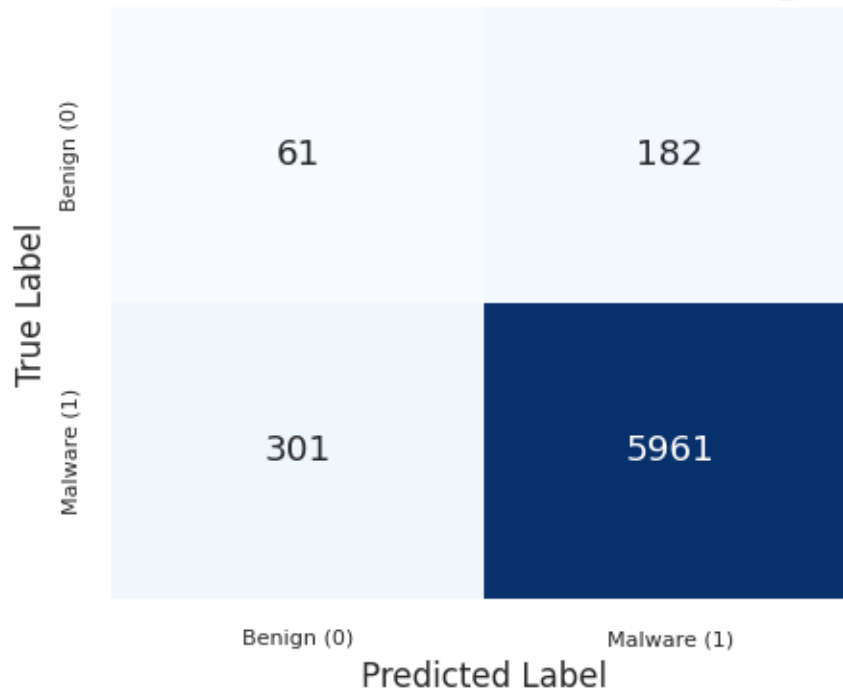
Generating confusion matrix for FFNN (Learnable Embeddings) with lr=1e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=1e_05).png

Confusion Matrix: FFNN Learnable Embeddings (lr=1e-05)



Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-06..
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_06).png

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-06)



Embedding size sensitivity analysis To complete the learnable-embedding experiment, we keep the selected FFNN architecture and learning rate fixed, and vary only the embedding dimensionality. This isolates the effect of representation capacity from the effect of the optimizer or hidden-layer design.

```
[76]: # --- Embedding size sensitivity analysis ---

# The LR grid above selects the best learning rate for the learnable-embedding_
↳FFNN.
# Here we keep that LR and the rest of the architecture fixed, changing only_
↳embedding_dim.
embedding_sizes = [8, 16, 32, 64, 128]
selected_embedding_lr = best_lr_emb[0] if 'best_lr_emb' in globals() else 5e-5

histories_emb_size = {}
models_emb_size = {}
elapsed_emb_size_times = {}
embedding_size_rows = []

def predict_ffnn(model, X_tensor):
    """Return binary predictions for an FFNN model."""
    model.eval()
```

```

model_device = next(model.parameters()).device
with torch.no_grad():
    logits = model(X_tensor.to(model_device))
    probs = torch.sigmoid(logits)
    preds = (probs > 0.5).long().cpu().numpy().flatten()
return preds

print(f"\n{'='*80}")
print("Embedding size sensitivity analysis for FFNN Learnable Embeddings")
print(f"Fixed learning rate: {selected_embedding_lr:.0e}")
print(f"Embedding sizes: {embedding_sizes}")
print(f"{'='*80}")

for emb_dim in embedding_sizes:
    print(f"\n--- Training FFNN Learnable Embeddings with_
↪embedding_dim={emb_dim} ---")

    model_emb_size = MalwareFFNN(
        seq_len,
        hidden_dims,
        dropout_rates,
        batchnorm_flags,
        use_embedding=True,
        vocab_size=vocab_size,
        embedding_dim=emb_dim,
    ).to(device)

    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
    optimizer = optim.AdamW(model_emb_size.parameters()),
↪lr=selected_embedding_lr, weight_decay=1e-4)

    model_emb_size, history, elapsed_emb_size = train_model(
        model_emb_size,
        train_loader,
        val_loader,
        epochs,
        optimizer,
        criterion,
        min_delta,
        patience=patience_early_stop,
        scheduler=None,
    )

    histories_emb_size[emb_dim] = history
    models_emb_size[emb_dim] = model_emb_size
    elapsed_emb_size_times[emb_dim] = elapsed_emb_size

```

```

val_pred = predict_ffnn(model_emb_size, X_val_tensor)
test_pred = predict_ffnn(model_emb_size, X_test_tensor)

val_report = classification_report(y_val, val_pred, output_dict=True,
↳zero_division=0)
test_report = classification_report(y_test, test_pred, output_dict=True,
↳zero_division=0)

embedding_size_rows.append({
    'embedding_dim': emb_dim,
    'min_val_loss': min(history['val_loss']),
    'best_val_macro_f1_curve': max(history['val_f1_macro']),
    'val_macro_f1': val_report['macro avg']['f1-score'],
    'val_benign_recall': val_report['0']['recall'],
    'test_macro_f1': test_report['macro avg']['f1-score'],
    'test_benign_recall': test_report['0']['recall'],
    'training_time_s': elapsed_emb_size,
})

embedding_size_results = pd.DataFrame(embedding_size_rows)
embedding_size_results = embedding_size_results.sort_values('embedding_dim').
↳reset_index(drop=True)

print("\nEmbedding size comparison:")
display(embedding_size_results)

best_embedding_dim_row = embedding_size_results.
↳loc[embedding_size_results['min_val_loss'].idxmin()]
best_embedding_dim = int(best_embedding_dim_row['embedding_dim'])
print(
    f"\nBest embedding size by minimum validation loss: {best_embedding_dim} "
    f"(min val loss = {best_embedding_dim_row['min_val_loss']:.4f}, "
    f"test macro F1 = {best_embedding_dim_row['test_macro_f1']:.4f})"
)

```

```

=====
Embedding size sensitivity analysis for FFNN Learnable Embeddings
Fixed learning rate: 5e-05
Embedding sizes: [8, 16, 32, 64, 128]
=====

```

```

--- Training FFNN Learnable Embeddings with embedding_dim=8 ---
Epoch 1/100 - Train Loss: 0.1838, Val Loss: 0.1767, Val F1(W): 0.1649
Epoch 5/100 - Train Loss: 0.1419, Val Loss: 0.1478, Val F1(W): 0.8332
Epoch 10/100 - Train Loss: 0.1145, Val Loss: 0.1243, Val F1(W): 0.9439
Epoch 15/100 - Train Loss: 0.1002, Val Loss: 0.1140, Val F1(W): 0.9435

```

Epoch 20/100 - Train Loss: 0.0896, Val Loss: 0.1025, Val F1(W): 0.9436
Epoch 25/100 - Train Loss: 0.0833, Val Loss: 0.0985, Val F1(W): 0.9400
Epoch 30/100 - Train Loss: 0.0788, Val Loss: 0.0931, Val F1(W): 0.9369
Epoch 35/100 - Train Loss: 0.0742, Val Loss: 0.0927, Val F1(W): 0.9264
Epoch 40/100 - Train Loss: 0.0665, Val Loss: 0.1065, Val F1(W): 0.9158
Epoch 45/100 - Train Loss: 0.0625, Val Loss: 0.0893, Val F1(W): 0.9156
Epoch 50/100 - Train Loss: 0.0568, Val Loss: 0.0900, Val F1(W): 0.9014
Epoch 55/100 - Train Loss: 0.0537, Val Loss: 0.0907, Val F1(W): 0.9187
Epoch 60/100 - Train Loss: 0.0544, Val Loss: 0.0886, Val F1(W): 0.9132
Early stopping at epoch 64 (best val loss: 0.087366)

Training completed in 00:00:54 (hh:mm:ss) - 54.66 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=16 ---

Epoch 1/100 - Train Loss: 0.1597, Val Loss: 0.1435, Val F1(W): 0.8965
Epoch 5/100 - Train Loss: 0.1248, Val Loss: 0.1230, Val F1(W): 0.9449
Epoch 10/100 - Train Loss: 0.1026, Val Loss: 0.1055, Val F1(W): 0.9379
Epoch 15/100 - Train Loss: 0.0889, Val Loss: 0.0928, Val F1(W): 0.9323
Epoch 20/100 - Train Loss: 0.0767, Val Loss: 0.0861, Val F1(W): 0.9299
Epoch 25/100 - Train Loss: 0.0673, Val Loss: 0.0824, Val F1(W): 0.9240
Epoch 30/100 - Train Loss: 0.0606, Val Loss: 0.0804, Val F1(W): 0.9261
Epoch 35/100 - Train Loss: 0.0518, Val Loss: 0.0777, Val F1(W): 0.9284
Epoch 40/100 - Train Loss: 0.0462, Val Loss: 0.0819, Val F1(W): 0.9397
Epoch 45/100 - Train Loss: 0.0377, Val Loss: 0.0856, Val F1(W): 0.9391
Early stopping at epoch 50 (best val loss: 0.077684)

Training completed in 00:00:41 (hh:mm:ss) - 41.95 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=32 ---

Epoch 1/100 - Train Loss: 0.1281, Val Loss: 0.1288, Val F1(W): 0.9440
Epoch 5/100 - Train Loss: 0.1061, Val Loss: 0.1125, Val F1(W): 0.9443
Epoch 10/100 - Train Loss: 0.0916, Val Loss: 0.0999, Val F1(W): 0.9450
Epoch 15/100 - Train Loss: 0.0758, Val Loss: 0.0901, Val F1(W): 0.9414
Epoch 20/100 - Train Loss: 0.0645, Val Loss: 0.0814, Val F1(W): 0.9355
Epoch 25/100 - Train Loss: 0.0541, Val Loss: 0.0778, Val F1(W): 0.9416
Epoch 30/100 - Train Loss: 0.0423, Val Loss: 0.0745, Val F1(W): 0.9441
Epoch 35/100 - Train Loss: 0.0340, Val Loss: 0.0746, Val F1(W): 0.9415
Epoch 40/100 - Train Loss: 0.0285, Val Loss: 0.0771, Val F1(W): 0.9438
Early stopping at epoch 45 (best val loss: 0.074494)

Training completed in 00:00:38 (hh:mm:ss) - 38.63 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=64 ---

Epoch 1/100 - Train Loss: 0.1957, Val Loss: 0.1689, Val F1(W): 0.3027
Epoch 5/100 - Train Loss: 0.1420, Val Loss: 0.1424, Val F1(W): 0.8254
Epoch 10/100 - Train Loss: 0.1041, Val Loss: 0.1078, Val F1(W): 0.9144
Epoch 15/100 - Train Loss: 0.0787, Val Loss: 0.0960, Val F1(W): 0.9303
Epoch 20/100 - Train Loss: 0.0581, Val Loss: 0.0776, Val F1(W): 0.9381

Epoch 25/100 - Train Loss: 0.0447, Val Loss: 0.0745, Val F1(W): 0.9424
 Epoch 30/100 - Train Loss: 0.0348, Val Loss: 0.0784, Val F1(W): 0.9518
 Epoch 35/100 - Train Loss: 0.0268, Val Loss: 0.0850, Val F1(W): 0.9572
 Epoch 40/100 - Train Loss: 0.0219, Val Loss: 0.0897, Val F1(W): 0.9540
 Early stopping at epoch 42 (best val loss: 0.072178)

Training completed in 00:00:35 (hh:mm:ss) - 35.46 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=128 ---

Epoch 1/100 - Train Loss: 0.1568, Val Loss: 0.1595, Val F1(W): 0.5722
 Epoch 5/100 - Train Loss: 0.1165, Val Loss: 0.1261, Val F1(W): 0.8777
 Epoch 10/100 - Train Loss: 0.0852, Val Loss: 0.0995, Val F1(W): 0.9213
 Epoch 15/100 - Train Loss: 0.0637, Val Loss: 0.0832, Val F1(W): 0.9310
 Epoch 20/100 - Train Loss: 0.0491, Val Loss: 0.0760, Val F1(W): 0.9424
 Epoch 25/100 - Train Loss: 0.0375, Val Loss: 0.0782, Val F1(W): 0.9371
 Epoch 30/100 - Train Loss: 0.0282, Val Loss: 0.0777, Val F1(W): 0.9497
 Epoch 35/100 - Train Loss: 0.0242, Val Loss: 0.0851, Val F1(W): 0.9566
 Epoch 40/100 - Train Loss: 0.0220, Val Loss: 0.0903, Val F1(W): 0.9575
 Early stopping at epoch 43 (best val loss: 0.073046)

Training completed in 00:00:38 (hh:mm:ss) - 38.35 seconds

Embedding size comparison:

	embedding_dim	min_val_loss	best_val_macro_f1_curve	val_macro_f1 \
0	8	0.087366	0.591308	0.586291
1	16	0.077684	0.644210	0.609871
2	32	0.074494	0.675511	0.658750
3	64	0.072178	0.688555	0.663711
4	128	0.073046	0.684105	0.677462

	val_benign_recall	test_macro_f1	test_benign_recall	training_time_s
0	0.385246	0.567507	0.300412	54.656821
1	0.418033	0.563643	0.333333	41.949461
2	0.450820	0.576060	0.308642	38.630303
3	0.385246	0.581816	0.279835	35.455317
4	0.450820	0.563320	0.246914	38.349021

Best embedding size by minimum validation loss: 64 (min val loss = 0.0722, test macro F1 = 0.5818)

```
[77]: # --- Plot embedding size comparison ---

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(
    embedding_size_results['embedding_dim'],
```

```

        embedding_size_results['min_val_loss'],
        marker='o',
        linewidth=2,
    )
    axes[0].set_title('Minimum Validation Loss vs Embedding Size')
    axes[0].set_xlabel('Embedding Size')
    axes[0].set_ylabel('Minimum Validation Loss')
    axes[0].grid(True, alpha=0.3)

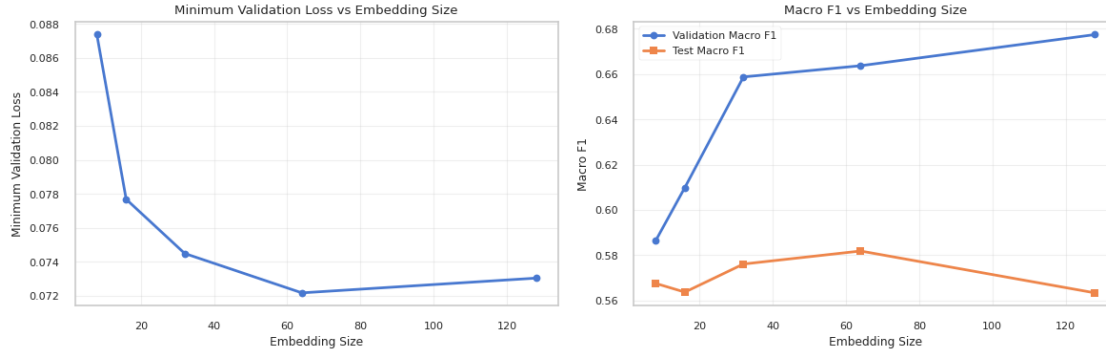
    axes[1].plot(
        embedding_size_results['embedding_dim'],
        embedding_size_results['val_macro_f1'],
        marker='o',
        linewidth=2,
        label='Validation Macro F1',
    )
    axes[1].plot(
        embedding_size_results['embedding_dim'],
        embedding_size_results['test_macro_f1'],
        marker='s',
        linewidth=2,
        label='Test Macro F1',
    )
    axes[1].set_title('Macro F1 vs Embedding Size')
    axes[1].set_xlabel('Embedding Size')
    axes[1].set_ylabel('Macro F1')
    axes[1].grid(True, alpha=0.3)
    axes[1].legend()

plt.tight_layout()
if save_dir:
    save_plot(fig, 'ffnn_embedding_size_comparison', save_dir)
plt.show()

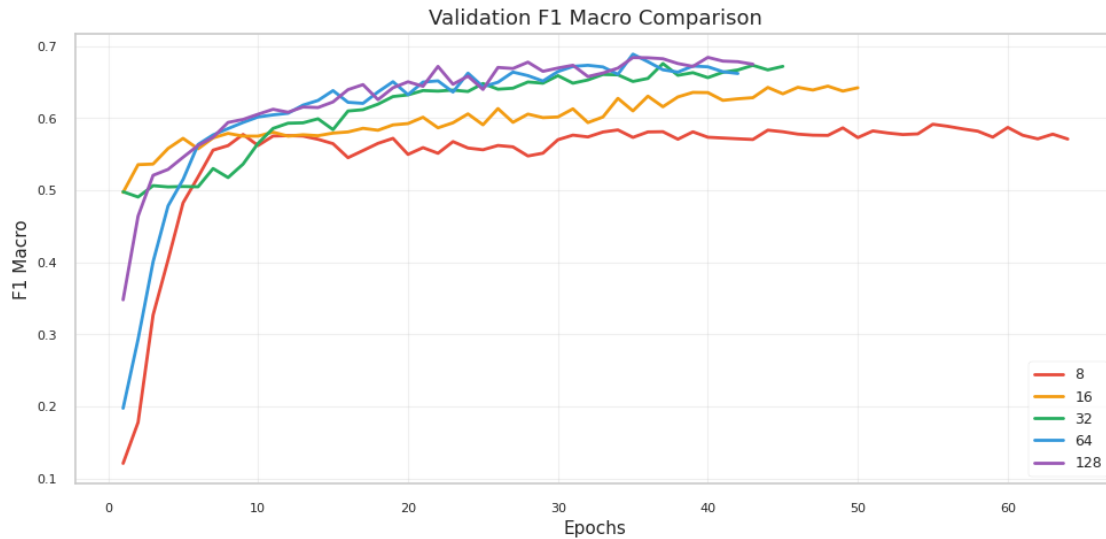
# Optional: compare the validation macro F1 curves across embedding sizes.
plot_training_metrics(
    histories_emb_size,
    save_dir=save_dir,
    plot_name='ffnn_embedding_size_f1_comparison'
)

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_embedding_size_comparison.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_embedding_size_f1_comparison.png



Q: Use a FFNN in both cases. Report how you selected the hyperparameters of your final model, and justify your choices. For Task 2, we implemented two distinct FFNN configurations: one using Sequential Identifiers and another utilizing Learnable Embeddings. The hyperparameter selection was driven by the need to handle a severely imbalanced dataset while maintaining architectural flexibility.

1. Architecture and Regularization Hidden Layers: We adopted a funnel-shaped architecture. For the Sequential IDs model, we used [256, 128, 64], while for the Embedding model, we used [128, 64, 32]. This design forces the network to compress information and learn increasingly abstract representations of API patterns.

Batch Normalization & Dropout: Every hidden layer includes Batch Normalization to stabilize training and accelerate convergence. We applied a high Dropout rate (up to 0.4) to mitigate overfitting, which is a significant risk given the small number of benign samples compared to the

majority class. Then, as we move to the next layers, the dropout layer decreased initially to 0.35, and at the end to 0.3

2. Addressing Class Imbalance Weighted Loss: We used BCEWithLogitsLoss with a `pos_weight` of 0.20 (calculated as the ratio of benign to malware samples). This penalizes errors on the minority class more heavily, directly aiming to improve the Recall for benign software, which is critical in a cybersecurity context.
3. Training Strategy Optimization: We used AdamW with `weight_decay` (1e-4 for Learnable Embeddings and 1e-5 for Sequential IDs) for effective weight regularization.

Learning Rate (LR) Selection: Through an LR grid search:

For Sequential IDs, 1e-04 proved to be the most stable.

For Learnable Embeddings, 5e-05 yielded the best performance, achieving the lowest minimum validation loss (0.0726).

Early Stopping: We implemented early stopping with a patience of 15 epochs based on validation loss to prevent the model from memorizing training noise.

Q: Can you obtain the same results for sequential identifiers and learnable embeddings? If not, why? No, the results demonstrate that the Learnable Embeddings approach significantly outperforms the Sequential Identifiers.

1. Performance Comparison (Macro F1-score) Sequential IDs: The best model (LR 1e-04) achieved a Macro F1-score of 0.56 in validation and 0.53 in test. The Recall for the benign class was extremely low (ranging between 0.11 and 0.22), showing that the model struggled to identify benign samples.

Learnable Embeddings: The best model (LR 5e-05) significantly improved these metrics, reaching a Macro F1-score of 0.66 in validation and 0.58 in test. Crucially, the benign Recall increased to 0.40 in the validation set.

2. Technical Justification: Why Embeddings are Superior Semantic Representation: Sequential identifiers treat API calls as ordinal numbers, implying a mathematical relationship (e.g., API 5 is “closer” to 6 than to 10) that does not exist. Embeddings, however, map each API into a continuous vector space where the network can learn semantic proximity. API calls with similar functional roles (e.g., registry modifications) cluster together, providing richer features.

Feature Enrichment: The embedding layer (dim=128) transforms a single integer ID into a high-dimensional representation. This allows the FFNN to capture complex interactions and hidden dependencies between API calls that raw identifiers simply cannot represent.

Generalization: The training curves show that the Embedding model reaches a much lower validation loss (0.07 vs 0.09). While the FFNN architecture still suffers from ignoring the strict temporal order of calls (addressed later with RNNs), the learnable embeddings provide a much more discriminative input for the classifier compared to raw IDs.

3. Embedding-size sensitivity The additional embedding-size experiment evaluates whether the improvement comes from the embedding mechanism itself or from the chosen dimensionality. Smaller embeddings are cheaper and can regularize the model, while larger embeddings

increase representation capacity but may overfit or require more training time.

```
[78]: # =====
# FINAL SUMMARY
# =====

# Helper function to extract macro avg from the best FFNN model
def get_macro_avg_stats_ffnn(model, X_tensor):
    """
    Predict and get macro average statistics for an FFNN model.
    """
    # Get the device from the model itself
    device = next(model.parameters()).device

    model.eval()
    with torch.no_grad():
        outputs = model(X_tensor.to(device)) # Move input tensor to device
        probs = torch.sigmoid(outputs)
        y_pred = (probs > 0.5).long().cpu().numpy().flatten()

    y_true = y_val # Using validation set
    report_dict = classification_report(y_true, y_pred, output_dict=True,
    ↪ zero_division=0)
    return report_dict['macro avg']

# --- 1. Prepare Stats for FFNN Sequential IDs ---
best_model_seq_obj = models_seq[best_lr_seq[0]]
macro_seq = get_macro_avg_stats_ffnn(best_model_seq_obj, X_val_tensor)

# --- 2. Prepare Stats for FFNN Learnable Embeddings ---
best_model_emb_obj = models_emb[best_lr_emb[0]]
macro_emb = get_macro_avg_stats_ffnn(best_model_emb_obj, X_val_tensor)

# --- PRINTING THE REPORT ---

print("\n" + "=" * 100)
print("TASK 2 SUMMARY: Best Learning Rates & Macro Avg Performance")
print("=" * 100)
# Header for the manual table
print(f"{'Model':<30} | {'Best LR':<10} | {'Val Loss':<10} | {'Macro P':<8} | ␣
    ↪ {'Macro R':<8} {'Macro F1':<8} | {'Time':<10}")
print("-" * 100)

# 1. FFNN Sequential IDs
t_seq = time.strftime('%H:%M:%S', time.
    ↪ gmtime(elapsed_seq_times[best_lr_seq[0]]))
```

```

print(f'{"FFNN (Sequential IDs)":<30} | {best_lr_seq[0]:.0e}      |␣
↳{min(best_lr_seq[1]['val_loss']):.4f}      | "
      f"{macro_seq['precision']:.4f}    {macro_seq['recall']:.4f}    ␣
↳{macro_seq['f1-score']:.4f}    | {t_seq}")

# 2. FFNN Learnable Embeddings
t_emb = time.strftime('%H:%M:%S', time.
↳gmtime(elapsed_emb_times[best_lr_emb[0]]))
print(f'{"FFNN (Learnable Embeddings)":<30} | {best_lr_emb[0]:.0e}      |␣
↳{min(best_lr_emb[1]['val_loss']):.4f}      | "
      f"{macro_emb['precision']:.4f}    {macro_emb['recall']:.4f}    ␣
↳{macro_emb['f1-score']:.4f}    | {t_emb}")

# --- 3. Optional Embedding Size Summary ---
if 'embedding_size_results' in globals():
    best_emb_loss_row = embedding_size_results.
↳loc[embedding_size_results['min_val_loss'].idxmin()]
    best_emb_f1_row = embedding_size_results.
↳loc[embedding_size_results['val_macro_f1'].idxmax()]

    print("\n" + "=" * 100)
    print("EMBEDDING SIZE SENSITIVITY SUMMARY")
    print("=" * 100)
    print(
        f"Best by min validation loss:␣
↳dim={int(best_emb_loss_row['embedding_dim'])} | "
        f"val loss={best_emb_loss_row['min_val_loss']:.4f} | "
        f"val macro F1={best_emb_loss_row['val_macro_f1']:.4f} | "
        f"test macro F1={best_emb_loss_row['test_macro_f1']:.4f} | "
        f"time={best_emb_loss_row['training_time_s']:.2f}s"
    )
    print(
        f"Best by validation macro F1:␣
↳dim={int(best_emb_f1_row['embedding_dim'])} | "
        f"val loss={best_emb_f1_row['min_val_loss']:.4f} | "
        f"val macro F1={best_emb_f1_row['val_macro_f1']:.4f} | "
        f"test macro F1={best_emb_f1_row['test_macro_f1']:.4f} | "
        f"time={best_emb_f1_row['training_time_s']:.2f}s"
    )
)

```

=====

TASK 2 SUMMARY: Best Learning Rates & Macro Avg Performance

=====

Model	Best LR	Val Loss	Macro P	Macro R
-------	---------	----------	---------	---------

Macro F1 | Time

```
-----  
-----  
FFNN (Sequential IDs)          | 1e-04      | 0.0914     | 0.5416    0.5634  
0.5490    | 00:00:33  
FFNN (Learnable Embeddings)   | 5e-05      | 0.0731     | 0.6475    0.6729  
0.6590    | 00:00:36
```

EMBEDDING SIZE SENSITIVITY SUMMARY

```
=====
```

Best by min validation loss: dim=64 | val loss=0.0722 | val macro F1=0.6637 |
test macro F1=0.5818 | time=35.46s
Best by validation macro F1: dim=128 | val loss=0.0730 | val macro F1=0.6775 |
test macro F1=0.5633 | time=38.35s

1.4 Task 3 - Recursive Neural Network (RNN)

```
[79]: # Create directory for plots  
save_dir = results_path + 'images/' + 'task3_plots/'  
os.makedirs(save_dir, exist_ok=True)
```

```
[80]: # --- Choose number of epochs depending on device (GPU vs CPU) ---  
  
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
  
# Use fewer epochs when running on CPU to avoid extremely long runs  
if torch.cuda.is_available():  
    num_epochs = 150  
else:  
    num_epochs = 15  
  
print(f"Number of epochs: {num_epochs}")
```

Number of epochs: 150

1.4.1 Sequence Modeling with RNNs

```
[81]: # --- Build vocabulary and API-to-ID mappings from training data ---  
  
def build_vocab_from_train(df_train, seq_col="api_call_sequence"):  
    """  
    Extract the set of unique API calls from the training set only.  
    Returns a sorted list of API call names (vocab).  
    """  
    vocab = sorted({api for seq in df_train[seq_col] for api in seq})
```

```

return vocab

def build_api_mappings(vocab):
    """
    Create dictionaries mapping API names to integer IDs and vice versa.
    Reserves indices 0 and 1 for <PAD> and <UNK> special tokens.
    """
    api_to_id = {"<PAD>": PAD_IDX, "<UNK>": UNK_IDX}
    for i, api in enumerate(vocab):
        api_to_id[api] = i + 2 # start after PAD/UNK
    id_to_api = {i: api for api, i in api_to_id.items()}
    return api_to_id, id_to_api

# Build vocabulary from training data only (no data leakage)
vocab_train = build_vocab_from_train(df_train)
api_to_id, id_to_api = build_api_mappings(vocab_train)

```

```

[82]: # --- Encode API call sequences to integer IDs ---

def encode_sequences(df, api_to_id, seq_col="api_call_sequence",
    label_col="is_malware", max_len=None):
    """
    Convert API call sequences to lists of integer IDs using the mapping.
    Unknown API calls (not seen in training) are mapped to <UNK> (PAD_IDX).
    Optionally truncate sequences to max_len (but we preserve full length for
    RNNs).
    """
    seqs, labels = [], []
    for seq, y in zip(df[seq_col], df[label_col]):
        ids = [api_to_id.get(api, PAD_IDX) for api in seq]
        if max_len is not None:
            ids = ids[:max_len] # only if you explicitly choose to truncate
        seqs.append(ids)
        labels.append(int(y))
    return seqs, labels

# Encode training and test sequences (no truncation - RNNs handle variable
lengths)
train_seqs, train_labels = encode_sequences(df_train, api_to_id)
test_seqs, test_labels = encode_sequences(df_test, api_to_id)

```

```

[83]: # --- Split training data into train/validation subsets ---

# Create indices for stratified train/val split (preserves class balance)
idx = list(range(len(train_seqs)))
train_idx, val_idx = train_test_split(
    idx, test_size=0.2, random_state=42, stratify=train_labels)

```

```

)

def subset_by_idx(seqs, labels, indices):
    """Helper to extract sequences and labels at given indices."""
    return [seqs[i] for i in indices], [labels[i] for i in indices]

# Create training and validation subsets
train_seqs_sub, train_labels_sub = subset_by_idx(train_seqs, train_labels,
    ↪train_idx)
val_seqs_sub, val_labels_sub = subset_by_idx(train_seqs, train_labels, val_idx)

```

[84]: # --- Define PyTorch Dataset for variable-length sequences ---

```

class SeqDataset(torch.utils.data.Dataset):
    """
    A PyTorch Dataset that wraps sequences and their labels.
    Each item returns a tensor of API IDs and a scalar label.
    Note: Sequences have variable lengths at this stage.
    """
    def __init__(self, sequences, labels):
        self.sequences = sequences
        self.labels = labels

    def __len__(self):
        return len(self.sequences)

    def __getitem__(self, idx):
        x = torch.tensor(self.sequences[idx], dtype=torch.long)
        y = torch.tensor(self.labels[idx], dtype=torch.float32) # scalar
        return x, y

```

[85]: # --- Custom collate function for padding and packing ---

```

def collate_fn(batch):
    """
    Custom collate function for DataLoader:
    1. Sorts sequences by length (descending) - required for
    ↪pack_padded_sequence
    2. Pads all sequences to the max length in this batch
    3. Returns padded sequences, original lengths, and labels (CPU tensors)
    Note: Device transfer is handled by the training loop, not the collate
    ↪function.
    """
    sequences, labels = zip(*batch)

    # Compute original lengths before padding
    lengths = torch.tensor([len(s) for s in sequences], dtype=torch.long)

```

```

# Sort by length (descending) for pack_padded_sequence compatibility
lengths_sorted, perm_idx = lengths.sort(descending=True)

# Reorder sequences and labels according to sorted indices
sequences = [sequences[i] for i in perm_idx]
labels = torch.stack([labels[i] for i in perm_idx]).unsqueeze(1)  # (B,1)

# Pad sequences to max length in batch (dynamic padding)
padded = pad_sequence(sequences, batch_first=True, padding_value=PAD_IDX)

# Return CPU tensors - training loop handles device transfer
return padded, lengths_sorted, labels

```

```

[86]: # --- Create DataLoaders with custom collate function ---

# Wrap sequences in Dataset objects
train_ds = SeqDataset(train_seqs_sub, train_labels_sub)
val_ds = SeqDataset(val_seqs_sub, val_labels_sub)
test_ds = SeqDataset(test_seqs, test_labels)

# Create DataLoaders with dynamic padding via collate_fn
# Note: collate_fn handles variable-length sequences by padding per batch
train_loader = DataLoader(train_ds, batch_size=64, shuffle=True,
    ↪collate_fn=collate_fn)
val_loader = DataLoader(val_ds, batch_size=64, shuffle=False,
    ↪collate_fn=collate_fn)
test_loader = DataLoader(test_ds, batch_size=64, shuffle=False,
    ↪collate_fn=collate_fn)

```

Q: With RNNs, do you still have to pad your data? If yes, how? Yes, when working with sequences of varying lengths using standard PyTorch RNN layers (`nn.RNN`, `nn.LSTM`, `nn.GRU`), you **still need to pad the data**.

While RNNs can theoretically process sequences of any length, the requirement for processing data in batches on hardware like GPUs necessitates that all sequences within a single batch have the same dimensions. Padding ensures this uniform shape.

The padding is typically done by adding a special padding token (often mapped to index 0) to the end of shorter sequences until they match the length of the longest sequence in the current batch (or a predefined maximum length).

However, to prevent the RNN from processing the padding tokens as if they were real data, a technique called “**packing padded sequences**” is used. This is implemented using `torch.nn.utils.rnn.pack_padded_sequence`. This function takes the padded sequences and their original lengths, and effectively tells the RNN to only perform computations on the non-padded elements. After the RNN processes the packed sequence, the output can be “unpacked” back into a padded sequence using `torch.nn.utils.rnn.pad_packed_sequence`.

In the provided code, the `collate_fn` function handles this. It pads sequences to the maxi-

mum length within each batch and also sorts the sequences by length, which is a requirement for `pack_padded_padded_sequence`. The `SimpleRNNClassifier` and `LSTMClassifier` models then use `pack_padded_sequence` within their `forward` methods to process the variable-length inputs efficiently while ignoring the padding.

Q: Do you have to truncate the testing sequences? Justify your answer with your understanding of why it is/it is not the case. When using RNNs with padding and packing (as implemented in Task 3), you **do not strictly have to truncate the testing sequences** if the model is designed to handle variable lengths via padding/packing.

Here's why:

1. **Padding and Packing:** The core mechanism used for handling variable-length sequences in the implemented RNN models is padding combined with `pack_padded_sequence`. This approach allows the RNN layers to process only the actual data in each sequence, effectively ignoring the padding tokens. The lengths of the original sequences are passed to `pack_padded_sequence`, which manages the computations for each time step up to the true length of each sequence in the batch.
2. **Variable Length Handling:** Because the packing/unpacking mechanism handles sequences of different lengths within a batch, the model can naturally handle test sequences that are longer or shorter than the training sequences (up to reasonable limits imposed by memory or potential vanishing/exploding gradients for extremely long sequences).
3. **Information Loss:** Truncating sequences, especially long ones, can lead to a loss of potentially important information that might be located at the end of the sequence. By using padding and packing, we avoid this information loss for sequences within the limits the model can handle.

In summary, while truncation was necessary for the fixed-input FFNN in Task 2, the padding and packing strategy employed for the RNNs in Task 3 allows the model to work with the full, variable-length sequences from the test set without needing to truncate them, thus retaining all available sequential information.

Unlike the FFNN in Task 2, which requires a fixed-size input vector due to its rigid weight matrix, RNNs do not require test sequence truncation. This is due to the temporal weight-sharing nature of the architecture: the same weights are applied at each time step, allowing the model to process sequences of any length. While we use padding and packing to manage batches efficiently and prevent 'zero-pollution' of the hidden state, the underlying RNN logic naturally accommodates the full API trace. By avoiding truncation, we ensure that malicious behaviors occurring late in a sequence are not discarded, preserving the full context of the software's execution.

```
[87]: # Reusable training loop for RNN-like modules
def train_rnn_common(model, train_loader, val_loader, optimizer, criterion,
    epochs, min_delta=None, patience=None, scheduler=None):
    """
    A generic training loop for RNN-based models that supports early stopping.
    Now also tracks validation F1 macro score.
    """
    device = next(model.parameters()).device
```

```

best_val = math.inf
best_state = None
trigger = 0
history = {'train_loss': [], 'val_loss': [], 'val_f1_macro': []}
start_time = time.time()

for epoch in range(1, epochs+1):
    # --- Training Phase ---
    model.train()
    train_losses = []
    for X_batch, lengths, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        outputs = model(X_batch, lengths)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())
    train_loss = float(np.mean(train_losses)) if train_losses else 0.0

    # --- Validation Phase ---
    model.eval()
    val_losses = []
    all_preds = []
    all_targets = []
    with torch.no_grad():
        for X_batch, lengths, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            outputs = model(X_batch, lengths)
            loss = criterion(outputs, y_batch)
            val_losses.append(loss.item())

            # Collect predictions for F1 score
            probs = torch.sigmoid(outputs).view(-1)
            preds = (probs > 0.5).long()
            all_preds.extend(preds.cpu().tolist())
            all_targets.extend(y_batch.view(-1).cpu().tolist())

    val_loss = float(np.mean(val_losses)) if val_losses else 0.0
    val_f1_macro = f1_score(all_targets, all_preds, average='macro',
↪zero_division=0)

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_f1_macro'].append(val_f1_macro)

```

```

    # --- Scheduler Step ---
    if scheduler is not None:
        last_lr = optimizer.param_groups[0]['lr']
        if isinstance(scheduler, torch.optim.lr_scheduler.
↪ReduceLROnPlateau):
            scheduler.step(val_loss)
        else:
            scheduler.step()
        curr_lr = optimizer.param_groups[0]['lr']
        if curr_lr != last_lr:
            print(f"Epoch {epoch}: LR reduced from {last_lr:.6f} to_
↪{curr_lr:.6f}")

    if (epoch % 5 == 0) or (epoch == 1) or (epoch == epochs):
        print(f'Epoch {epoch}/{epochs} - Train Loss: {train_loss:.4f} -_
↪Val Loss: {val_loss:.4f} - Val F1: {val_f1_macro:.4f}')

    # --- Early Stopping ---
    if val_loss < best_val - (min_delta if min_delta is not None else 0):
        best_val = val_loss
        best_state = {k: v.cpu().clone() for k, v in model.state_dict().
↪items()}
        trigger = 0
    else:
        trigger += 1
        if trigger >= patience:
            print(f'Early stopping at epoch {epoch} (best val loss_
↪{best_val:.6f})')
            break

    elapsed = time.time() - start_time

    if best_state is not None:
        model.load_state_dict(best_state)

    return model, history, elapsed

```

```

[88]: # Prediction helper (binary classification threshold)
def predict_loader(model, loader):
    """
    Run the model on a DataLoader and return integer arrays for
    ground-truth labels and predicted labels (0/1).
    """
    # Set model to evaluation mode (e.g., disables dropout)
    model.eval()

    # Detect device from model parameters

```

```

device = next(model.parameters()).device

ys_true = [] # To store all true labels
ys_pred = [] # To store all predicted labels

# Disable gradient calculations to save memory and computation
with torch.no_grad():
    # Iterate over all batches in the data loader
    for X_batch, lengths, y_batch in loader:
        # Move batch to device
        X_batch = X_batch.to(device)

        # Get model outputs (logits)
        out = model(X_batch, lengths)

        # Convert logits to probabilities using sigmoid
        probs = torch.sigmoid(out).cpu().numpy().ravel()

        # Apply threshold for binary classification
        preds = (probs >= 0.5).astype(int)

        # Store the ground-truth and predicted labels from this batch
        ys_true.extend(y_batch.cpu().numpy().ravel().astype(int).tolist())
        ys_pred.extend(preds.tolist())

# Convert the lists of all labels/preds into final numpy arrays
return np.array(ys_true, dtype=int), np.array(ys_pred, dtype=int)

```

```

[89]: def plot_cm_task3(model, loader, model_name, save_dir=None):
    """
    Specific for RNNs. Uses your existing 'predict_loader' logic.
    """
    # Use the helper function you already defined in the notebook
    y_true, y_pred = predict_loader(model, loader)

    # Plot
    _internal_plot_cm(y_true, y_pred, model_name, save_dir)

```

```

[90]: # Function to train a model with a specific learning rate
def train_with_lr(model_class, model_params, initial_lr, train_loader,
    ↪ val_loader, criterion, num_epochs, min_delta, patience, scheduler_params,
    ↪ device):
    """
    Train a model with a specific initial learning rate and return the history.
    """
    # Create a fresh instance of the model
    model = model_class(**model_params).to(device)

```

```

# Create optimizer with the specified learning rate
optimizer = optim.AdamW(model.parameters(), lr=initial_lr)

# Create scheduler
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    **scheduler_params
)

# Train the model
model, history, elapsed = train_rnn_common(
    model,
    train_loader,
    val_loader,
    optimizer,
    criterion,
    epochs=num_epochs,
    min_delta=min_delta,
    patience=patience,
    scheduler=scheduler
)

return model, history, elapsed

```

Q: Is the RNN padding more memory efficient compared to the FFNN's one? Why? Yes, the padding method used with RNNs, particularly when combined with `pack_padded_sequence`, is generally **more memory efficient** than the padding required for the FFNN in Task 2. Here's the key difference:

1. **FFNN Padding (Task 2):** For the FFNN, each sequence needed to be padded or truncated to a *single, fixed maximum length* to create a uniform input tensor. This meant that even short sequences would occupy the same large memory footprint as the longest sequences after padding. This can be memory-intensive, especially when the distribution of sequence lengths is wide and the chosen fixed length is large (like the maximum length).
2. **RNN Padding with Packing (Task 3):** With RNNs and `pack_padded_sequence`, padding is still used to create batches of uniform shape, but the `pack_padded_sequence` function allows the RNN to effectively ignore the padded elements during computation. The RNN processes the actual data points sequentially based on their original lengths. This means that while the padded tensor exists, the *computations* and the resulting hidden states primarily reflect the actual length of each sequence, leading to more efficient memory usage during the core processing, especially within the RNN layers themselves.

In summary, the FFNN approach forces all inputs to the same large, fixed size, leading to potentially wasted memory for shorter sequences. The RNN with packing, while still using padding for batching, is more efficient in how it processes and uses memory for variable-length inputs by focusing computation only on the real data points.

```
[91]: # --- Variation 1: Simple one-directional RNN (nn.RNN) ---

class SimpleRNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=128, num_layers=2,
↳ num_classes=1, bidirectional=False, dropout=0.25, padding=0):
        super(SimpleRNNClassifier, self).__init__()

        # Embedding layer: Maps vocabulary indices (integers) to dense vectors
↳ (embeddings).
        # padding_idx=0 ensures the padding token doesn't contribute to
↳ gradients.
        self.embedding = nn.Embedding(vocab_size, embedding_dim=embed_dim,
↳ padding_idx=padding)

        # Simple RNN layer: Processes the embedded sequences.
        self.rnn = nn.RNN(input_size=embed_dim, hidden_size=hidden_size,
↳ num_layers=num_layers,
                                batch_first=True, nonlinearity='tanh',
↳ bidirectional=bidirectional)

        self.bidirectional = bidirectional
        fc_in = hidden_size * (2 if bidirectional else 1)

        # Final fully connected layer for classification.
        self.fc = nn.Linear(fc_in, num_classes)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, lengths):
        # Embed the input indices
        emb = self.embedding(x) # Shape: (batch_size, max_seq_len, embed_dim)
        emb = self.dropout(emb)

        # Pack the sequence to let the RNN ignore padding
        packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True,
↳ enforce_sorted=True)

        # Process the packed sequence through the RNN
        packed_out, hn = self.rnn(packed) # hn is the final hidden state(s)

        # hn shape: (num_layers * num_directions, batch_size, hidden_size)

        # Get the final hidden state to represent the whole sequence.
        if self.bidirectional:
            # Concatenate the final forward (hn[-2]) and backward (hn[-1])
↳ states
            h_last = torch.cat([hn[-2], hn[-1]], dim=1)

```

```

    else:
        # Use the final hidden state from the last layer (hn[-1])
        h_last = hn[-1]

        # Pass the final hidden state through the classifier
        out = self.fc(h_last)
        return out

```

```

[92]: # --- Variation 1: Simple Unidirectional RNN ---

print("\n" + "=" * 80)
print("Training Simple RNN with different learning rates")
print("=" * 80)

# Define learning rates to test
learning_rates = [2e-04, 1e-04, 8e-05, 5e-05]

# Model parameters
vocab_size_rnn = len(api_to_id)
simple_rnn_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': False,
    'dropout': 0.3
}

# Initialize early stopping parameters
min_delta = 0.00001
patience = 20

# Compute class balance from training subset and set pos_weight for
↳ BCEWithLogitsLoss
pos = sum(train_labels_sub) # number of positive (1) samples
neg = len(train_labels_sub) - pos

# Compute weights
pos_weight = compute_pos_weight(train_labels_sub).to(device)

criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

# Setup Scheduler
scheduler_params_simple = {
    'mode': 'min', # reduce when metric stops decreasing (val_loss)
    'factor': 0.5, # halve the LR when triggered

```

```

    'patience': 8, # wait 8 epochs with no improvement before cutting LR
    'min_lr': 1e-7
}

# Store histories and models for each LR
histories_simple = {}
models_simple = {}
elapsed_simple = {}

for lr in learning_rates:
    print(f"\n--- Training Simple RNN with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=SimpleRNNClassifier,
        model_params=simple_rnn_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_simple,
        device=device
    )
    histories_simple[lr] = history
    models_simple[lr] = model
    elapsed_simple[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final_
    ↪val loss: {history['val_loss'][-1]:.4f}")

```

```

=====
Training Simple RNN with different learning rates
=====

```

```

Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000

```

```

--- Training Simple RNN with lr=2e-04 ---
Epoch 1/150 - Train Loss: 0.0961 - Val Loss: 0.0875 - Val F1: 0.5845
Epoch 5/150 - Train Loss: 0.0752 - Val Loss: 0.0806 - Val F1: 0.6176
Epoch 10/150 - Train Loss: 0.0686 - Val Loss: 0.0749 - Val F1: 0.6540
Epoch 15/150 - Train Loss: 0.0599 - Val Loss: 0.0764 - Val F1: 0.6713
Epoch 20/150 - Train Loss: 0.0580 - Val Loss: 0.0690 - Val F1: 0.6889
Epoch 25/150 - Train Loss: 0.0516 - Val Loss: 0.0699 - Val F1: 0.6834
Epoch 27: LR reduced from 0.000200 to 0.000100
Epoch 30/150 - Train Loss: 0.0469 - Val Loss: 0.0677 - Val F1: 0.6936
Epoch 35/150 - Train Loss: 0.0436 - Val Loss: 0.0673 - Val F1: 0.7087

```

Epoch 40/150 - Train Loss: 0.0411 - Val Loss: 0.0692 - Val F1: 0.7217
 Epoch 45/150 - Train Loss: 0.0397 - Val Loss: 0.0674 - Val F1: 0.7247
 Epoch 50/150 - Train Loss: 0.0375 - Val Loss: 0.0682 - Val F1: 0.7267
 Epoch 52: LR reduced from 0.000100 to 0.000050
 Epoch 55/150 - Train Loss: 0.0341 - Val Loss: 0.0656 - Val F1: 0.7134
 Epoch 60/150 - Train Loss: 0.0311 - Val Loss: 0.0656 - Val F1: 0.7384
 Epoch 65/150 - Train Loss: 0.0298 - Val Loss: 0.0639 - Val F1: 0.7380
 Epoch 68: LR reduced from 0.000050 to 0.000025
 Epoch 70/150 - Train Loss: 0.0282 - Val Loss: 0.0682 - Val F1: 0.7340
 Epoch 75/150 - Train Loss: 0.0287 - Val Loss: 0.0682 - Val F1: 0.7257
 Epoch 77: LR reduced from 0.000025 to 0.000013
 Early stopping at epoch 79 (best val loss 0.062561)
 Completed. Training time: 00:03:32 (212.16s) - Final val loss: 0.0646

--- Training Simple RNN with lr=1e-04 ---

Epoch 1/150 - Train Loss: 0.1027 - Val Loss: 0.0863 - Val F1: 0.5572
 Epoch 5/150 - Train Loss: 0.0783 - Val Loss: 0.0785 - Val F1: 0.6424
 Epoch 10/150 - Train Loss: 0.0710 - Val Loss: 0.0731 - Val F1: 0.6526
 Epoch 15/150 - Train Loss: 0.0668 - Val Loss: 0.0717 - Val F1: 0.6509
 Epoch 20/150 - Train Loss: 0.0640 - Val Loss: 0.0724 - Val F1: 0.6815
 Epoch 25/150 - Train Loss: 0.0607 - Val Loss: 0.0754 - Val F1: 0.6409
 Epoch 30: LR reduced from 0.000100 to 0.000050
 Epoch 30/150 - Train Loss: 0.0573 - Val Loss: 0.0742 - Val F1: 0.6897
 Epoch 35/150 - Train Loss: 0.0550 - Val Loss: 0.0687 - Val F1: 0.6781
 Epoch 40/150 - Train Loss: 0.0539 - Val Loss: 0.0694 - Val F1: 0.6937
 Epoch 45/150 - Train Loss: 0.0527 - Val Loss: 0.0670 - Val F1: 0.6734
 Epoch 50/150 - Train Loss: 0.0510 - Val Loss: 0.0691 - Val F1: 0.7030
 Epoch 54: LR reduced from 0.000050 to 0.000025
 Epoch 55/150 - Train Loss: 0.0484 - Val Loss: 0.0703 - Val F1: 0.7086
 Epoch 60/150 - Train Loss: 0.0472 - Val Loss: 0.0712 - Val F1: 0.7138
 Epoch 63: LR reduced from 0.000025 to 0.000013
 Epoch 65/150 - Train Loss: 0.0462 - Val Loss: 0.0713 - Val F1: 0.7078
 Early stopping at epoch 65 (best val loss 0.066977)
 Completed. Training time: 00:02:45 (165.81s) - Final val loss: 0.0713

--- Training Simple RNN with lr=8e-05 ---

Epoch 1/150 - Train Loss: 0.1035 - Val Loss: 0.0844 - Val F1: 0.5814
 Epoch 5/150 - Train Loss: 0.0779 - Val Loss: 0.0745 - Val F1: 0.6674
 Epoch 10/150 - Train Loss: 0.0705 - Val Loss: 0.0781 - Val F1: 0.6575
 Epoch 15/150 - Train Loss: 0.0661 - Val Loss: 0.0766 - Val F1: 0.6948
 Epoch 20/150 - Train Loss: 0.0639 - Val Loss: 0.0742 - Val F1: 0.6413
 Epoch 25/150 - Train Loss: 0.0606 - Val Loss: 0.0682 - Val F1: 0.6711
 Epoch 30/150 - Train Loss: 0.0586 - Val Loss: 0.0725 - Val F1: 0.6342
 Epoch 35/150 - Train Loss: 0.0553 - Val Loss: 0.0761 - Val F1: 0.7001
 Epoch 40: LR reduced from 0.000080 to 0.000040
 Epoch 40/150 - Train Loss: 0.0523 - Val Loss: 0.0723 - Val F1: 0.6918
 Epoch 45/150 - Train Loss: 0.0477 - Val Loss: 0.0614 - Val F1: 0.7223
 Epoch 50/150 - Train Loss: 0.0448 - Val Loss: 0.0589 - Val F1: 0.7042

Epoch 55/150 - Train Loss: 0.0444 - Val Loss: 0.0575 - Val F1: 0.7523
 Epoch 56: LR reduced from 0.000040 to 0.000020
 Epoch 60/150 - Train Loss: 0.0431 - Val Loss: 0.0639 - Val F1: 0.7077
 Epoch 65/150 - Train Loss: 0.0417 - Val Loss: 0.0591 - Val F1: 0.7192
 Epoch 70/150 - Train Loss: 0.0395 - Val Loss: 0.0642 - Val F1: 0.6834
 Epoch 75/150 - Train Loss: 0.0403 - Val Loss: 0.0600 - Val F1: 0.7060
 Epoch 80: LR reduced from 0.000020 to 0.000010
 Epoch 80/150 - Train Loss: 0.0388 - Val Loss: 0.0562 - Val F1: 0.7361
 Epoch 85/150 - Train Loss: 0.0390 - Val Loss: 0.0579 - Val F1: 0.7121
 Epoch 89: LR reduced from 0.000010 to 0.000005
 Epoch 90/150 - Train Loss: 0.0387 - Val Loss: 0.0588 - Val F1: 0.7253
 Early stopping at epoch 91 (best val loss 0.054692)
 Completed. Training time: 00:03:52 (232.38s) - Final val loss: 0.0568

--- Training Simple RNN with lr=5e-05 ---

Epoch 1/150 - Train Loss: 0.1076 - Val Loss: 0.0914 - Val F1: 0.4902
 Epoch 5/150 - Train Loss: 0.0819 - Val Loss: 0.0792 - Val F1: 0.6081
 Epoch 10/150 - Train Loss: 0.0750 - Val Loss: 0.0752 - Val F1: 0.6353
 Epoch 15/150 - Train Loss: 0.0710 - Val Loss: 0.0709 - Val F1: 0.6483
 Epoch 20/150 - Train Loss: 0.0667 - Val Loss: 0.0687 - Val F1: 0.6631
 Epoch 25/150 - Train Loss: 0.0625 - Val Loss: 0.0659 - Val F1: 0.6576
 Epoch 30/150 - Train Loss: 0.0615 - Val Loss: 0.0757 - Val F1: 0.6655
 Epoch 35/150 - Train Loss: 0.0609 - Val Loss: 0.0789 - Val F1: 0.6569
 Epoch 40/150 - Train Loss: 0.0609 - Val Loss: 0.0610 - Val F1: 0.6666
 Epoch 45/150 - Train Loss: 0.0563 - Val Loss: 0.0639 - Val F1: 0.7024
 Epoch 49: LR reduced from 0.000050 to 0.000025
 Epoch 50/150 - Train Loss: 0.0541 - Val Loss: 0.0615 - Val F1: 0.6555
 Epoch 55/150 - Train Loss: 0.0521 - Val Loss: 0.0649 - Val F1: 0.6701
 Epoch 60/150 - Train Loss: 0.0527 - Val Loss: 0.0653 - Val F1: 0.6397
 Epoch 63: LR reduced from 0.000025 to 0.000013
 Epoch 65/150 - Train Loss: 0.0491 - Val Loss: 0.0611 - Val F1: 0.6689
 Epoch 70/150 - Train Loss: 0.0497 - Val Loss: 0.0598 - Val F1: 0.6783
 Epoch 75/150 - Train Loss: 0.0492 - Val Loss: 0.0653 - Val F1: 0.6459
 Epoch 80: LR reduced from 0.000013 to 0.000006
 Epoch 80/150 - Train Loss: 0.0479 - Val Loss: 0.0627 - Val F1: 0.6544
 Epoch 85/150 - Train Loss: 0.0480 - Val Loss: 0.0634 - Val F1: 0.6560
 Epoch 89: LR reduced from 0.000006 to 0.000003
 Epoch 90/150 - Train Loss: 0.0469 - Val Loss: 0.0639 - Val F1: 0.6432
 Early stopping at epoch 91 (best val loss 0.059344)
 Completed. Training time: 00:04:02 (242.82s) - Final val loss: 0.0625

[93]: # --- Plot loss curves ---

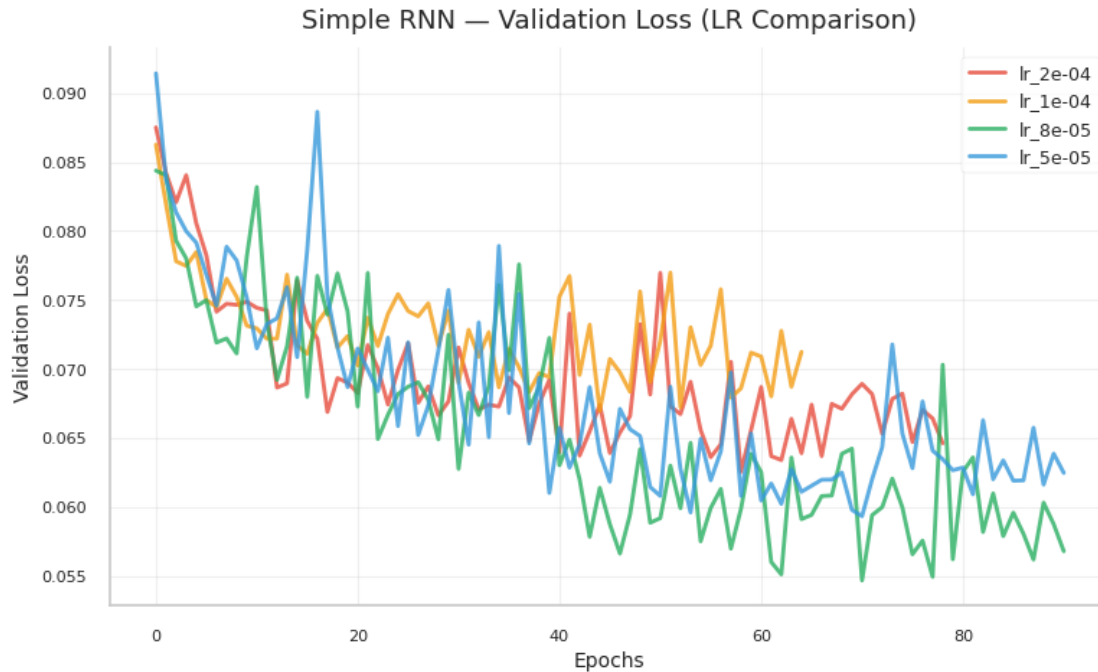
Plot comparison

plot_lr_comparison(histories_simple, "Simple RNN", save_dir)

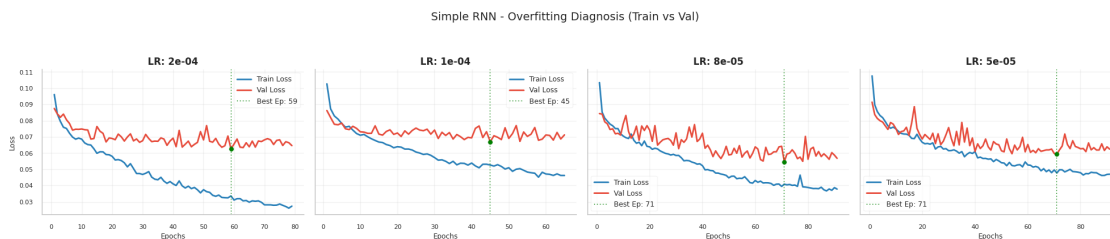
#Plot Train vs Validation

```
plot_train_val_grid(histories_simple, "Simple RNN", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/simple_rnn_lr_comparison.png

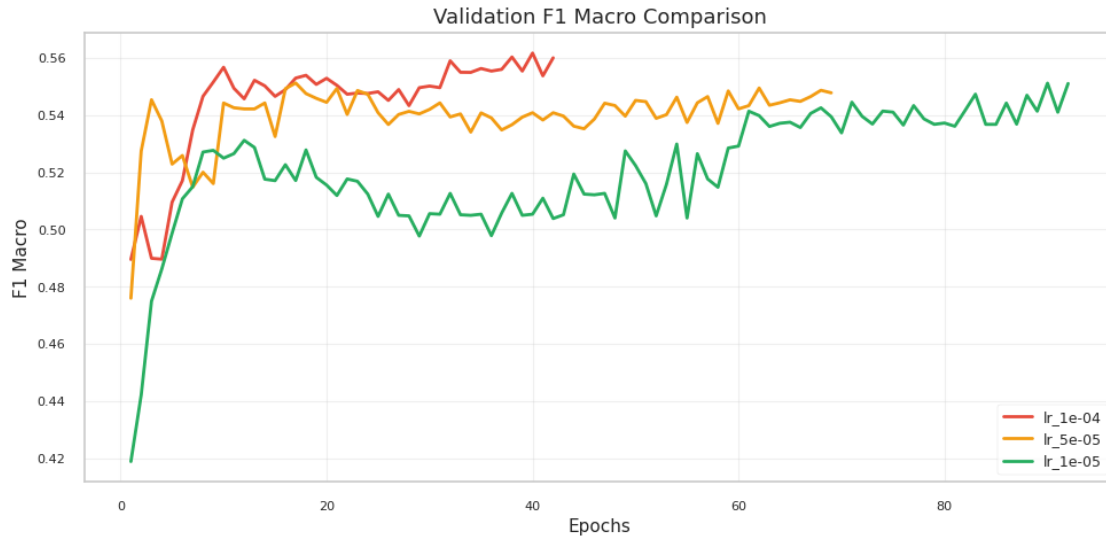


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/simple_rnn_diagnosis.png



```
[94]: # --- Plot Validation F1 Macro for Simple RNN ---
plot_training_metrics(histories_seq, save_dir=save_dir,
    plot_name="simple_rnn_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/simple_rnn_f1_comparison.png



```
[95]: # --- Evaluate validation and test sets for ALL learning rates ---
print("\n" + "=" * 80)
print("Evaluation for ALL Simple RNN learning rates")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]

    print(f"\n{'='*60}")
    print(f"Simple RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (Simple RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,
    ↪zero_division=0))

    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
    print(f'Test report (Simple RNN - lr={lr:.0e}):')
    print(classification_report(y_test_true, y_test_pred, digits=4,
    ↪zero_division=0))

    # Find and highlight best model
    best_lr_simple = min(histories_simple.items(), key=lambda x:
    ↪min(x[1]['val_loss']))
    print(f"\n{'='*60}")
```

```

print(f" Best Simple RNN - LR: {best_lr_simple[0]:.0e} (Min Val Loss:␣
↪{min(best_lr_simple[1]['val_loss']):.4f})")
print(f"{' '*60}")

```

```

=====
Evaluation for ALL Simple RNN learning rates
=====

```

```

=====
Simple RNN with lr=2e-04
=====

```

Validation report (Simple RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.4046	0.5738	0.4746	122
1	0.9832	0.9672	0.9751	3143
accuracy			0.9525	3265
macro avg	0.6939	0.7705	0.7249	3265
weighted avg	0.9616	0.9525	0.9564	3265

Test report (Simple RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.4049	0.6132	0.4877	243
1	0.9847	0.9650	0.9748	6262
accuracy			0.9519	6505
macro avg	0.6948	0.7891	0.7312	6505
weighted avg	0.9630	0.9519	0.9566	6505

```

=====
Simple RNN with lr=1e-04
=====

```

Validation report (Simple RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.2946	0.5410	0.3815	122
1	0.9816	0.9497	0.9654	3143
accuracy			0.9345	3265
macro avg	0.6381	0.7454	0.6734	3265
weighted avg	0.9559	0.9345	0.9436	3265

Test report (Simple RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.2612	0.5021	0.3437	243
1	0.9800	0.9449	0.9621	6262
accuracy			0.9284	6505
macro avg	0.6206	0.7235	0.6529	6505
weighted avg	0.9531	0.9284	0.9390	6505

=====

Simple RNN with lr=8e-05

=====

Validation report (Simple RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.3874	0.7049	0.5000	122
1	0.9882	0.9567	0.9722	3143
accuracy			0.9473	3265
macro avg	0.6878	0.8308	0.7361	3265
weighted avg	0.9657	0.9473	0.9546	3265

Test report (Simple RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.3443	0.6461	0.4492	243
1	0.9858	0.9523	0.9687	6262
accuracy			0.9408	6505
macro avg	0.6650	0.7992	0.7090	6505
weighted avg	0.9618	0.9408	0.9493	6505

=====

Simple RNN with lr=5e-05

=====

Validation report (Simple RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.3128	0.5820	0.4069	122
1	0.9832	0.9504	0.9665	3143
accuracy			0.9366	3265
macro avg	0.6480	0.7662	0.6867	3265

weighted avg	0.9582	0.9366	0.9456	3265
--------------	--------	--------	--------	------

Test report (Simple RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.3010	0.6255	0.4064	243
1	0.9848	0.9436	0.9638	6262
accuracy			0.9317	6505
macro avg	0.6429	0.7846	0.6851	6505
weighted avg	0.9593	0.9317	0.9430	6505

=====
Best Simple RNN - LR: 8e-05 (Min Val Loss: 0.0547)
=====

```
[96]: # --- Plot confusion matrices for ALL learning rates - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL Simple RNN learning rates - VALIDATION")
print("=" * 80)

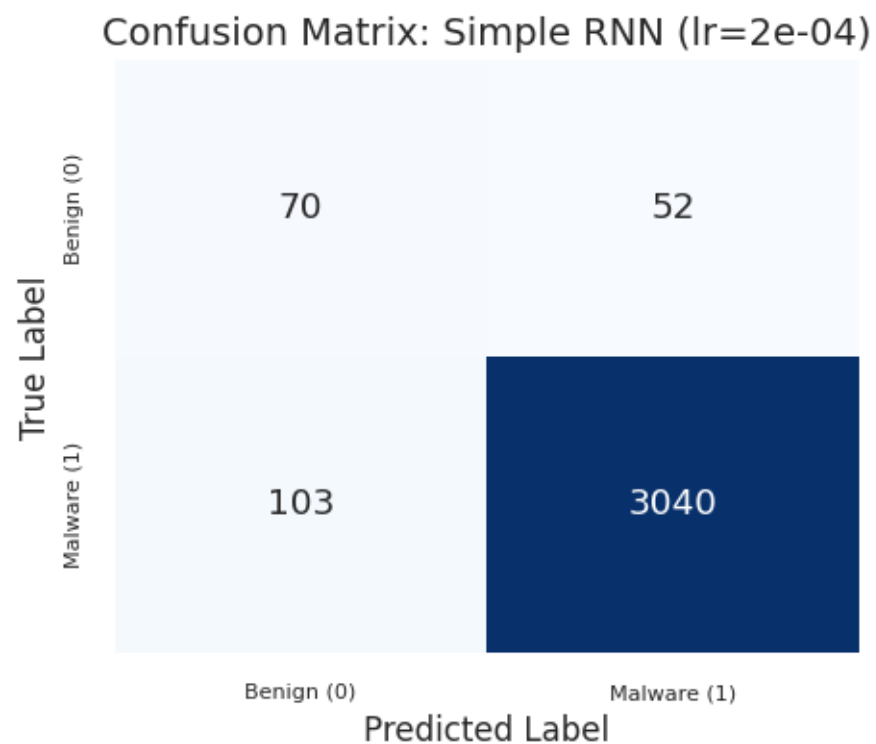
for lr in learning_rates:
    model = models_simple[lr]
    print(f"\nGenerating confusion matrix for Simple RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"Simple RNN_
↪(lr={lr:.0e})", save_dir=save_dir)
```

=====
Confusion matrices for ALL Simple RNN learning rates - VALIDATION
=====

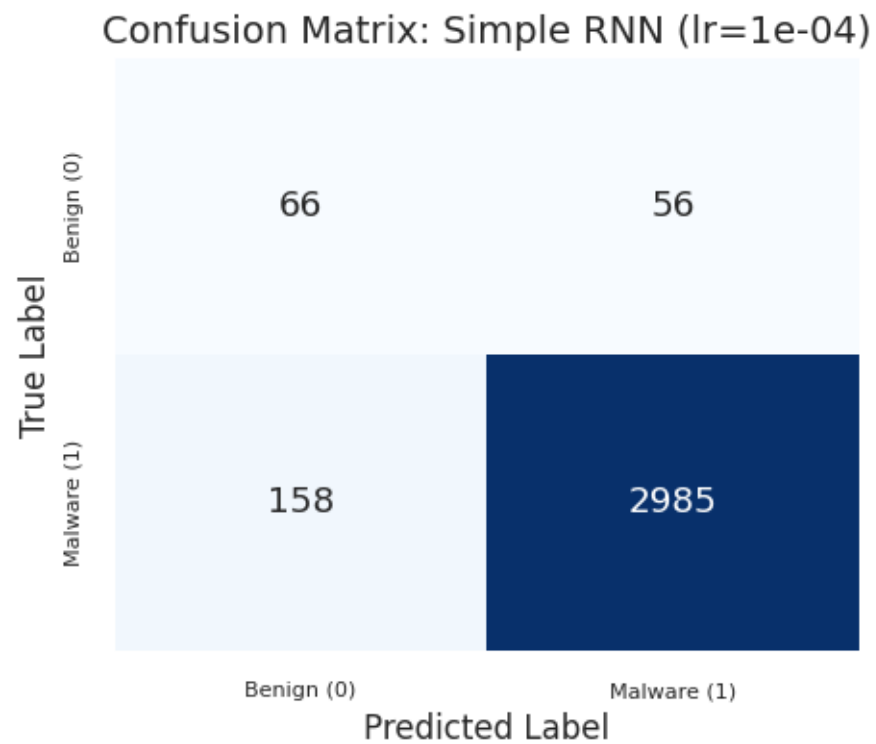
Generating confusion matrix for Simple RNN with lr=2e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=2e_04).png



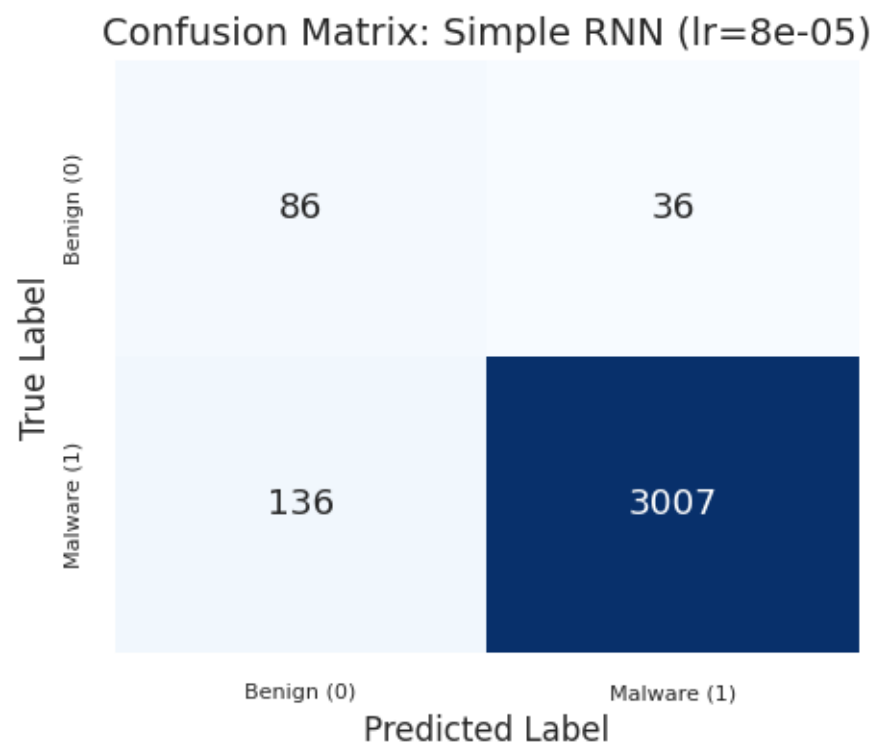
Generating confusion matrix for Simple RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=1e_04).png



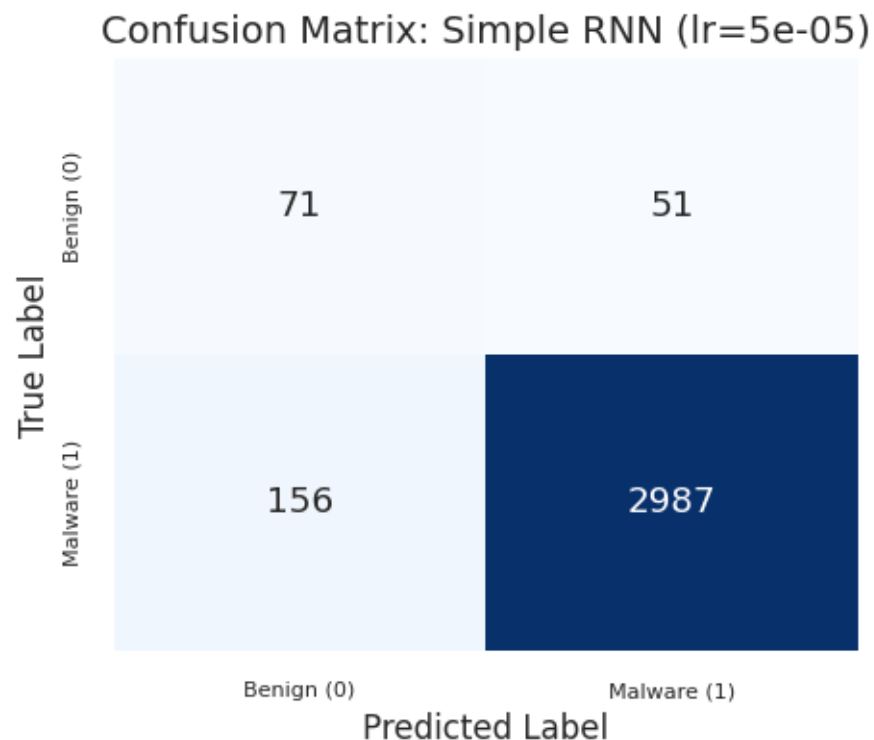
Generating confusion matrix for Simple RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=8e_05).png



Generating confusion matrix for Simple RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=5e_05).png



```
[97]: # --- Plot confusion matrices for ALL learning rates - TEST ---

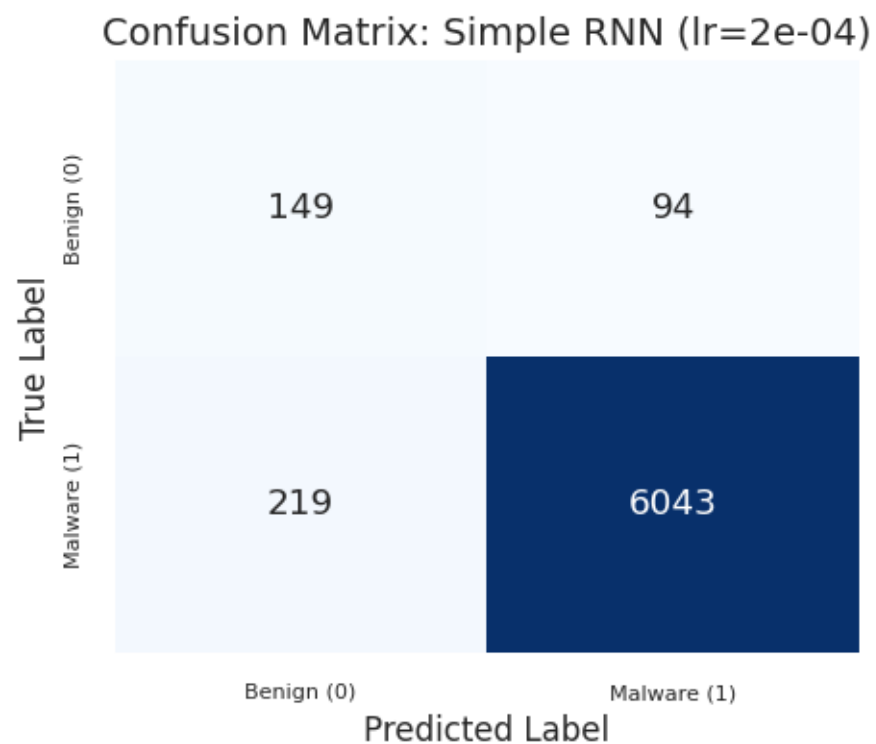
print("\n" + "=" * 80)
print("Confusion matrices for ALL Simple RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]
    print(f"\nGenerating confusion matrix for Simple RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"Simple RNN_
↵(lr={lr:.0e})", save_dir=save_dir)
```

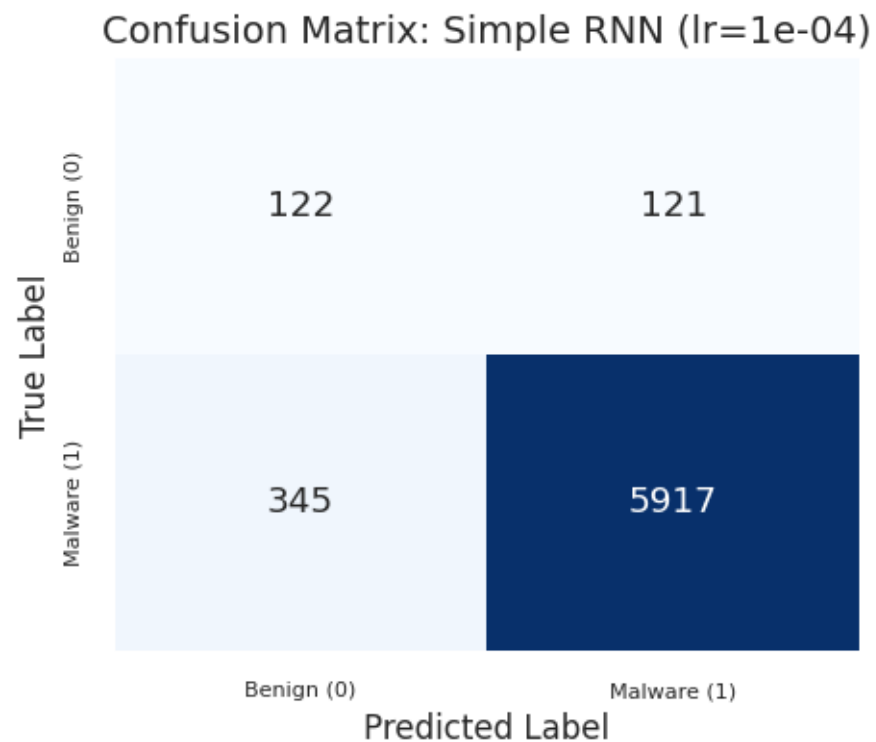
```
=====
Confusion matrices for ALL Simple RNN learning rates - TEST
=====
```

```
Generating confusion matrix for Simple RNN with lr=2e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=2e_04).png
```



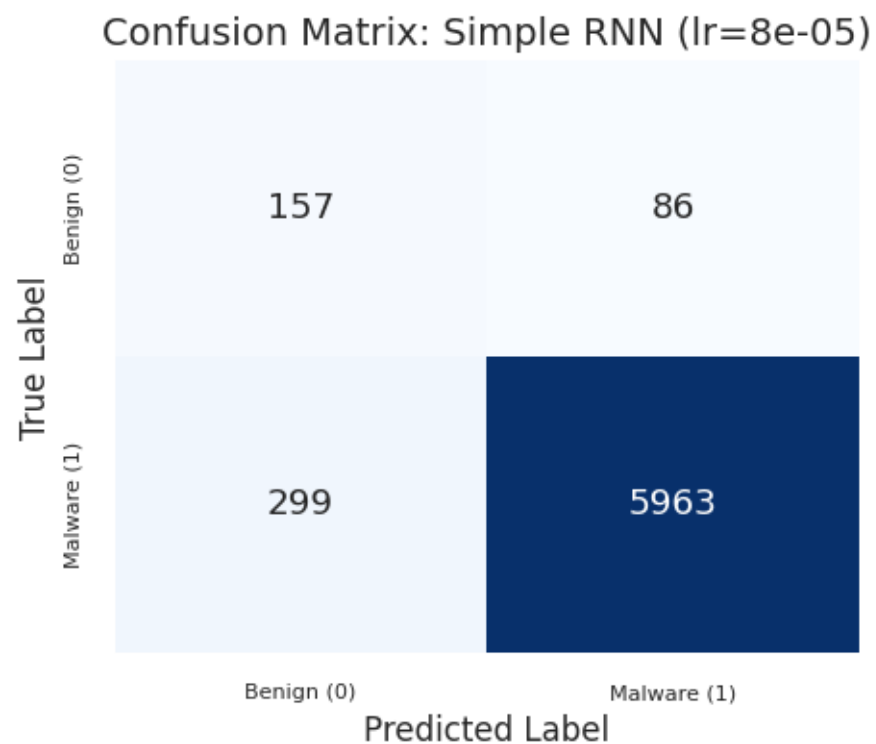
Generating confusion matrix for Simple RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=1e_04).png



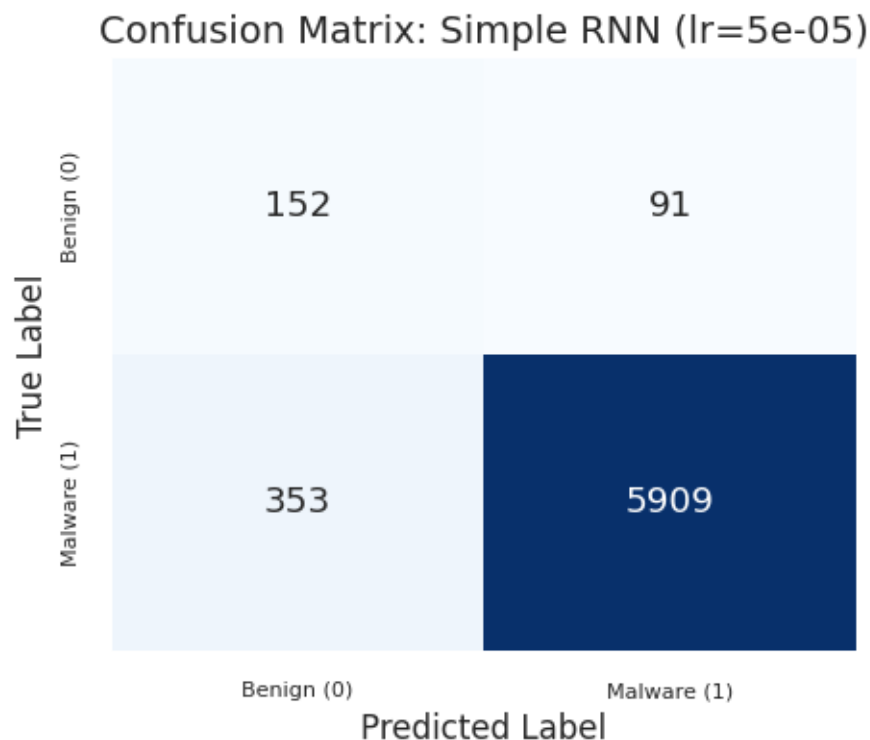
Generating confusion matrix for Simple RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=8e_05).png



Generating confusion matrix for Simple RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=5e_05).png



Q: Start with a simple one-directional RNN. Is your network as fast as the FFNN? If not, why? Based on the training times recorded in the notebook:

- **FFNN (Sequential IDs)** training time: 00:01:24 — 84.52s seconds
- **FFNN (Learnable Embeddings)** training time: 00:01:36 96.88s— 96.88 seconds
- **Simple RNN** training time: 00:05:04 — 304 seconds

The simple one-directional RNN is **significantly slower** than both FFNN variations.

Here's why RNNs are typically slower than FFNNs for processing sequences:

1. **Sequential Processing:** RNNs process sequences step-by-step, one element at a time. The computation for the current time step depends on the output of the previous time step. This inherent sequential nature makes it difficult to parallelize computations across the time dimension, which is a major bottleneck, especially for long sequences.
2. **FFNN Parallelization:** FFNNs, on the other hand, process the entire input vector (the padded sequence in our case) in a single pass through its layers. The computations within each layer are independent across the elements of the input and can be highly parallelized across the batch and feature dimensions, making them much faster on hardware like GPUs.
3. **Backpropagation Through Time (BPTT):** During the training phase, RNNs utilize BPTT to update weights. This requires storing all intermediate hidden states for each time step to compute gradients through the entire chain of the sequence. This process is not only computationally expensive but also significantly more memory-intensive than the standard backpropagation used in FFNNs.

Even with techniques like `pack_padded_sequence` to optimize RNN training by avoiding computation on padding, the fundamental sequential dependency limits the extent of parallelization possible compared to the dense matrix multiplications in an FFNN.

1.4.2 Network Variations

```
[98]: # --- Variation 2: Bidirectional RNN ---

print("\n" + "=" * 80)
print("Training Bidirectional RNN with different learning rates")
print("=" * 80)

# Define learning rates to test
learning_rates = [3e-04, 1e-04, 8e-05, 5e-05]

# Model parameters
bi_rnn_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': True,
    'dropout': 0.35
}

# Initialize early stopping parameters
min_delta = 0.00001
patience = 20

# Scheduler parameters
scheduler_params_bi = {
    'mode': 'min',
    'factor': 0.7,
    'patience': 7,
    'min_lr': 1e-7
}

# Store histories and models for each LR
histories_bi = {}
models_bi = {}
elapsed_bi = {}

for lr in learning_rates:
    print(f"\n--- Training Bi-RNN with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=SimpleRNClassifier,
```

```

        model_params=bi_rnn_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_bi,
        device=device
    )
    histories_bi[lr] = history
    models_bi[lr] = model
    elapsed_bi[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final_
↪val loss: {history['val_loss'][-1]:.4f}")

```

```

=====
Training Bidirectional RNN with different learning rates
=====

```

```

--- Training Bi-RNN with lr=3e-04 ---

```

```

Epoch 1/150 - Train Loss: 0.0957 - Val Loss: 0.0834 - Val F1: 0.6242
Epoch 5/150 - Train Loss: 0.0711 - Val Loss: 0.0729 - Val F1: 0.6627
Epoch 10/150 - Train Loss: 0.0654 - Val Loss: 0.0696 - Val F1: 0.6738
Epoch 15/150 - Train Loss: 0.0533 - Val Loss: 0.0648 - Val F1: 0.7010
Epoch 20/150 - Train Loss: 0.0517 - Val Loss: 0.0649 - Val F1: 0.7258
Epoch 25/150 - Train Loss: 0.0410 - Val Loss: 0.0616 - Val F1: 0.6945
Epoch 29: LR reduced from 0.000300 to 0.000210
Epoch 30/150 - Train Loss: 0.0322 - Val Loss: 0.0655 - Val F1: 0.7276
Epoch 35/150 - Train Loss: 0.0273 - Val Loss: 0.0608 - Val F1: 0.7326
Epoch 39: LR reduced from 0.000210 to 0.000147
Epoch 40/150 - Train Loss: 0.0227 - Val Loss: 0.0638 - Val F1: 0.7405
Epoch 45/150 - Train Loss: 0.0200 - Val Loss: 0.0620 - Val F1: 0.7489
Epoch 47: LR reduced from 0.000147 to 0.000103
Epoch 50/150 - Train Loss: 0.0165 - Val Loss: 0.0770 - Val F1: 0.7477
Early stopping at epoch 51 (best val loss 0.057965)
Completed. Training time: 00:03:14 (194.22s) - Final val loss: 0.0720

```

```

--- Training Bi-RNN with lr=1e-04 ---

```

```

Epoch 1/150 - Train Loss: 0.1002 - Val Loss: 0.0872 - Val F1: 0.5424
Epoch 5/150 - Train Loss: 0.0731 - Val Loss: 0.0735 - Val F1: 0.6482
Epoch 10/150 - Train Loss: 0.0661 - Val Loss: 0.0708 - Val F1: 0.6382
Epoch 15/150 - Train Loss: 0.0610 - Val Loss: 0.0717 - Val F1: 0.6307
Epoch 20/150 - Train Loss: 0.0560 - Val Loss: 0.0638 - Val F1: 0.6794
Epoch 25/150 - Train Loss: 0.0584 - Val Loss: 0.0656 - Val F1: 0.6484

```

Epoch 30/150 - Train Loss: 0.0487 - Val Loss: 0.0613 - Val F1: 0.7066
 Epoch 35/150 - Train Loss: 0.0454 - Val Loss: 0.0631 - Val F1: 0.7134
 Epoch 40/150 - Train Loss: 0.0448 - Val Loss: 0.0587 - Val F1: 0.7043
 Epoch 45/150 - Train Loss: 0.0399 - Val Loss: 0.0582 - Val F1: 0.7111
 Epoch 50/150 - Train Loss: 0.0368 - Val Loss: 0.0580 - Val F1: 0.7333
 Epoch 55/150 - Train Loss: 0.0330 - Val Loss: 0.0602 - Val F1: 0.7378
 Epoch 57: LR reduced from 0.000100 to 0.000070
 Epoch 60/150 - Train Loss: 0.0284 - Val Loss: 0.0570 - Val F1: 0.7467
 Epoch 65/150 - Train Loss: 0.0273 - Val Loss: 0.0598 - Val F1: 0.7512
 Epoch 70/150 - Train Loss: 0.0264 - Val Loss: 0.0629 - Val F1: 0.7579
 Epoch 71: LR reduced from 0.000070 to 0.000049
 Epoch 75/150 - Train Loss: 0.0237 - Val Loss: 0.0590 - Val F1: 0.7474
 Epoch 79: LR reduced from 0.000049 to 0.000034
 Epoch 80/150 - Train Loss: 0.0221 - Val Loss: 0.0568 - Val F1: 0.7351
 Early stopping at epoch 83 (best val loss 0.053808)
 Completed. Training time: 00:05:12 (312.56s) - Final val loss: 0.0598

--- Training Bi-RNN with lr=8e-05 ---

Epoch 1/150 - Train Loss: 0.1018 - Val Loss: 0.0885 - Val F1: 0.5107
 Epoch 5/150 - Train Loss: 0.0767 - Val Loss: 0.0761 - Val F1: 0.6601
 Epoch 10/150 - Train Loss: 0.0699 - Val Loss: 0.0705 - Val F1: 0.6862
 Epoch 15/150 - Train Loss: 0.0671 - Val Loss: 0.0711 - Val F1: 0.6615
 Epoch 20/150 - Train Loss: 0.0614 - Val Loss: 0.0671 - Val F1: 0.6677
 Epoch 22: LR reduced from 0.000080 to 0.000056
 Epoch 25/150 - Train Loss: 0.0580 - Val Loss: 0.0721 - Val F1: 0.6771
 Epoch 30/150 - Train Loss: 0.0555 - Val Loss: 0.0667 - Val F1: 0.6874
 Epoch 35/150 - Train Loss: 0.0544 - Val Loss: 0.0676 - Val F1: 0.6801
 Epoch 40/150 - Train Loss: 0.0524 - Val Loss: 0.0645 - Val F1: 0.6815
 Epoch 45/150 - Train Loss: 0.0513 - Val Loss: 0.0712 - Val F1: 0.6942
 Epoch 50/150 - Train Loss: 0.0479 - Val Loss: 0.0684 - Val F1: 0.6796
 Epoch 52: LR reduced from 0.000056 to 0.000039
 Epoch 55/150 - Train Loss: 0.0448 - Val Loss: 0.0710 - Val F1: 0.7025
 Epoch 60: LR reduced from 0.000039 to 0.000027
 Epoch 60/150 - Train Loss: 0.0431 - Val Loss: 0.0644 - Val F1: 0.7083
 Early stopping at epoch 64 (best val loss 0.059639)
 Completed. Training time: 00:04:01 (241.53s) - Final val loss: 0.0663

--- Training Bi-RNN with lr=5e-05 ---

Epoch 1/150 - Train Loss: 0.1047 - Val Loss: 0.0923 - Val F1: 0.4904
 Epoch 5/150 - Train Loss: 0.0796 - Val Loss: 0.0802 - Val F1: 0.6169
 Epoch 10/150 - Train Loss: 0.0756 - Val Loss: 0.0765 - Val F1: 0.6266
 Epoch 15/150 - Train Loss: 0.0701 - Val Loss: 0.0700 - Val F1: 0.6452
 Epoch 20/150 - Train Loss: 0.0645 - Val Loss: 0.0651 - Val F1: 0.6543
 Epoch 25/150 - Train Loss: 0.0629 - Val Loss: 0.0619 - Val F1: 0.6642
 Epoch 30/150 - Train Loss: 0.0599 - Val Loss: 0.0632 - Val F1: 0.6458
 Epoch 31: LR reduced from 0.000050 to 0.000035
 Epoch 35/150 - Train Loss: 0.0552 - Val Loss: 0.0581 - Val F1: 0.6788
 Epoch 40/150 - Train Loss: 0.0554 - Val Loss: 0.0543 - Val F1: 0.6995

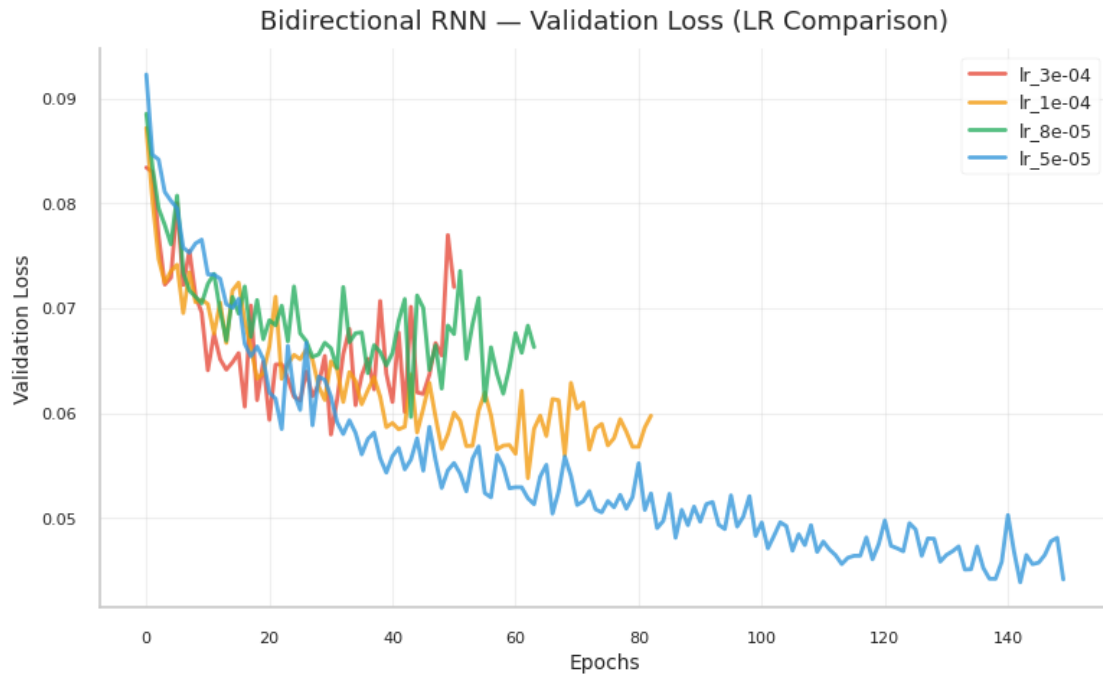
Epoch 45/150 - Train Loss: 0.0536 - Val Loss: 0.0576 - Val F1: 0.6638
Epoch 48: LR reduced from 0.000035 to 0.000024
Epoch 50/150 - Train Loss: 0.0482 - Val Loss: 0.0546 - Val F1: 0.6852
Epoch 55/150 - Train Loss: 0.0475 - Val Loss: 0.0568 - Val F1: 0.7006
Epoch 60/150 - Train Loss: 0.0483 - Val Loss: 0.0528 - Val F1: 0.7141
Epoch 65/150 - Train Loss: 0.0434 - Val Loss: 0.0539 - Val F1: 0.7039
Epoch 70/150 - Train Loss: 0.0434 - Val Loss: 0.0540 - Val F1: 0.7266
Epoch 75: LR reduced from 0.000024 to 0.000017
Epoch 75/150 - Train Loss: 0.0414 - Val Loss: 0.0505 - Val F1: 0.7337
Epoch 80/150 - Train Loss: 0.0402 - Val Loss: 0.0520 - Val F1: 0.7000
Epoch 83: LR reduced from 0.000017 to 0.000012
Epoch 85/150 - Train Loss: 0.0399 - Val Loss: 0.0497 - Val F1: 0.7074
Epoch 90/150 - Train Loss: 0.0393 - Val Loss: 0.0511 - Val F1: 0.6921
Epoch 95: LR reduced from 0.000012 to 0.000008
Epoch 95/150 - Train Loss: 0.0367 - Val Loss: 0.0490 - Val F1: 0.7030
Epoch 100/150 - Train Loss: 0.0373 - Val Loss: 0.0483 - Val F1: 0.7148
Epoch 105/150 - Train Loss: 0.0339 - Val Loss: 0.0492 - Val F1: 0.7090
Epoch 110/150 - Train Loss: 0.0348 - Val Loss: 0.0468 - Val F1: 0.7133
Epoch 115/150 - Train Loss: 0.0344 - Val Loss: 0.0462 - Val F1: 0.7192
Epoch 120/150 - Train Loss: 0.0348 - Val Loss: 0.0475 - Val F1: 0.7220
Epoch 122: LR reduced from 0.000008 to 0.000006
Epoch 125/150 - Train Loss: 0.0328 - Val Loss: 0.0495 - Val F1: 0.7242
Epoch 130: LR reduced from 0.000006 to 0.000004
Epoch 130/150 - Train Loss: 0.0330 - Val Loss: 0.0458 - Val F1: 0.7321
Epoch 135/150 - Train Loss: 0.0321 - Val Loss: 0.0451 - Val F1: 0.7199
Epoch 140/150 - Train Loss: 0.0318 - Val Loss: 0.0458 - Val F1: 0.7177
Epoch 145/150 - Train Loss: 0.0320 - Val Loss: 0.0456 - Val F1: 0.7199
Epoch 150/150 - Train Loss: 0.0310 - Val Loss: 0.0441 - Val F1: 0.7270
Completed. Training time: 00:09:36 (576.60s) - Final val loss: 0.0441

[99]: `# --- Plot loss curves ---`

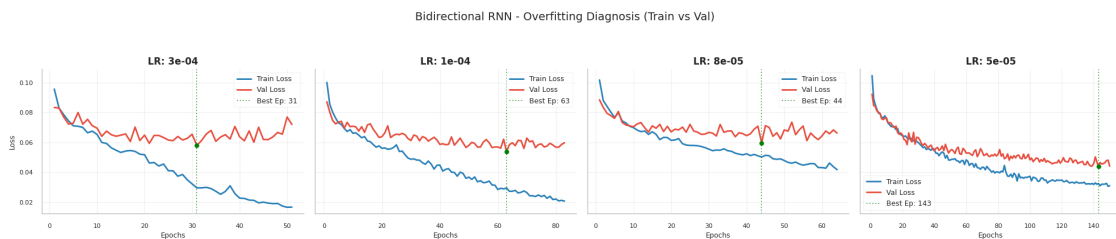
```
# Plot comparison
plot_lr_comparison(histories_bi, "Bidirectional RNN", save_dir)

#Plot Train vs Validation
plot_train_val_grid(histories_bi, "Bidirectional RNN", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/bidirectional_rnn_lr_comparison.png

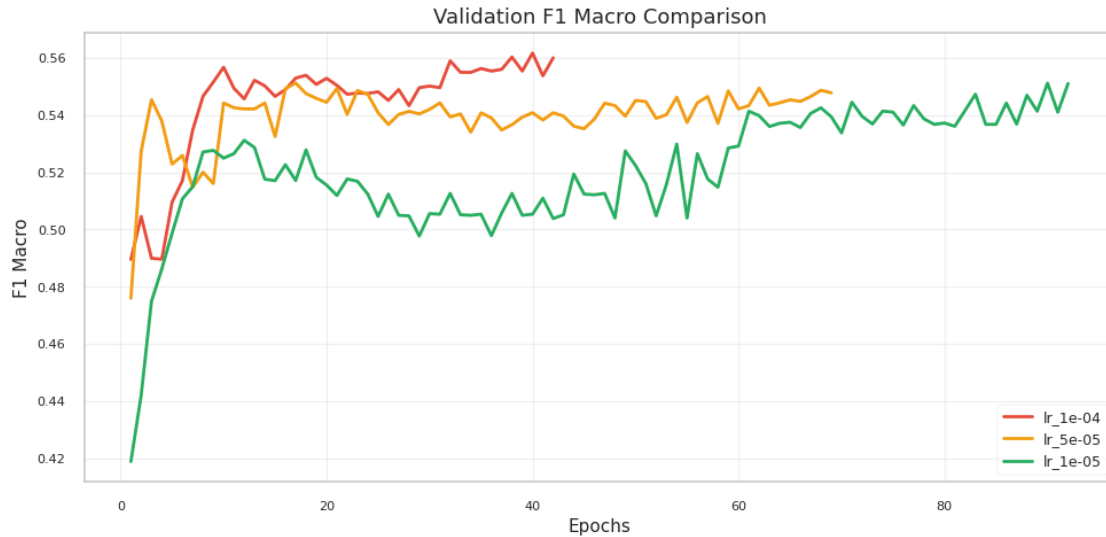


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/bidirectional_rnn_diagnosis.png



```
[100]: # --- Plot Validation F1 Macro for Bidirectional RNN ---
plot_training_metrics(histories_seq, save_dir=save_dir,
    plot_name="bidirectional_rnn_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/bidirectional_rnn_f1_comparison.png



```
[101]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL Bi-RNN learning rates")
print("=" * 80)

for lr in learning_rates:
    model = models_bi[lr]

    print(f"\n{'='*60}")
    print(f"Bi-RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (Bi-RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,
    ↪zero_division=0))

    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
    print(f'Test report (Bi-RNN - lr={lr:.0e}):')
    print(classification_report(y_test_true, y_test_pred, digits=4,
    ↪zero_division=0))

    # Find and highlight best model
    best_lr_bi = min(histories_bi.items(), key=lambda x: min(x[1]['val_loss']))
    print(f"\n{'='*60}")
```

```

print(f" Best Bi-RNN - LR: {best_lr_bi[0]:.0e} (Min Val Loss:␣
↪{min(best_lr_bi[1]['val_loss']):.4f})")
print(f"{' '*60}")

```

```

=====
Evaluation for ALL Bi-RNN learning rates
=====

```

```

=====
Bi-RNN with lr=3e-04
=====

```

Validation report (Bi-RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.3895	0.6066	0.4744	122
1	0.9844	0.9631	0.9736	3143
accuracy			0.9498	3265
macro avg	0.6869	0.7848	0.7240	3265
weighted avg	0.9622	0.9498	0.9550	3265

Test report (Bi-RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.3162	0.5556	0.4030	243
1	0.9822	0.9534	0.9676	6262
accuracy			0.9385	6505
macro avg	0.6492	0.7545	0.6853	6505
weighted avg	0.9573	0.9385	0.9465	6505

```

=====
Bi-RNN with lr=1e-04
=====

```

Validation report (Bi-RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.3687	0.5984	0.4562	122
1	0.9840	0.9602	0.9720	3143
accuracy			0.9467	3265
macro avg	0.6764	0.7793	0.7141	3265
weighted avg	0.9610	0.9467	0.9527	3265

Test report (Bi-RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.3744	0.6132	0.4649	243
1	0.9846	0.9602	0.9723	6262
accuracy			0.9473	6505
macro avg	0.6795	0.7867	0.7186	6505
weighted avg	0.9618	0.9473	0.9533	6505

=====

Bi-RNN with lr=8e-05

=====

Validation report (Bi-RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.3390	0.4918	0.4013	122
1	0.9799	0.9628	0.9713	3143
accuracy			0.9452	3265
macro avg	0.6595	0.7273	0.6863	3265
weighted avg	0.9560	0.9452	0.9500	3265

Test report (Bi-RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.3582	0.5144	0.4223	243
1	0.9808	0.9642	0.9725	6262
accuracy			0.9474	6505
macro avg	0.6695	0.7393	0.6974	6505
weighted avg	0.9576	0.9474	0.9519	6505

=====

Bi-RNN with lr=5e-05

=====

Validation report (Bi-RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.3667	0.7213	0.4862	122
1	0.9888	0.9516	0.9698	3143
accuracy			0.9430	3265
macro avg	0.6777	0.8365	0.7280	3265

weighted avg 0.9655 0.9430 0.9518 3265

Test report (Bi-RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.3649	0.7449	0.4899	243
1	0.9897	0.9497	0.9693	6262
accuracy			0.9420	6505
macro avg	0.6773	0.8473	0.7296	6505
weighted avg	0.9663	0.9420	0.9514	6505

=====
Best Bi-RNN - LR: 5e-05 (Min Val Loss: 0.0439)
=====

```
[102]: # --- Plot confusion matrices for ALL learning rates - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL Bi-RNN learning rates - VALIDATION")
print("=" * 80)

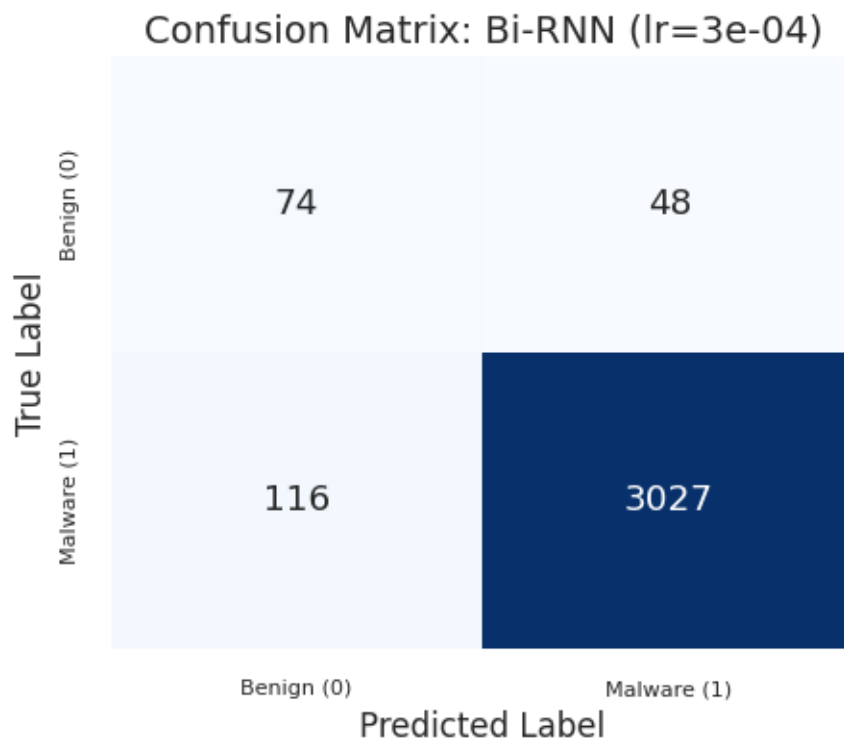
for lr in learning_rates:
    model = models_bi[lr]
    print(f"\nGenerating confusion matrix for Bi-RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"Bi-RNN (lr={lr:.
↪0e})", save_dir=save_dir)
```

=====
Confusion matrices for ALL Bi-RNN learning rates - VALIDATION
=====

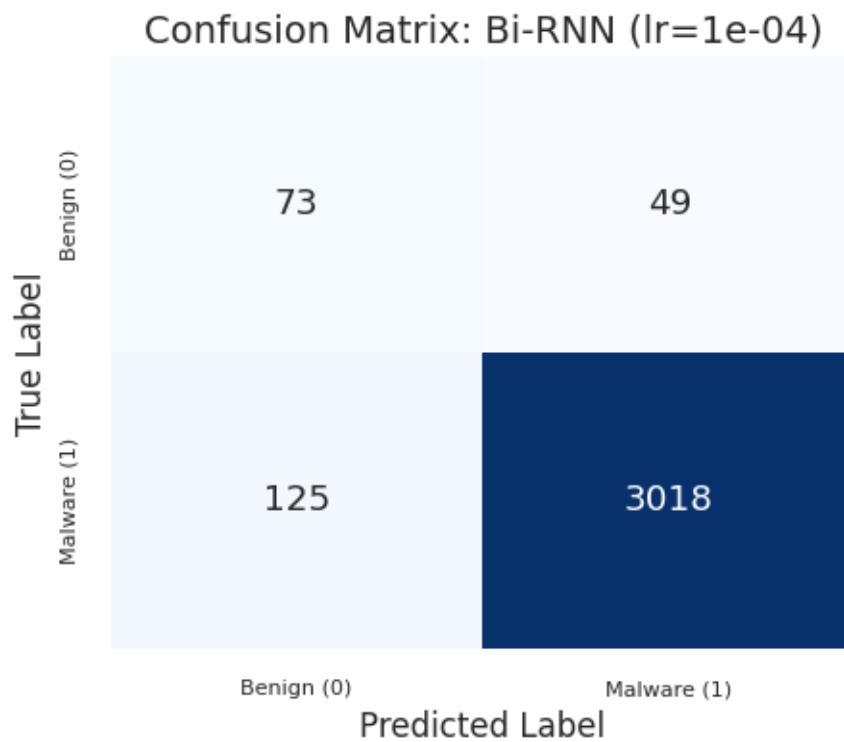
Generating confusion matrix for Bi-RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=3e_04).png



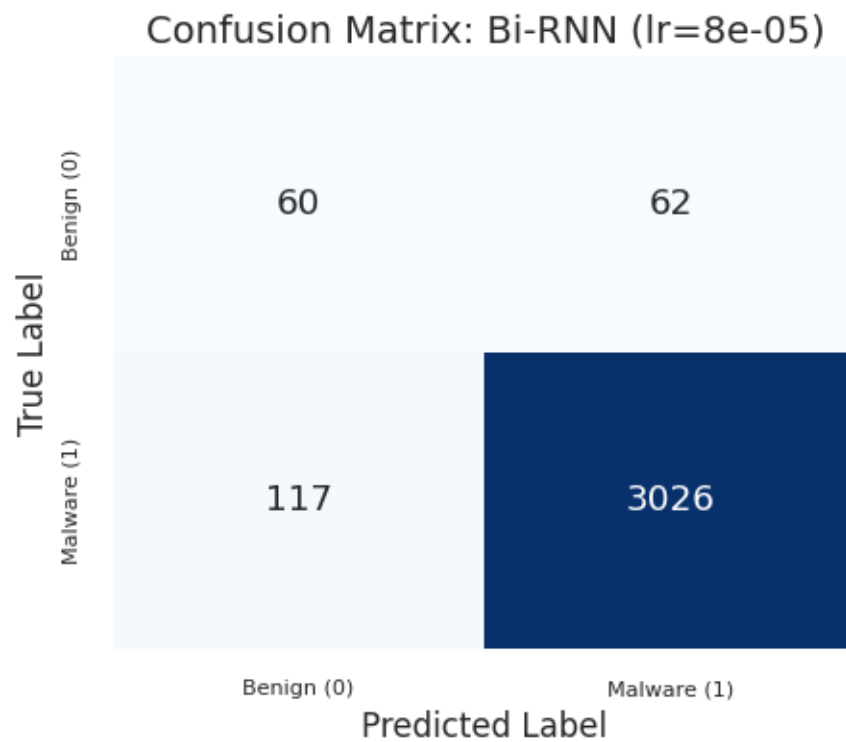
Generating confusion matrix for Bi-RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn(lr=1e_04).png



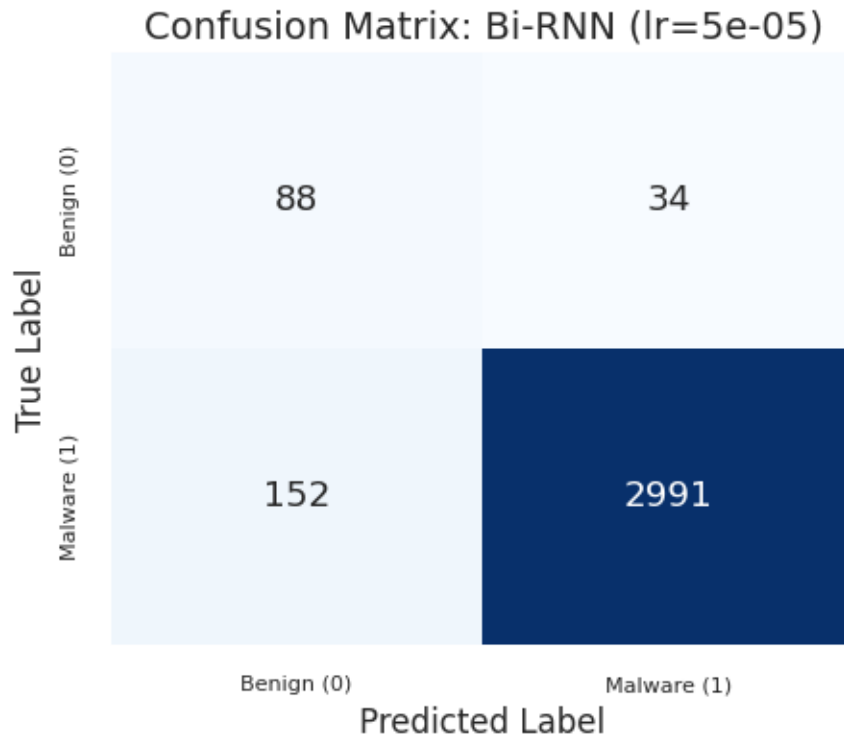
Generating confusion matrix for Bi-RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=8e_05).png



Generating confusion matrix for Bi-RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=5e_05).png



```
[103]: # --- Plot confusion matrices for ALL learning rates - TEST ---

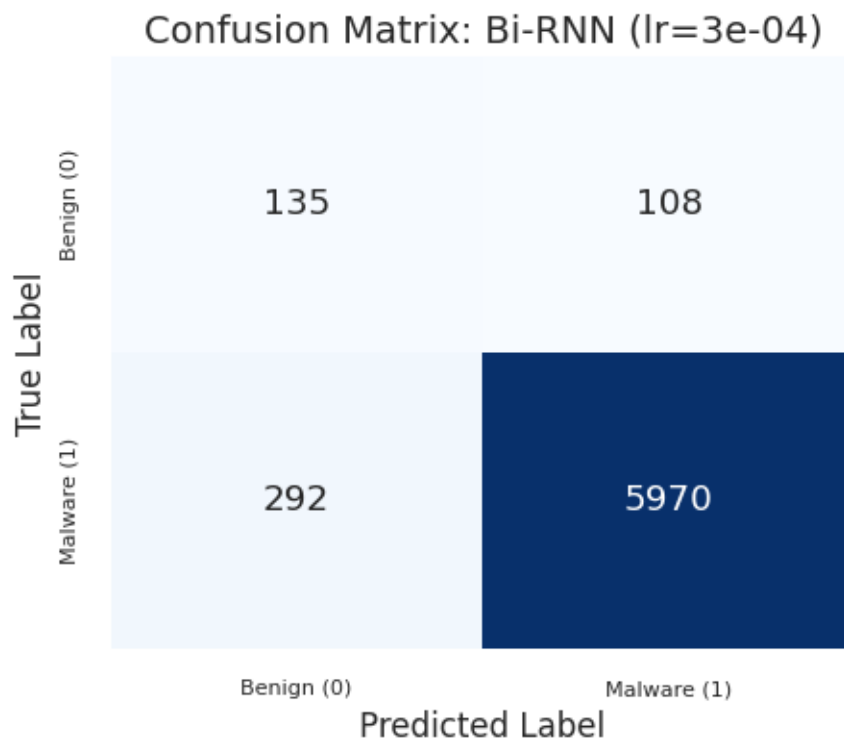
print("\n" + "=" * 80)
print("Confusion matrices for ALL Bi-RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates:
    model = models_bi[lr]
    print(f"\nGenerating confusion matrix for Bi-RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"Bi-RNN (lr={lr:.0e})", save_dir=save_dir)
```

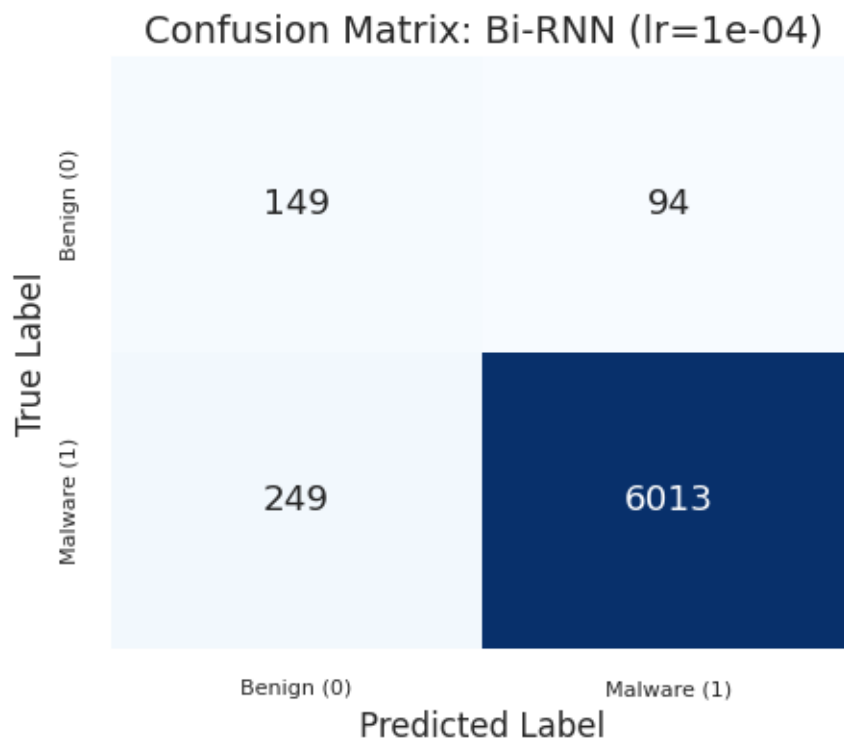
```
=====
Confusion matrices for ALL Bi-RNN learning rates - TEST
=====

Generating confusion matrix for Bi-RNN with lr=3e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=3e_04).png
```



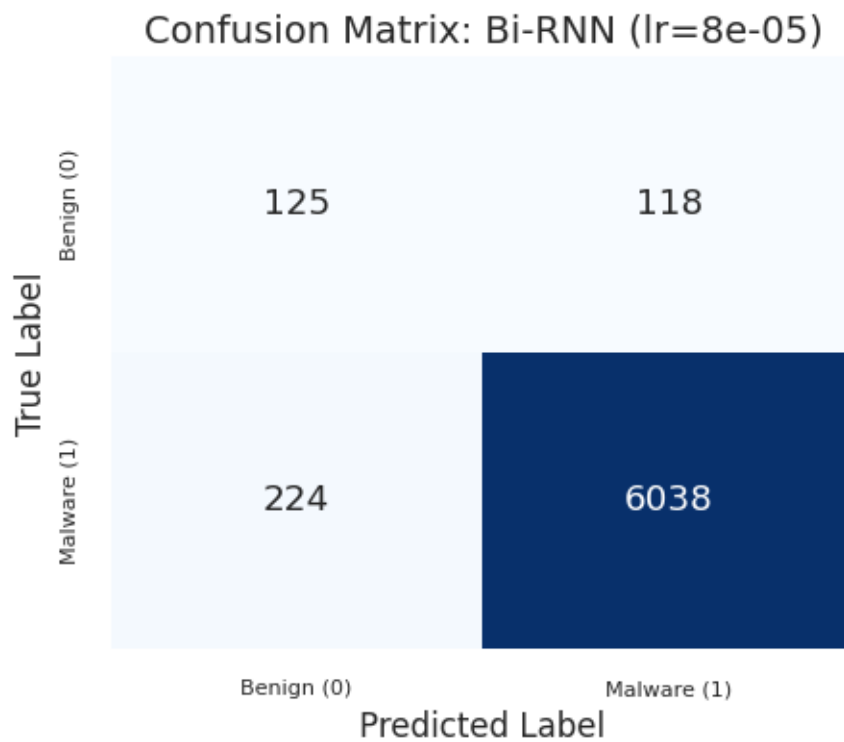
Generating confusion matrix for Bi-RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn(lr=1e_04).png



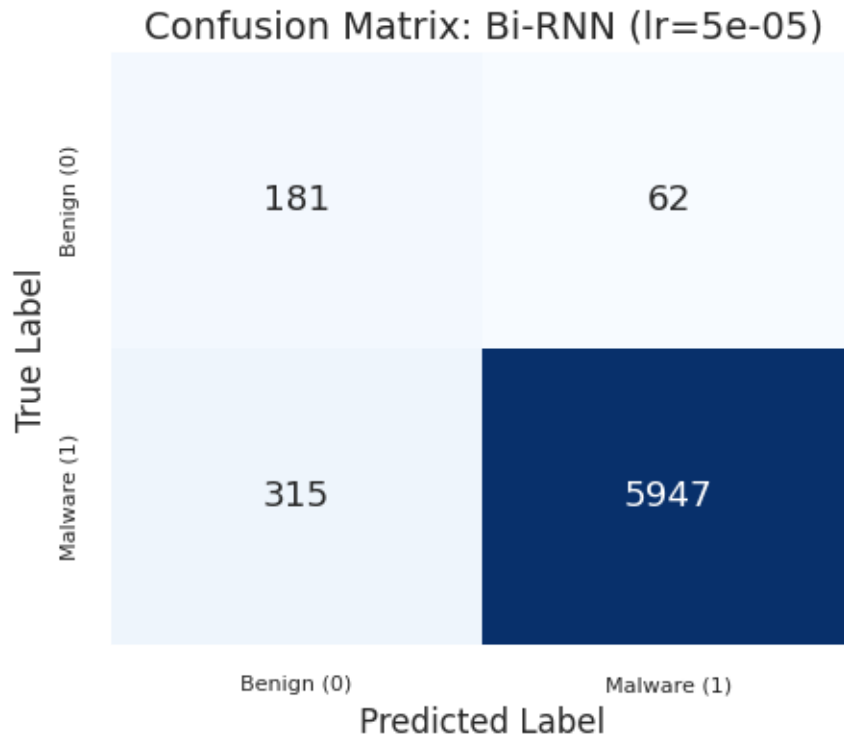
Generating confusion matrix for Bi-RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=8e_05).png



Generating confusion matrix for Bi-RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=5e_05).png



```
[104]: # --- Variation 3: LSTM (bidirectional) ---

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=128, num_layers=2,
        ↪ num_classes=1, bidirectional=True, dropout=0.3, padding=0):
        super(LSTMClassifier, self).__init__()

        # Embedding layer: Maps API call indices to dense vectors. padding_idx
        ↪ is set.
        self.embedding = nn.Embedding(vocab_size, embedding_dim=embed_dim,
        ↪ padding_idx=padding)

        # LSTM layer: A more complex RNN unit that helps prevent vanishing
        ↪ gradients.
        # batch_first=True expects input shape (batch_size, seq_len, features).
        self.lstm = nn.LSTM(input_size=embed_dim, hidden_size=hidden_size,
        ↪ num_layers=num_layers,
                               batch_first=True, bidirectional=bidirectional)

        self.bidirectional = bidirectional
        fc_in = hidden_size * (2 if bidirectional else 1)
```

```

self.fc = nn.Linear(fc_in, num_classes)
self.dropout = nn.Dropout(dropout)

def forward(self, x, lengths):
    # Embed the input indices
    emb = self.embedding(x) # (batch_size, max_seq_len, embed_dim)
    emb = self.dropout(emb)

    # Pack the padded sequence to efficiently process variable lengths
    packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True,
    ↪enforce_sorted=True)

    # LSTM returns packed_output, (final_hidden_state, final_cell_state)
    packed_out, (hn, cn) = self.lstm(packed)

    # hn shape: (num_layers * num_directions, batch_size, hidden_size)

    # Get the final hidden state from the last layer
    if self.bidirectional:
        # Concatenate the final forward state (hn[-2]) and backward state
    ↪(hn[-1])
        h_last = torch.cat([hn[-2], hn[-1]], dim=1)
    else:
        # Use the final hidden state of the last layer (hn[-1])
        h_last = hn[-1]

    # Pass the final hidden state through the classifier
    out = self.fc(h_last)
    return out

```

```

[105]: # --- Variation 3: LSTM (bidirectional) ---

print("\n" + "=" * 80)
print("Training LSTM with different learning rates")
print("=" * 80)

# Model parameters
lstm_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': True,
    'dropout': 0.4
}

```

```

# Initialize early stopping parameters
min_delta = 0.00001
patience = 30

# Scheduler parameters
scheduler_params_lstm = {
    'mode': 'min',
    'factor': 0.5,
    'patience': 6,
    'min_lr': 1e-7
}

# Test slightly different LRs for LSTM (typically handles higher LRs better)
learning_rates_lstm = [5e-04, 3e-04, 1e-04, 8e-05]

# Store histories and models for each LR
histories_lstm = {}
models_lstm = {}
elapsed_lstm = {}

for lr in learning_rates_lstm:
    print(f"\n--- Training LSTM with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=LSTMClassifier,
        model_params=lstm_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_lstm,
        device=device
    )
    histories_lstm[lr] = history
    models_lstm[lr] = model
    elapsed_lstm[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final_
↪val loss: {history['val_loss'][-1]:.4f}")

```

```

=====
Training LSTM with different learning rates
=====

```

```

--- Training LSTM with lr=5e-04 ---

```

Epoch 1/150 - Train Loss: 0.0927 - Val Loss: 0.0786 - Val F1: 0.5924
 Epoch 5/150 - Train Loss: 0.0734 - Val Loss: 0.0720 - Val F1: 0.6416
 Epoch 10/150 - Train Loss: 0.0653 - Val Loss: 0.0602 - Val F1: 0.7217
 Epoch 15/150 - Train Loss: 0.0497 - Val Loss: 0.0482 - Val F1: 0.7129
 Epoch 20/150 - Train Loss: 0.0441 - Val Loss: 0.0517 - Val F1: 0.7026
 Epoch 25/150 - Train Loss: 0.0318 - Val Loss: 0.0386 - Val F1: 0.8028
 Epoch 30/150 - Train Loss: 0.0262 - Val Loss: 0.0357 - Val F1: 0.7804
 Epoch 35/150 - Train Loss: 0.0175 - Val Loss: 0.0376 - Val F1: 0.8146
 Epoch 40/150 - Train Loss: 0.0163 - Val Loss: 0.0336 - Val F1: 0.8148
 Epoch 45/150 - Train Loss: 0.0124 - Val Loss: 0.0425 - Val F1: 0.8197
 Epoch 47: LR reduced from 0.000500 to 0.000250
 Epoch 50/150 - Train Loss: 0.0082 - Val Loss: 0.0396 - Val F1: 0.8438
 Epoch 54: LR reduced from 0.000250 to 0.000125
 Epoch 55/150 - Train Loss: 0.0065 - Val Loss: 0.0371 - Val F1: 0.8476
 Epoch 60/150 - Train Loss: 0.0057 - Val Loss: 0.0393 - Val F1: 0.8424
 Epoch 61: LR reduced from 0.000125 to 0.000063
 Epoch 65/150 - Train Loss: 0.0054 - Val Loss: 0.0440 - Val F1: 0.8592
 Epoch 68: LR reduced from 0.000063 to 0.000031
 Epoch 70/150 - Train Loss: 0.0047 - Val Loss: 0.0464 - Val F1: 0.8525
 Early stopping at epoch 70 (best val loss 0.033648)
 Completed. Training time: 00:04:59 (299.56s) - Final val loss: 0.0464

--- Training LSTM with lr=3e-04 ---

Epoch 1/150 - Train Loss: 0.0955 - Val Loss: 0.0811 - Val F1: 0.5287
 Epoch 5/150 - Train Loss: 0.0728 - Val Loss: 0.0661 - Val F1: 0.6529
 Epoch 10/150 - Train Loss: 0.0602 - Val Loss: 0.0633 - Val F1: 0.7076
 Epoch 15/150 - Train Loss: 0.0520 - Val Loss: 0.0615 - Val F1: 0.7079
 Epoch 20/150 - Train Loss: 0.0447 - Val Loss: 0.0533 - Val F1: 0.7284
 Epoch 25/150 - Train Loss: 0.0395 - Val Loss: 0.0519 - Val F1: 0.6948
 Epoch 30/150 - Train Loss: 0.0333 - Val Loss: 0.0521 - Val F1: 0.7475
 Epoch 35/150 - Train Loss: 0.0301 - Val Loss: 0.0439 - Val F1: 0.7670
 Epoch 40/150 - Train Loss: 0.0249 - Val Loss: 0.0453 - Val F1: 0.7919
 Epoch 42: LR reduced from 0.000300 to 0.000150
 Epoch 45/150 - Train Loss: 0.0182 - Val Loss: 0.0438 - Val F1: 0.7809
 Epoch 50/150 - Train Loss: 0.0179 - Val Loss: 0.0411 - Val F1: 0.8075
 Epoch 55/150 - Train Loss: 0.0147 - Val Loss: 0.0402 - Val F1: 0.8100
 Epoch 60/150 - Train Loss: 0.0136 - Val Loss: 0.0496 - Val F1: 0.8143
 Epoch 63: LR reduced from 0.000150 to 0.000075
 Epoch 65/150 - Train Loss: 0.0110 - Val Loss: 0.0442 - Val F1: 0.8325
 Epoch 70: LR reduced from 0.000075 to 0.000037
 Epoch 70/150 - Train Loss: 0.0107 - Val Loss: 0.0445 - Val F1: 0.8318
 Epoch 75/150 - Train Loss: 0.0100 - Val Loss: 0.0448 - Val F1: 0.8197
 Epoch 77: LR reduced from 0.000037 to 0.000019
 Epoch 80/150 - Train Loss: 0.0087 - Val Loss: 0.0461 - Val F1: 0.8262
 Epoch 84: LR reduced from 0.000019 to 0.000009
 Epoch 85/150 - Train Loss: 0.0083 - Val Loss: 0.0470 - Val F1: 0.8222
 Early stopping at epoch 86 (best val loss 0.037377)
 Completed. Training time: 00:06:10 (370.09s) - Final val loss: 0.0484

--- Training LSTM with lr=1e-04 ---

Epoch 1/150 - Train Loss: 0.1095 - Val Loss: 0.0887 - Val F1: 0.4905
Epoch 5/150 - Train Loss: 0.0748 - Val Loss: 0.0724 - Val F1: 0.6212
Epoch 10/150 - Train Loss: 0.0652 - Val Loss: 0.0651 - Val F1: 0.6519
Epoch 15/150 - Train Loss: 0.0576 - Val Loss: 0.0603 - Val F1: 0.6627
Epoch 20/150 - Train Loss: 0.0543 - Val Loss: 0.0605 - Val F1: 0.7027
Epoch 25/150 - Train Loss: 0.0484 - Val Loss: 0.0564 - Val F1: 0.6677
Epoch 30/150 - Train Loss: 0.0459 - Val Loss: 0.0539 - Val F1: 0.6939
Epoch 35/150 - Train Loss: 0.0438 - Val Loss: 0.0506 - Val F1: 0.7378
Epoch 40/150 - Train Loss: 0.0383 - Val Loss: 0.0505 - Val F1: 0.7548
Epoch 45/150 - Train Loss: 0.0356 - Val Loss: 0.0453 - Val F1: 0.7717
Epoch 50/150 - Train Loss: 0.0368 - Val Loss: 0.0509 - Val F1: 0.7665
Epoch 55/150 - Train Loss: 0.0309 - Val Loss: 0.0455 - Val F1: 0.7821
Epoch 60: LR reduced from 0.000100 to 0.000050
Epoch 60/150 - Train Loss: 0.0326 - Val Loss: 0.0439 - Val F1: 0.7720
Epoch 65/150 - Train Loss: 0.0274 - Val Loss: 0.0407 - Val F1: 0.7864
Epoch 67: LR reduced from 0.000050 to 0.000025
Epoch 70/150 - Train Loss: 0.0263 - Val Loss: 0.0382 - Val F1: 0.7981
Epoch 75/150 - Train Loss: 0.0241 - Val Loss: 0.0415 - Val F1: 0.7888
Epoch 77: LR reduced from 0.000025 to 0.000013
Epoch 80/150 - Train Loss: 0.0237 - Val Loss: 0.0381 - Val F1: 0.7985
Epoch 85/150 - Train Loss: 0.0233 - Val Loss: 0.0373 - Val F1: 0.7861
Epoch 90/150 - Train Loss: 0.0228 - Val Loss: 0.0406 - Val F1: 0.8134
Epoch 92: LR reduced from 0.000013 to 0.000006
Epoch 95/150 - Train Loss: 0.0223 - Val Loss: 0.0396 - Val F1: 0.8061
Epoch 99: LR reduced from 0.000006 to 0.000003
Epoch 100/150 - Train Loss: 0.0239 - Val Loss: 0.0395 - Val F1: 0.8025
Epoch 105/150 - Train Loss: 0.0218 - Val Loss: 0.0395 - Val F1: 0.8014
Epoch 106: LR reduced from 0.000003 to 0.000002
Epoch 110/150 - Train Loss: 0.0229 - Val Loss: 0.0394 - Val F1: 0.7968
Epoch 113: LR reduced from 0.000002 to 0.000001
Epoch 115/150 - Train Loss: 0.0225 - Val Loss: 0.0391 - Val F1: 0.7923
Early stopping at epoch 115 (best val loss 0.037264)
Completed. Training time: 00:08:09 (489.64s) - Final val loss: 0.0391

--- Training LSTM with lr=8e-05 ---

Epoch 1/150 - Train Loss: 0.1101 - Val Loss: 0.0862 - Val F1: 0.4905
Epoch 5/150 - Train Loss: 0.0795 - Val Loss: 0.0747 - Val F1: 0.5848
Epoch 10/150 - Train Loss: 0.0692 - Val Loss: 0.0725 - Val F1: 0.6218
Epoch 15/150 - Train Loss: 0.0634 - Val Loss: 0.0657 - Val F1: 0.6641
Epoch 20/150 - Train Loss: 0.0604 - Val Loss: 0.0617 - Val F1: 0.6630
Epoch 25/150 - Train Loss: 0.0553 - Val Loss: 0.0575 - Val F1: 0.6811
Epoch 30/150 - Train Loss: 0.0513 - Val Loss: 0.0557 - Val F1: 0.7141
Epoch 35/150 - Train Loss: 0.0481 - Val Loss: 0.0523 - Val F1: 0.6793
Epoch 40/150 - Train Loss: 0.0475 - Val Loss: 0.0527 - Val F1: 0.6900
Epoch 43: LR reduced from 0.000080 to 0.000040
Epoch 45/150 - Train Loss: 0.0414 - Val Loss: 0.0517 - Val F1: 0.7199

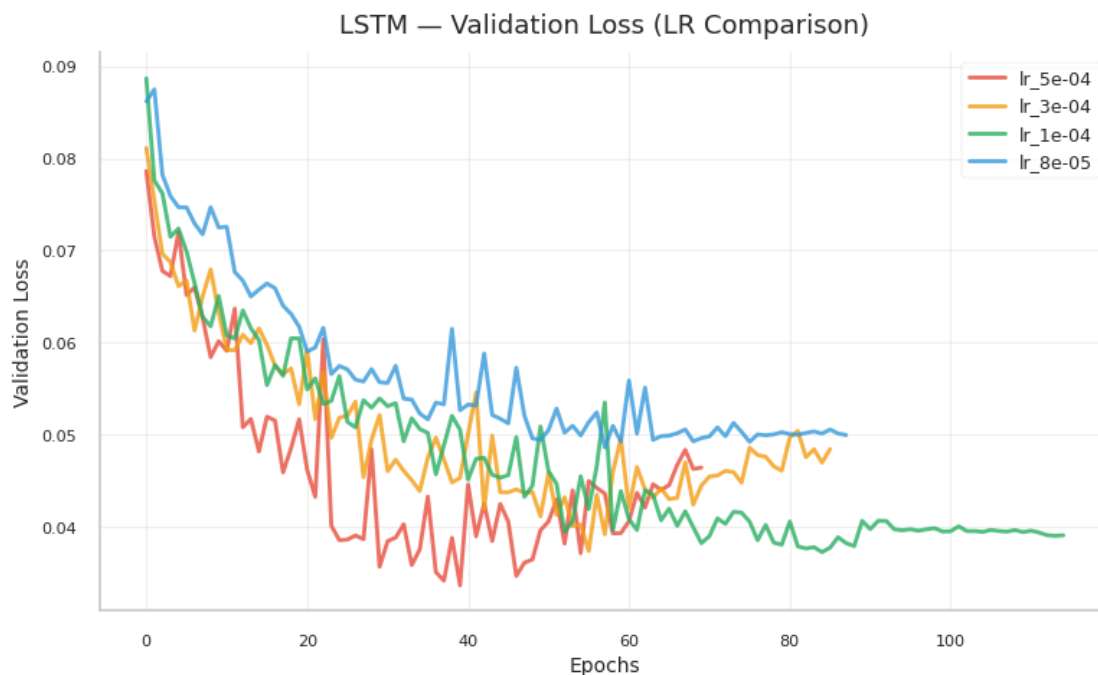
Epoch 50/150 - Train Loss: 0.0413 - Val Loss: 0.0494 - Val F1: 0.7215
 Epoch 55/150 - Train Loss: 0.0394 - Val Loss: 0.0499 - Val F1: 0.7035
 Epoch 57: LR reduced from 0.000040 to 0.000020
 Epoch 60/150 - Train Loss: 0.0373 - Val Loss: 0.0492 - Val F1: 0.7272
 Epoch 65: LR reduced from 0.000020 to 0.000010
 Epoch 65/150 - Train Loss: 0.0366 - Val Loss: 0.0498 - Val F1: 0.7312
 Epoch 70/150 - Train Loss: 0.0353 - Val Loss: 0.0496 - Val F1: 0.7348
 Epoch 72: LR reduced from 0.000010 to 0.000005
 Epoch 75/150 - Train Loss: 0.0335 - Val Loss: 0.0503 - Val F1: 0.7309
 Epoch 79: LR reduced from 0.000005 to 0.000003
 Epoch 80/150 - Train Loss: 0.0337 - Val Loss: 0.0503 - Val F1: 0.7292
 Epoch 85/150 - Train Loss: 0.0335 - Val Loss: 0.0501 - Val F1: 0.7269
 Epoch 86: LR reduced from 0.000003 to 0.000001
 Early stopping at epoch 88 (best val loss 0.048619)
 Completed. Training time: 00:06:14 (374.96s) - Final val loss: 0.0500

[106]: # --- Plot loss curves ---

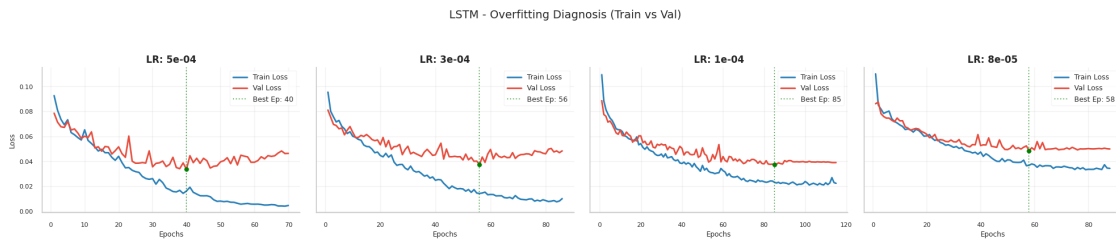
```
# Plot comparison
plot_lr_comparison(histories_lstm, "LSTM", save_dir)

#Plot Train vs Validation
plot_train_val_grid(histories_lstm, "LSTM", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/lstm_lr_comparison.png

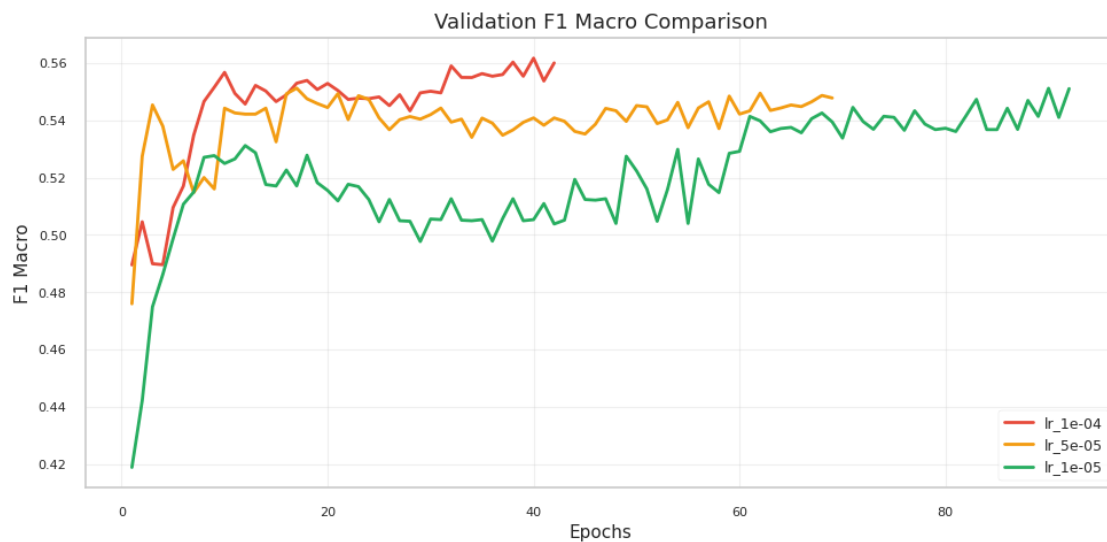


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/lstm_diagnosis.png



```
[107]: # --- Plot Validation F1 Macro for LSTM ---
plot_training_metrics(histories_seq, save_dir=save_dir,
    plot_name="lstm_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/lstm_f1_comparison.png



```
[108]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL LSTM RNN learning rates")
print("=" * 80)

for lr in learning_rates_lstm: # Changed 'learning_rates' to
    'learning_rates_lstm'
    model = models_lstm[lr]
```

```

print(f"\n{'='*60}")
print(f"LSTM RNN with lr={lr:.0e}")
print(f"{'='*60}")

# Validation evaluation
y_val_true, y_val_pred = predict_loader(model, val_loader)
print(f'\nValidation report (LSTM RNN - lr={lr:.0e}):')
print(classification_report(y_val_true, y_val_pred, digits=4,
↪zero_division=0))

# Test evaluation
y_test_true, y_test_pred = predict_loader(model, test_loader)
print(f'Test report (LSTM RNN - lr={lr:.0e}):')
print(classification_report(y_test_true, y_test_pred, digits=4,
↪zero_division=0))

# Find and highlight best model
best_lr_lstm = min(histories_lstm.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f" Best LSTM RNN - LR: {best_lr_lstm[0]:.0e} (Min Val Loss:
↪{min(best_lr_lstm[1]['val_loss']):.4f})")
print(f"{'='*60}")

```

```

=====
Evaluation for ALL LSTM RNN learning rates
=====

```

```

=====
LSTM RNN with lr=5e-04
=====

```

Validation report (LSTM RNN - lr=5e-04):

	precision	recall	f1-score	support
0	0.5486	0.7869	0.6465	122
1	0.9916	0.9749	0.9832	3143
accuracy			0.9678	3265
macro avg	0.7701	0.8809	0.8148	3265
weighted avg	0.9750	0.9678	0.9706	3265

Test report (LSTM RNN - lr=5e-04):

	precision	recall	f1-score	support
0	0.5392	0.7078	0.6121	243

1	0.9885	0.9765	0.9825	6262
accuracy			0.9665	6505
macro avg	0.7639	0.8422	0.7973	6505
weighted avg	0.9717	0.9665	0.9687	6505

```
=====
LSTM RNN with lr=3e-04
=====
```

Validation report (LSTM RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.5424	0.7869	0.6421	122
1	0.9916	0.9742	0.9828	3143
accuracy			0.9672	3265
macro avg	0.7670	0.8806	0.8125	3265
weighted avg	0.9748	0.9672	0.9701	3265

Test report (LSTM RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.5100	0.7366	0.6027	243
1	0.9896	0.9725	0.9810	6262
accuracy			0.9637	6505
macro avg	0.7498	0.8546	0.7918	6505
weighted avg	0.9717	0.9637	0.9669	6505

```
=====
LSTM RNN with lr=1e-04
=====
```

Validation report (LSTM RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.4945	0.7377	0.5921	122
1	0.9896	0.9707	0.9801	3143
accuracy			0.9620	3265
macro avg	0.7421	0.8542	0.7861	3265
weighted avg	0.9711	0.9620	0.9656	3265

Test report (LSTM RNN - lr=1e-04):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.4751	0.7078	0.5686	243
1	0.9884	0.9697	0.9790	6262
accuracy			0.9599	6505
macro avg	0.7318	0.8387	0.7738	6505
weighted avg	0.9693	0.9599	0.9636	6505

```
=====
LSTM RNN with lr=8e-05
=====
```

Validation report (LSTM RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.3897	0.6803	0.4955	122
1	0.9872	0.9586	0.9727	3143
accuracy			0.9482	3265
macro avg	0.6884	0.8195	0.7341	3265
weighted avg	0.9649	0.9482	0.9549	3265

Test report (LSTM RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.4075	0.6255	0.4935	243
1	0.9852	0.9647	0.9748	6262
accuracy			0.9520	6505
macro avg	0.6963	0.7951	0.7342	6505
weighted avg	0.9636	0.9520	0.9568	6505

```
=====
Best LSTM RNN - LR: 5e-04 (Min Val Loss: 0.0336)
=====
```

```
[109]: # --- Plot confusion matrices for ALL learning rates - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL LSTM RNN learning rates - VALIDATION")
print("=" * 80)

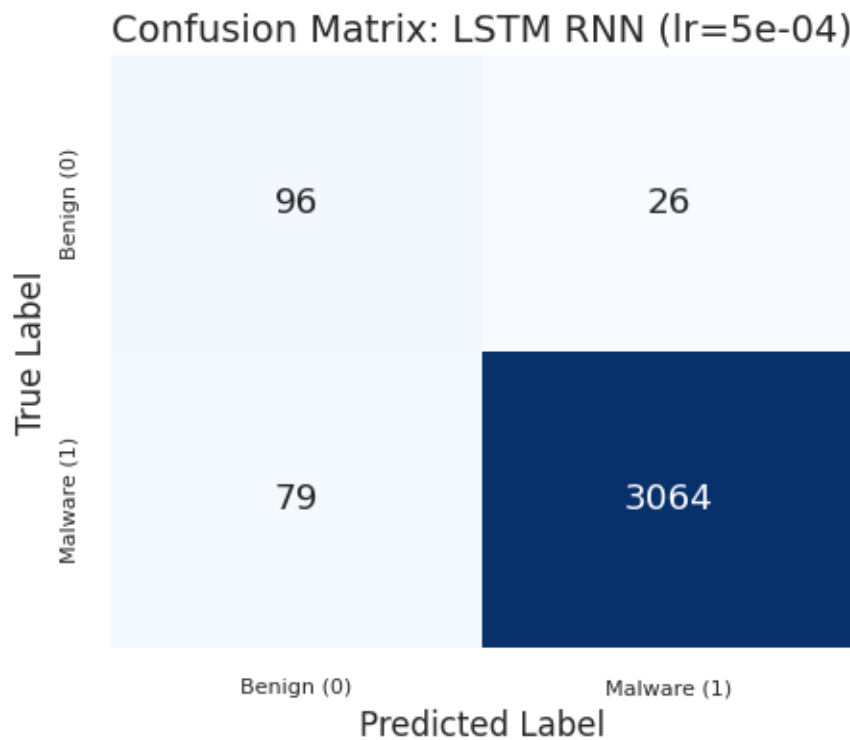
for lr in learning_rates_lstm:
    model = models_lstm[lr]
    print(f"\nGenerating confusion matrix for LSTM RNN with lr={lr:.0e}...")
```

```
plot_cm_task3(model=model, loader=val_loader, model_name=f"LSTM RNN (lr={lr:  
↪.0e})", save_dir=save_dir)
```

=====
Confusion matrices for ALL LSTM RNN learning rates - VALIDATION
=====

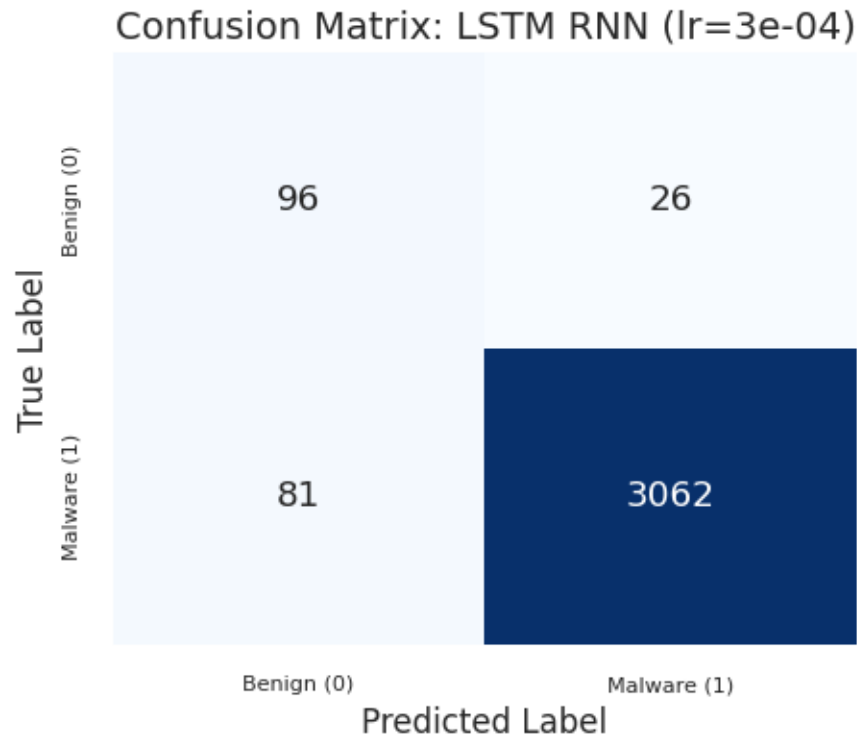
Generating confusion matrix for LSTM RNN with lr=5e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=5e_04).png



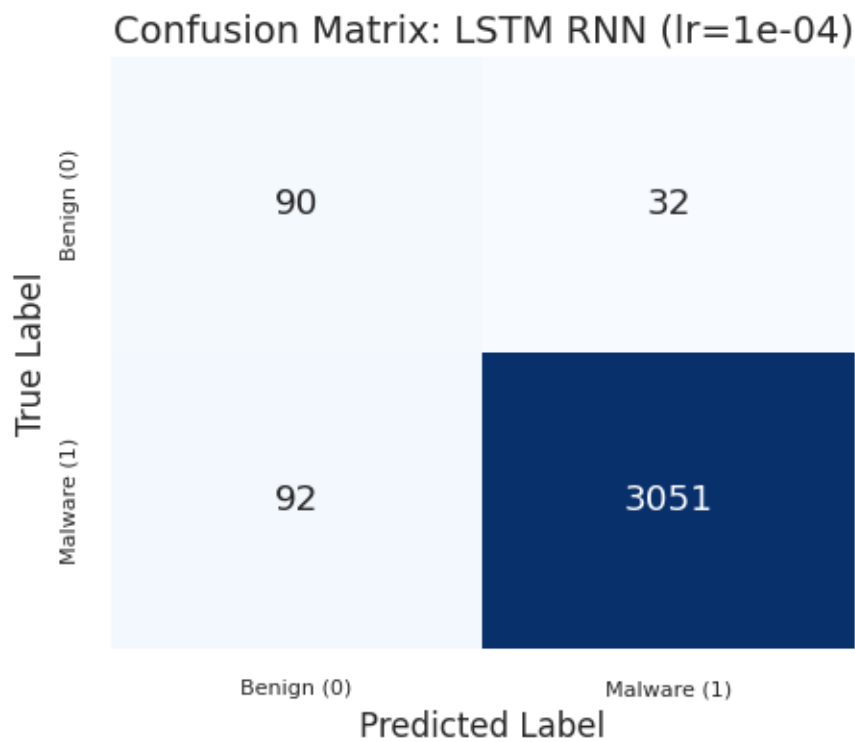
Generating confusion matrix for LSTM RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=3e_04).png



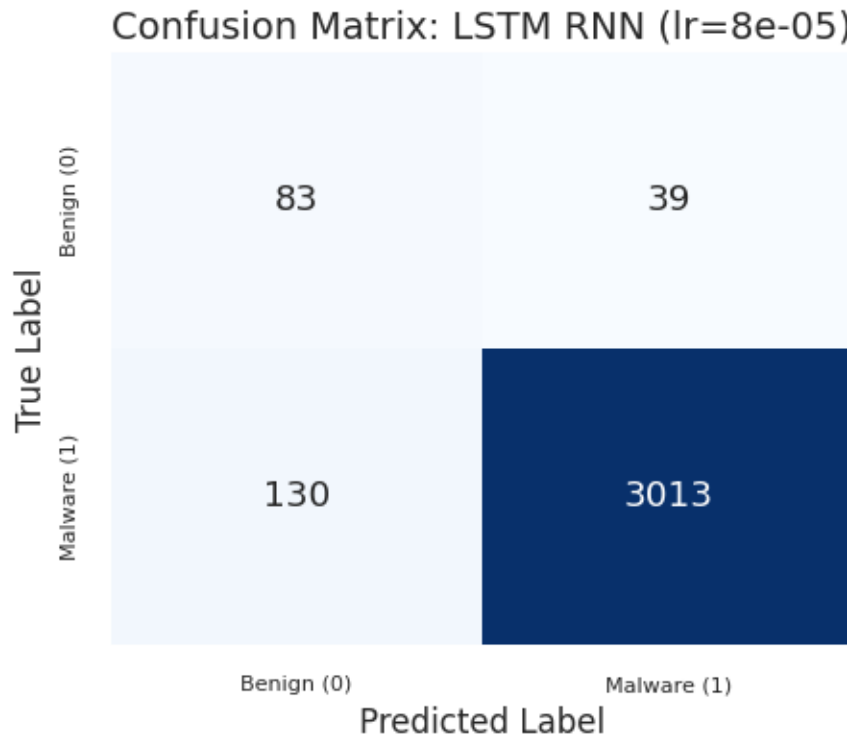
Generating confusion matrix for LSTM RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=1e_04).png



Generating confusion matrix for LSTM RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=8e_05).png



```
[110]: # --- Plot confusion matrices for ALL learning rates - TEST ---

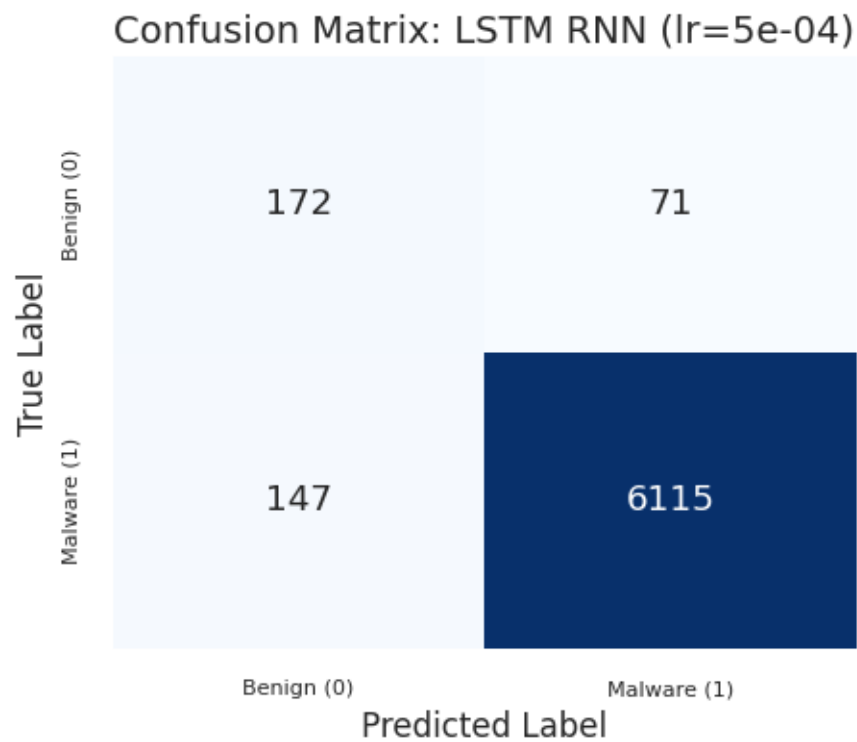
print("\n" + "=" * 80)
print("Confusion matrices for ALL LSTM RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates_lstm:
    model = models_lstm[lr]
    print(f"\nGenerating confusion matrix for LSTM RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"LSTM RNN_
↳(lr={lr:.0e})", save_dir=save_dir)
```

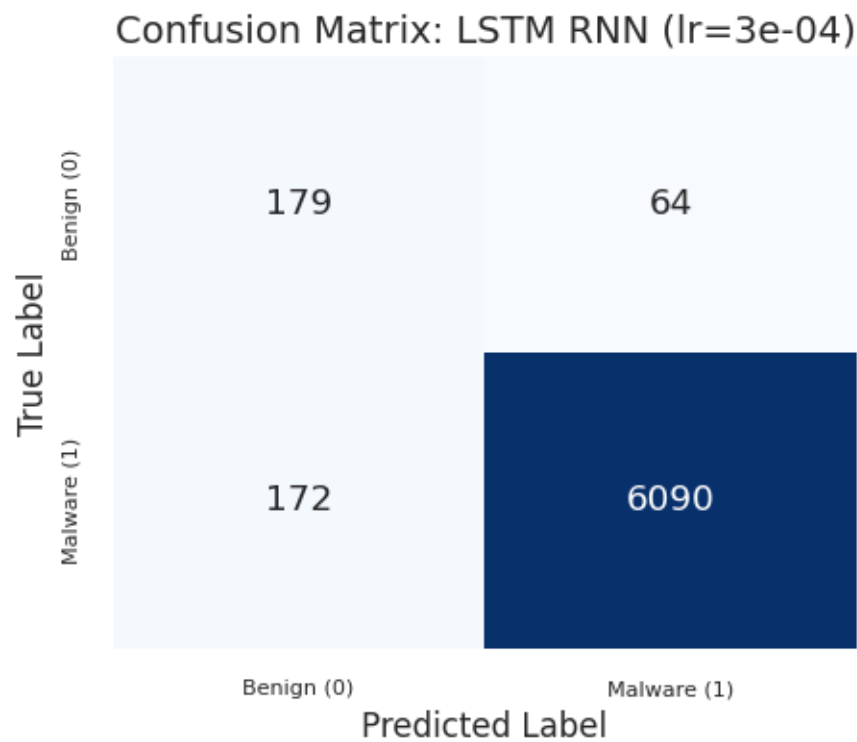
```
=====
Confusion matrices for ALL LSTM RNN learning rates - TEST
=====

Generating confusion matrix for LSTM RNN with lr=5e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=5e_04).png
```



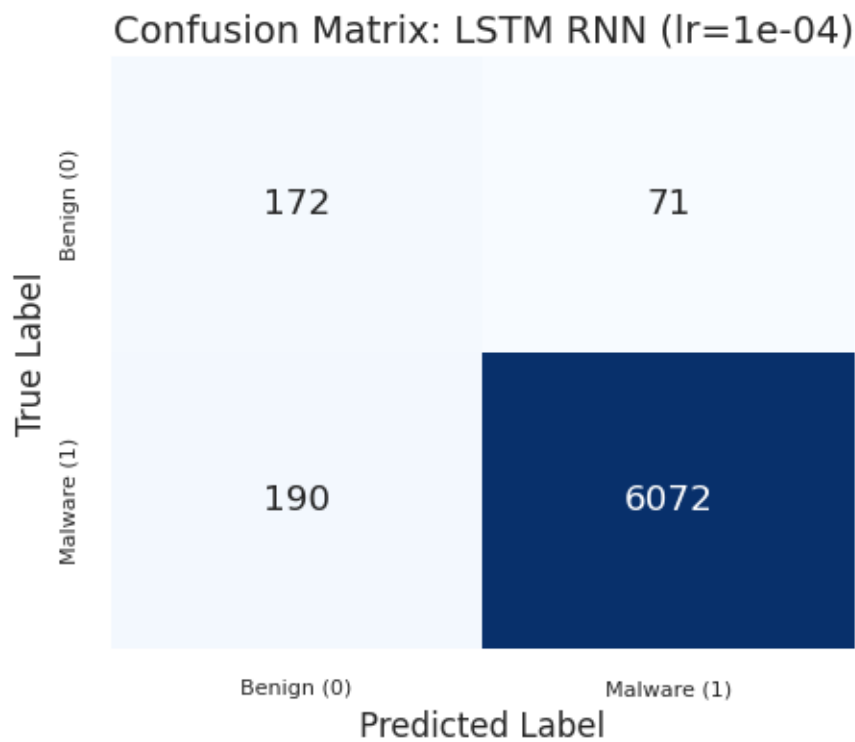
Generating confusion matrix for LSTM RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=3e_04).png



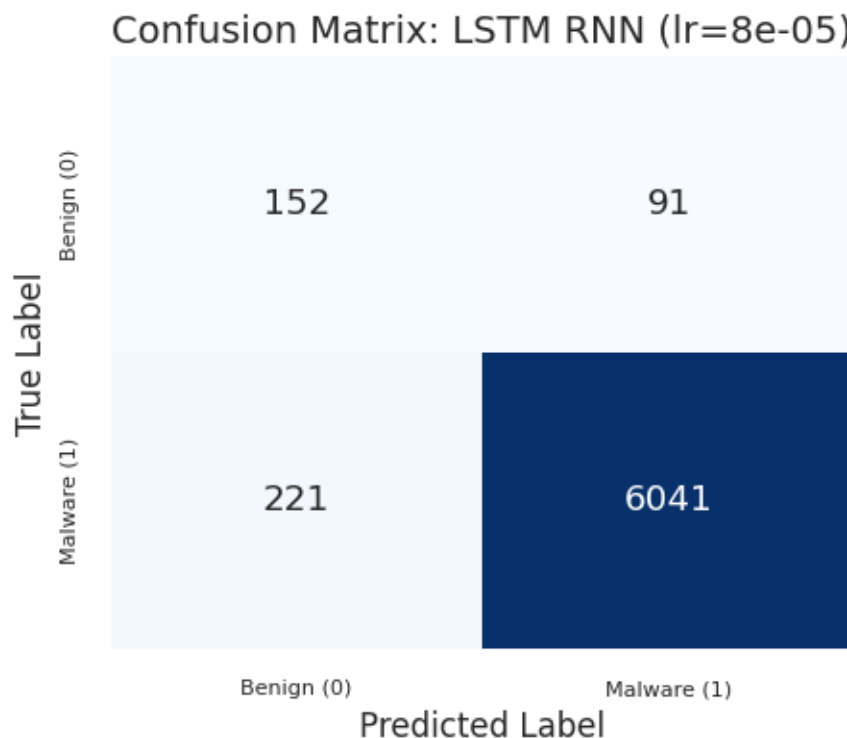
Generating confusion matrix for LSTM RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=1e_04).png



Generating confusion matrix for LSTM RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=8e_05).png



Q: Is the RNN training as stable as the FFNN's one? No, the RNN training is noticeably **less stable** than the FFNN's. While our FFNN loss curves (Task 2) showed a smooth, monotonic decrease, the RNN variations exhibited more fluctuations and sensitivity to hyperparameters.

1. **Gradient Path Complexity:** Unlike FFNNs, where gradients flow through a fixed number of layers, RNNs must backpropagate through time (BPTT). For our sequences of up to 100 API calls, this creates a much deeper computational graph, increasing the risk of **vanishing and exploding gradients**.
2. **Stability of LSTM:** We observed that the LSTM was the most stable among the recurrent architectures. Thanks to its gating mechanisms (input, forget, and output gates), it effectively mitigated the vanishing gradient problem, allowing it to reach a lower **validation loss** compared to the **Simple RNN**.
3. **Oscillations in Validation Loss:** As seen in the Overfitting Diagnosis plots, the validation curves for RNNs are “spikier.” This suggests that the hidden state dynamics are highly sensitive to small weight updates, making the optimization landscape more complex than the relatively straightforward convex-like surface of a standard FFNN.

Q: How does your model's performance compare to the simple frequency baseline? While the simple frequency **baseline (Logistic Regression)** performed surprisingly well in terms of **total accuracy**, the RNN models—particularly the Bidirectional LSTM—provide a much more robust and “security-aware” classification.

Key Observations:

1. **The Accuracy Trap:** The baseline has a very high accuracy, but it is heavily biased toward the majority class. It fails to identify nearly of benign samples.
2. **Superior Minority Class Detection:** The LSTM dramatically improves the detection of the minority class, achieving a better Benign Recall (more than double the baseline). In a real-world cybersecurity pipeline, this is crucial as it significantly reduces the number of false positives (benign files flagged as malware).
3. **Beyond Frequency:** The baseline only knows how many times an API was called. The fact that the LSTM achieves a higher Macro F1 proves that the order and temporal context of API calls (e.g., opening a file before writing to it) contain discriminative patterns that frequency counts alone cannot capture.
4. **Trade-off:** The baseline remains the best at catching every single malware (Recall 0.99), but at the cost of being almost useless for benign file identification. The LSTM offers a much better balance for a production environment.

```
[111]: # =====  
# FINAL SUMMARY  
# =====  
  
# Helper function to extract macro avg from the best model  
def get_macro_avg_stats(model, loader):  
    # Predict using your existing helper function  
    y_true, y_pred = predict_loader(model, loader)  
    # Get report as a DICTIONARY, not a string  
    report_dict = classification_report(y_true, y_pred, output_dict=True,   
    ↪ zero_division=0)  
    return report_dict['macro avg']  
  
# --- 1. Prepare Stats for Simple RNN ---  
# Retrieve the actual model object using the best LR key  
best_model_simple_obj = models_simple[best_lr_simple[0]]  
macro_simple = get_macro_avg_stats(best_model_simple_obj, val_loader)  
  
# --- 2. Prepare Stats for Bi-RNN ---  
best_model_bi_obj = models_bi[best_lr_bi[0]]  
macro_bi = get_macro_avg_stats(best_model_bi_obj, val_loader)  
  
# --- 3. Prepare Stats for LSTM ---  
best_model_lstm_obj = models_lstm[best_lr_lstm[0]]  
macro_lstm = get_macro_avg_stats(best_model_lstm_obj, val_loader)  
  
# --- PRINTING THE REPORT ---  
  
print("\n" + "=" * 90)  
print("TASK 3 SUMMARY: Best Learning Rates & Macro Avg Performance")
```

```

print("=" * 90)
# Header for the manual table we are creating
print(f"{'Model':<20} | {'Best LR':<10} | {'Val Loss':<10} | {'Macro P':<8} |  

      ↳{'Macro R':<8} {'Macro F1':<8} | {'Time':<10}")
print("-" * 90)

# 1. Simple RNN
t_simple = time.strftime('%H:%M:%S', time.  

    ↳gmtime(elapsed_simple[best_lr_simple[0]]))
print(f"{'Simple RNN':<20} | {best_lr_simple[0]:.0e} |  

      ↳{min(best_lr_simple[1]['val_loss']):.4f} | "  

      f"{'macro_simple['precision']:.4f} {'macro_simple['recall']:.4f} |  

      ↳{'macro_simple['f1-score']:.4f} | {t_simple}")

# 2. Bi-RNN
t_bi = time.strftime('%H:%M:%S', time.gmtime(elapsed_bi[best_lr_bi[0]]))
print(f"{'Bidirectional RNN':<20} | {best_lr_bi[0]:.0e} |  

      ↳{min(best_lr_bi[1]['val_loss']):.4f} | "  

      f"{'macro_bi['precision']:.4f} {'macro_bi['recall']:.4f} |  

      ↳{'macro_bi['f1-score']:.4f} | {t_bi}")

# 3. LSTM
t_lstm = time.strftime('%H:%M:%S', time.gmtime(elapsed_lstm[best_lr_lstm[0]]))
print(f"{'LSTM':<20} | {best_lr_lstm[0]:.0e} |  

      ↳{min(best_lr_lstm[1]['val_loss']):.4f} | "  

      f"{'macro_lstm['precision']:.4f} {'macro_lstm['recall']:.4f} |  

      ↳{'macro_lstm['f1-score']:.4f} | {t_lstm}")

```

TASK 3 SUMMARY: Best Learning Rates & Macro Avg Performance

Model	Best LR	Val Loss	Macro P	Macro R	Macro F1	Time
Simple RNN	8e-05	0.0547	0.6878	0.8308	0.7361	00:03:52
Bidirectional RNN	5e-05	0.0439	0.6777	0.8365	0.7280	00:09:36
LSTM	5e-04	0.0336	0.7701	0.8809	0.8148	00:04:59

1.5 Task 4 - Graph Neural Network (GNN)

In this task we model API-call sequences as graphs and train GNN architectures (GCN, GraphSAGE, GAT) to classify samples. We reuse the lab preprocessing and reporting utilities for consistency.

```
[112]: # Create directory for plots
save_dir = results_path + 'images/' + 'task4_plots/'
os.makedirs(save_dir, exist_ok=True)
```

1.5.1 Modeling API Sequences as Graphs

```
[113]: df_train
```

```
[113]:
```

		api_call_sequence	is_malware	\
0		[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	1	
1		[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1	
2		[FindResourceExW, LoadResource, FindResourceEx...	1	
3		[FindResourceExW, LoadResource, FindResourceEx...	1	
4		[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	1	
...		...	...	
16320		[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	1	
16321		[NtClose, LdrGetProcedureAddress, CryptCreateH...	1	
16322		[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1	
16323		[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1	
16324		[SetFilePointer, NtReadFile, SetFilePointer, N...	1	

	seq_length		seq_fixed	\
0	73	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...		
1	88	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...		
2	79	[FindResourceExW, LoadResource, FindResourceEx...		
3	71	[FindResourceExW, LoadResource, FindResourceEx...		
4	63	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...		
...		...	...	
16320	64	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...		
16321	78	[NtClose, LdrGetProcedureAddress, CryptCreateH...		
16322	84	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...		
16323	62	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...		
16324	77	[SetFilePointer, NtReadFile, SetFilePointer, N...		

	seq_sequential_ids
0	[117, 118, 119, 118, 117, 118, 86, 123, 201, 2...
1	[133, 119, 118, 82, 80, 137, 82, 152, 153, 163...
2	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
3	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
4	[118, 219, 119, 219, 81, 46, 121, 46, 121, 46,...
...	...
16320	[118, 119, 118, 119, 118, 119, 118, 119, 118, ...

```

16321  [134, 118, 24, 118, 30, 118, 135, 66, 221, 166...
16322  [118, 117, 118, 81, 216, 119, 118, 46, 121, 46...
16323  [118, 117, 118, 117, 118, 117, 118, 117, 118, ...
16324  [221, 166, 221, 166, 221, 45, 121, 133, 146, 1...

```

[16325 rows x 5 columns]

```

[114]: df_train_split, df_val = train_test_split(
        df_train,
        test_size=0.2,
        random_state=42,
        stratify=df_train['is_malware'], # Preserve class balance
        shuffle=True
    )

```

```

[115]: # Use original api_call_sequence (not truncated seq_sequential_ids)
# GNNs can handle variable-length graphs, so we use full sequences
X_train_split = df_train_split['api_call_sequence'].values
y_train_split = df_train_split['is_malware'].values

X_val = df_val['api_call_sequence'].values
y_val = df_val['is_malware'].values

X_test = df_test['api_call_sequence'].values
y_test = df_test['is_malware'].values

print(f"Train length: {len(X_train_split)}")
print(f"Val length: {len(X_val)}")
print(f"Test length: {len(X_test)}")

```

Train length: 13060

Val length: 3265

Test length: 6505

```

[116]: # --- Graph Construction ---

def create_graph_from_sequence(sequence, api_to_id, label, window_size=1):
    """
    Create a sparse graph containing ONLY the nodes present in the sequence.
    Accepts API name strings and converts them to integer IDs using api_to_id_
    ↪mapping.
    Unknown APIs are mapped to UNK_IDX (index 1).
    """
    UNK_IDX = 1 # Same as defined in Task 3

    # 1. Convert API names to integer IDs (handle unknown APIs with UNK_IDX)
    sequence_ids = [api_to_id.get(api, UNK_IDX) for api in sequence]

```

```

# 2. Identify unique node IDs present in this sequence
# IMPORTANT: Filter out padding (0) before creating node set
unique_api_ids = sorted(list(set([api_id for api_id in sequence_ids if
↪api_id != 0])))
num_nodes = len(unique_api_ids)

# 3. Create a mapping from Global API ID -> Local Graph Node Index
global_to_local = {global_id: local_id for local_id, global_id in
↪enumerate(unique_api_ids)}

# 4. Create Node Features (The Global IDs to be fed into the Embedding
↪Layer)
x = torch.tensor(unique_api_ids, dtype=torch.long).unsqueeze(1)

# 5. Create Edges (Transition Graph)
src_list = []
dst_list = []

# Sliding window to find transitions
for i in range(len(sequence_ids) - window_size):
    global_src = sequence_ids[i]
    global_dst = sequence_ids[i + window_size]

    # Skip padding (0) if present
    if global_src == 0 or global_dst == 0:
        continue

    # Map to local indices
    src_list.append(global_to_local[global_src])
    dst_list.append(global_to_local[global_dst])

edge_index = torch.tensor([src_list, dst_list], dtype=torch.long)

data = Data(
    x=x,
    edge_index=edge_index,
    y=torch.tensor([label], dtype=torch.float32),
    num_nodes=num_nodes
)

return data

def create_graphs_from_sequences(sequences, api_to_id, labels, window_size=1):
    graph_data_list = []
    for sequence, label in zip(sequences, labels):
        # Ensure sequence is a list (handles numpy arrays)
        seq_list = sequence if isinstance(sequence, list) else list(sequence)

```

```

        data = create_graph_from_sequence(seq_list, api_to_id, label,
↪window_size)
        graph_data_list.append(data)

        # Sanity check for each graph
        if data.edge_index.numel() > 0 and data.edge_index.max().item() >= data.
↪num_nodes:
            print(f"Warning: Graph {idx} has invalid edge index!")

        return graph_data_list

# --- Execute Creation ---
print("Creating sparse graphs...")
train_graphs = create_graphs_from_sequences(X_train_split, api_to_id,
↪y_train_split)
val_graphs = create_graphs_from_sequences(X_val, api_to_id, y_val)
test_graphs = create_graphs_from_sequences(X_test, api_to_id, y_test)

print(f"Training graphs: {len(train_graphs)}")
print(f"First graph nodes: {train_graphs[0].num_nodes} (Subset of total vocab)")
print(f"First graph edges: {train_graphs[0].num_edges}")

```

Creating sparse graphs...

Training graphs: 13060

First graph nodes: 21 (Subset of total vocab)

First graph edges: 80

```

[117]: import networkx as nx
import matplotlib.pyplot as plt
from torch_geometric.utils import to_networkx

def visualize_graph(data, id_to_api_mapping=None):
    # Convert PyG data to NetworkX
    G = to_networkx(data, to_undirected=False) # Keep it directed to see flow

    plt.figure(figsize=(12, 8))

    # Layout (Spring layout tends to show clusters and loops well)
    pos = nx.spring_layout(G, seed=42)

    # Draw nodes
    nx.draw_networkx_nodes(G, pos, node_size=800, node_color='lightblue',
↪alpha=0.8)

    # Draw edges (with arrows)
    nx.draw_networkx_edges(G, pos, width=1.0, alpha=0.5, arrowstyle='->',
↪arrowsize=20)

```

```

# Labels: If you have the mapping, show API names. Otherwise show IDs.
if id_to_api_mapping:
    # Get the Global API ID from data.x for each local node index
    labels = {i: id_to_api_mapping.get(data.x[i].item(), str(i)) for i in
↪range(data.num_nodes)}
    nx.draw_networkx_labels(G, pos, labels, font_size=10,
↪font_weight="bold")
else:
    nx.draw_networkx_labels(G, pos)

plt.title(f"Malware Execution Graph\nLabel: {data.y.item()} (1=Malware,
↪0=Benign)")
plt.axis('off')
plt.show()

# --- RUN THE CHECK ---
# Plot example graphs from each dataset
print("Plotting example graphs...\n")
print("Training set example:")
visualize_graph(train_graphs[0], id_to_api)

print("\nValidation set example:")
visualize_graph(val_graphs[0], id_to_api)

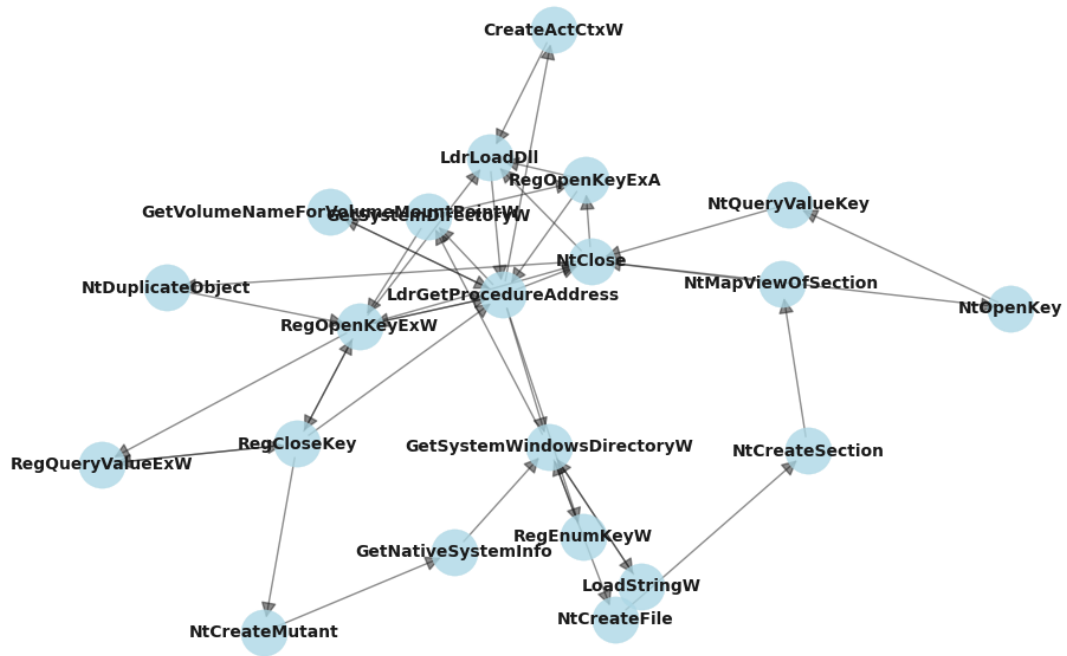
print("\nTest set example:")
visualize_graph(test_graphs[0], id_to_api)

```

Plotting example graphs...

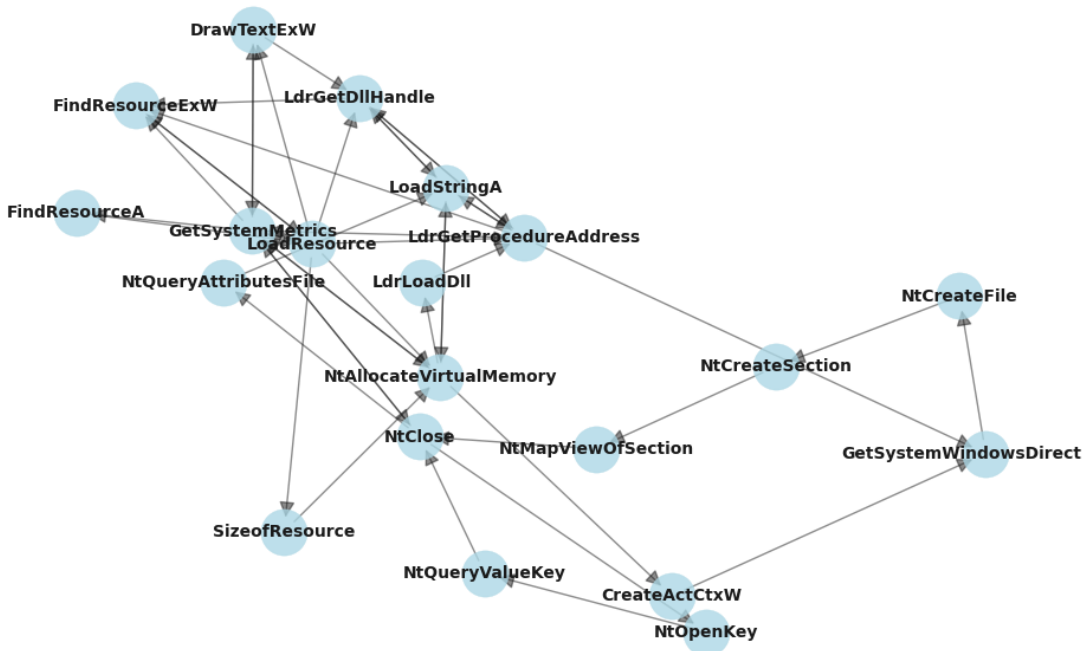
Training set example:

Malware Execution Graph
Label: 1.0 (1=Malware, 0=Benign)



Validation set example:

Malware Execution Graph
Label: 1.0 (1=Malware, 0=Benign)



```

[118]: data = train_graphs[0]

print(f"1. Node Features Shape: {data.x.shape}")
print(f"    → {data.num_nodes} unique APIs in this sequence")

print(f"\n2. Edge Index Shape: {data.edge_index.shape}")
print(f"    → {data.num_edges} transitions (edges) in the sequence")

print(f"\n3. Max Node Index: {data.edge_index.max().item()}")
print(f"    → Valid range: [0, {data.num_nodes - 1}]")
if data.edge_index.numel() > 0:
    is_valid = data.edge_index.max().item() < data.num_nodes
    print(f"    → {' VALID' if is_valid else ' INVALID - edges point to\nc\n    ↪non-existent nodes'}")
else:
    print(f"    → No edges in this graph")

print(f"\n4. Label Shape: {data.y.shape}")
print(f"    → Label value: {data.y.item()} ({'Malware' if data.y.item() == 1\nc\n    ↪else 'Benign'})")

print(f"\n5. Summary:")

```

```
print(f"    → Graph has {data.num_nodes} nodes and {data.num_edges} edges")
print(f"    → Sparsity: {(data.num_edges / (data.num_nodes * (data.num_nodes - 1))) * 100:.2f}% (if fully connected)")
```

1. Node Features Shape: torch.Size([21, 1])
→ 21 unique APIs in this sequence
2. Edge Index Shape: torch.Size([2, 80])
→ 80 transitions (edges) in the sequence
3. Max Node Index: 20
→ Valid range: [0, 20]
→ VALID
4. Label Shape: torch.Size([1])
→ Label value: 1.0 (Malware)
5. Summary:
→ Graph has 21 nodes and 80 edges
→ Sparsity: 19.05% (if fully connected)

```
[119]: # Get the raw sequence and the graph
raw_seq_ids = X_train_split[0] # The list of IDs: [117, 118, 119...]
graph = train_graphs[0]

# Check 1: Unique counts
unique_apis_raw = len(set(raw_seq_ids))
# Padding (0) is usually removed in your graph logic, so:
if 0 in raw_seq_ids: unique_apis_raw -= 1

print(f"Unique APIs in raw sequence: {unique_apis_raw}")
print(f"Nodes in Graph: {graph.num_nodes}")
# THESE SHOULD MATCH EXACTLY.

# Check 2: Transitions vs Edges
# Raw edges = length of sequence - 1 (minus padding transitions)
# Note: Your graph combines duplicate edges.
# If raw sequence is A->B->A->B, graph has edges (A,B) and (B,A).
# This check is qualitative: Does graph.num_edges roughly track with sequence
# complexity?
print(f"Raw sequence length: {len([x for x in raw_seq_ids if x != 0])}")
print(f"Graph edges: {graph.num_edges}")
```

```
Unique APIs in raw sequence: 21
Nodes in Graph: 21
Raw sequence length: 81
Graph edges: 80
```

Q: Do you still have to pad your data? If yes, how? No, the graph-based representation eliminates the need for padding entirely.

Unlike FFNNs or RNNs, which require fixed-size dense tensors, Graph Neural Networks (GNNs) operate on sparse structures. In our implementation:

1. **Topological Flexibility:** Each API sequence is transformed into a directed graph where nodes represent unique API calls and edges represent transitions. Longer or more complex sequences simply result in a higher number of edges or a larger subset of nodes, naturally accommodating variable lengths without “dummy” data.
2. **PyG Batching Strategy:** We leverage PyTorch Geometric’s specialized batching, which merges multiple individual graphs into a single large, disjoint graph.

This approach is significantly more memory-efficient than Task 2 and Task 3, as it avoids any redundant computation on padding tokens and preserves the exact structural relationships of the API execution flow.

Q: Do you have to truncate the testing sequences? Justify your answer. TODO: check

No, you absolutely do not truncate the testing sequences.

- **Justification:** The GNN approach, like the RNN, is designed to work with variable-sized inputs. Each sequence is converted into its own graph structure. The size of this graph (number of nodes and edges) is unique to that sequence.
- The model (e.g., GCNConv) and the pooling layer (`global_mean_pool`) are designed to operate on these variable-sized graphs and produce a single, fixed-size vector representation for the *entire graph* regardless of its size.
- Truncating the sequences would mean removing nodes and edges from our graphs, which would be **throwing away crucial structural information**. This would defeat the entire purpose of using a GNN, which is to learn from the complete graph structure of the API calls.

Q: What is the advantage of modeling your problem with a GNN compared to an RNN? What do you lose?

- **Advantage (What you gain):**
 - **Richer Structure:** An RNN only sees the sequence as a linear chain (A -> B -> C). A GNN, as we’ve defined it, captures more complex relationships. For example, in the sequence A -> B -> A -> C, the GNN sees that node A has connections to both B and C, and that B also connects back to A. This creates a small “graphlet” or structural pattern. The GNN can learn that this *pattern* is malicious, whereas the RNN just processes the linear flow.
 - **Order Insensitivity (in this model):** By using `global_mean_pool`, the model becomes permutation-invariant. It cares about the types of connections and patterns that exist (e.g., “this graph contains a lot of A->B edges”) rather than *when* they happened, which is similar to the baseline but on a structural level.
- **Disadvantage (What you lose):**

- **Strict Sequential Order:** The primary thing you lose is the *strict, temporal, step-by-step order* that the RNN explicitly models. The GNN model (as built here with `global_mean_pool`) treats the graph as a single structure. It knows A is connected to B, but it doesn't explicitly know if that connection happened at step 1 or step 50. The RNN's hidden state, by definition, is different at step 50 than at step 1 because it has processed all prior steps.

1.5.2 GNN Training and Helper Functions

We need new training and prediction functions, as the `PyGDataLoader` returns batches differently from the standard `DataLoader`.

```
[120]: # Create DataLoader for batching
train_loader = PyGDataLoader(train_graphs, batch_size=32, shuffle=True)
val_loader = PyGDataLoader(val_graphs, batch_size=32, shuffle=False)
test_loader = PyGDataLoader(test_graphs, batch_size=32, shuffle=False)

print(f"Train loader: {len(train_loader)}, Validation loader: {len(val_loader)}, Test loader: {len(test_loader)}")
```

Train loader: 409, Validation loader: 103, Test loader: 204

```
[121]: # =====
# GNN TRAINING FUNCTION (with Macro F1 tracking)
# =====

def train_gnn_common(model, train_loader, val_loader, optimizer, criterion,
    epochs=50, min_delta=0.00001, patience=20):
    """
    Train a GNN model with early stopping and track macro F1 score.
    """
    device = next(model.parameters()).device

    best_val = math.inf
    best_state = None
    trigger = 0
    history = {'train_loss': [], 'val_loss': [], 'val_f1_macro': []}
    start_time = time.time()

    for epoch in range(1, epochs + 1):
        model.train()
        train_losses = []
        for data in train_loader:
            data = data.to(device)
            optimizer.zero_grad()
            outputs = model(data)
            loss = criterion(outputs, data.y.unsqueeze(1).float())
            loss.backward()
```

```

        optimizer.step()
        train_losses.append(loss.item())
    train_loss = float(np.mean(train_losses)) if train_losses else 0.0

    # Validation
    model.eval()
    val_losses = []
    all_preds = []
    all_targets = []
    with torch.no_grad():
        for data in val_loader:
            data = data.to(device)
            outputs = model(data)
            loss = criterion(outputs, data.y.unsqueeze(1).float())
            val_losses.append(loss.item())

            # Collect predictions for F1 score
            # FIX: use view(-1) instead of squeeze() to handle
↪single-element batches
            probs = torch.sigmoid(outputs).view(-1)
            preds = (probs > 0.5).long()

            # FIX: convert to list properly to handle both single and
↪multi-element tensors
            all_preds.extend(preds.cpu().tolist())
            all_targets.extend(data.y.cpu().tolist())

    val_loss = float(np.mean(val_losses)) if val_losses else 0.0
    val_f1_macro = f1_score(all_targets, all_preds, average='macro',
↪zero_division=0)

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_f1_macro'].append(val_f1_macro)

    if (epoch % 10 == 0) or (epoch == 1) or (epoch == epochs):
        print(f'Epoch {epoch}/{epochs} - Train Loss: {train_loss:.4f} - Val
↪Loss: {val_loss:.4f} - Val F1: {val_f1_macro:.4f}')

    # Early stopping logic
    if val_loss < best_val - min_delta:
        best_val = val_loss
        best_state = {k: v.cpu().clone() for k, v in model.state_dict().
↪items()}
        trigger = 0
    else:
        trigger += 1

```

```

        if trigger >= patience:
            print(f'Early stopping at epoch {epoch} (best val loss_
↪{best_val:.6f})')
            break

    elapsed = time.time() - start_time

    if best_state is not None:
        model.load_state_dict(best_state)
        model.to(device)

    return model, history, elapsed

```

[122]: *# --- GNN Prediction Helper ---*

```

def predict_loader_gnn(model, loader, threshold=0.5):
    model.eval()

    # Detect device from model parameters
    device = next(model.parameters()).device

    ys_true = []
    ys_pred = []
    with torch.no_grad():
        for data in loader:
            # Move batch to device
            data = data.to(device)

            out = model(data)
            probs = torch.sigmoid(out).cpu().numpy().ravel()
            preds = (probs >= threshold).astype(int)

            # Get true labels from the DataBatch object
            ys_true.extend(data.y.cpu().numpy().ravel().astype(int).tolist())
            ys_pred.extend(preds.tolist())

    return np.array(ys_true, dtype=int), np.array(ys_pred, dtype=int)

```

[123]:

```

def plot_cm_task4(model, loader, model_name, save_dir=None):
    """
    Specific for GNNs. Uses your existing 'predict_loader_gnn' logic.
    """
    # Use the GNN specific helper
    y_true, y_pred = predict_loader_gnn(model, loader)

    # Plot
    _internal_plot_cm(y_true, y_pred, model_name, save_dir)

```

1.5.3 GNN Variations

Now we define and train the three GNN architectures as requested by the lab PDF. We will use the same hyperparameters (`vocab_size`, `embed_dim`, `hidden_dim`) for a fair comparison. We also re-use the `pos_weight` from Task 3 to handle the imbalanced dataset.

```
[124]: # Hyperparameters for all GNNs
VOCAB_SIZE_GNN = len(api_to_id)
EMBED_DIM_GNN = 32
HIDDEN_DIM_GNN = 64
OUTPUT_DIM_GNN = 1
EPOCHS_GNN = 100
PATIENCE_GNN = 15
MIN_DELTA_GNN = 0.0001
LR_GNN = 5e-4 # Learning rate

# Re-compute pos_weight for the GNN training subset
pos_weight_gnn = compute_pos_weight(y_train_split).to(device)

criterion_gnn = nn.BCEWithLogitsLoss(pos_weight=pos_weight_gnn)

print(f"GNN Vocab Size: {VOCAB_SIZE_GNN}")
print(f"GNN Positive Weight: {pos_weight_gnn.item():.4f}") # Use .item() for
    ↪ scalar tensor print
```

```
Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000
GNN Vocab Size: 260
GNN Positive Weight: 0.2000
```

1. Simple GCN This model uses Graph Convolutional Networks (GCN).

```
[125]: # --- 1. GCN Model Definition ---

class GCNmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(GCNmodel, self).__init__()

        # Embedding: Maps Global API IDs -> Dense Vector
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # GCN Layers
        self.conv1 = GCNConv(embed_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, hidden_dim)
        self.conv3 = GCNConv(hidden_dim, hidden_dim)

        # Pooling
        self.pool = global_mean_pool
```

```

    # Classifier
    self.classifier = nn.Sequential(
        nn.Linear(hidden_dim, hidden_dim),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(hidden_dim, output_dim)
    )

def forward(self, data):
    x, edge_index, batch = data.x, data.edge_index, data.batch

    # x comes in as [num_nodes_in_batch, 1]
    x = x.squeeze(1) # [num_nodes_in_batch]

    # 1. Embedding
    x = self.embedding(x) # [num_nodes, embed_dim]

    # 2. GCN
    x = self.conv1(x, edge_index)
    x = F.relu(x)
    x = F.dropout(x, p=0.2, training=self.training)

    x = self.conv2(x, edge_index)
    x = F.relu(x)
    x = F.dropout(x, p=0.2, training=self.training)

    x = self.conv3(x, edge_index)
    x = F.relu(x)
    x = F.dropout(x, p=0.2, training=self.training)

    # 3. Global Pooling
    # Averages node vectors belonging to the same graph
    x = self.pool(x, batch)

    # 4. Classification
    out = self.classifier(x)
    return out

```

```

[126]: # =====
# GCN: Training with Multiple Learning Rates
# =====

gcn_learning_rates = [1e-4, 5e-4, 1e-3]
gcn_lr_labels = ['LR=1e-4', 'LR=5e-4', 'LR=1e-3']

# Storage for results

```

```

gcn_histories = {}
gcn_models = {}
gcn_elapsed_times = {}

for lr, lr_label in zip(gcn_learning_rates, gcn_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GCN with {lr_label}")
    print('='*60)

    # Set seed for reproducibility
    torch.manual_seed(42)
    np.random.seed(42)

    # Create model
    model_gcn_temp = GCNmodel(
        vocab_size=VOCAB_SIZE_GNN,
        embed_dim=EMBED_DIM_GNN,
        hidden_dim=HIDDEN_DIM_GNN,
        output_dim=OUTPUT_DIM_GNN
    ).to(device)

    optimizer_temp = optim.AdamW(model_gcn_temp.parameters(), lr=lr)

    model_gcn_temp, history_temp, elapsed_temp = train_gnn_common(
        model_gcn_temp,
        train_loader,
        val_loader,
        optimizer_temp,
        criterion_gnn,
        epochs=EPOCHS_GNN,
        min_delta=MIN_DELTA_GNN,
        patience=PATIENCE_GNN
    )

    gcn_models[lr_label] = model_gcn_temp
    gcn_histories[lr_label] = history_temp
    gcn_elapsed_times[lr_label] = elapsed_temp

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
    print(f'\n{lr_label} - Training time: {elapsed_str}')

```

```

=====
Training GCN with LR=1e-4
=====

```

```

Epoch 1/100 - Train Loss: 0.1204 - Val Loss: 0.1005 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0745 - Val Loss: 0.0731 - Val F1: 0.6072
Epoch 20/100 - Train Loss: 0.0622 - Val Loss: 0.0666 - Val F1: 0.6556

```

Epoch 30/100 - Train Loss: 0.0579 - Val Loss: 0.0632 - Val F1: 0.6878
 Epoch 40/100 - Train Loss: 0.0539 - Val Loss: 0.0607 - Val F1: 0.6803
 Epoch 50/100 - Train Loss: 0.0499 - Val Loss: 0.0581 - Val F1: 0.7130
 Epoch 60/100 - Train Loss: 0.0475 - Val Loss: 0.0566 - Val F1: 0.7130
 Epoch 70/100 - Train Loss: 0.0447 - Val Loss: 0.0552 - Val F1: 0.7372
 Epoch 80/100 - Train Loss: 0.0417 - Val Loss: 0.0529 - Val F1: 0.7310
 Epoch 90/100 - Train Loss: 0.0406 - Val Loss: 0.0514 - Val F1: 0.7369
 Epoch 100/100 - Train Loss: 0.0385 - Val Loss: 0.0518 - Val F1: 0.7461

LR=1e-4 - Training time: 00:05:31

=====
 Training GCN with LR=5e-4
 =====

Epoch 1/100 - Train Loss: 0.1031 - Val Loss: 0.0874 - Val F1: 0.4905
 Epoch 10/100 - Train Loss: 0.0529 - Val Loss: 0.0578 - Val F1: 0.7058
 Epoch 20/100 - Train Loss: 0.0389 - Val Loss: 0.0516 - Val F1: 0.7438
 Epoch 30/100 - Train Loss: 0.0323 - Val Loss: 0.0513 - Val F1: 0.7975
 Epoch 40/100 - Train Loss: 0.0266 - Val Loss: 0.0476 - Val F1: 0.7614
 Early stopping at epoch 49 (best val loss 0.043990)

LR=5e-4 - Training time: 00:02:42

=====
 Training GCN with LR=1e-3
 =====

Epoch 1/100 - Train Loss: 0.0957 - Val Loss: 0.0754 - Val F1: 0.5865
 Epoch 10/100 - Train Loss: 0.0466 - Val Loss: 0.0552 - Val F1: 0.7208
 Epoch 20/100 - Train Loss: 0.0299 - Val Loss: 0.0560 - Val F1: 0.8005
 Epoch 30/100 - Train Loss: 0.0247 - Val Loss: 0.0580 - Val F1: 0.7961
 Epoch 40/100 - Train Loss: 0.0203 - Val Loss: 0.0616 - Val F1: 0.8160
 Early stopping at epoch 43 (best val loss 0.048108)

LR=1e-3 - Training time: 00:02:22

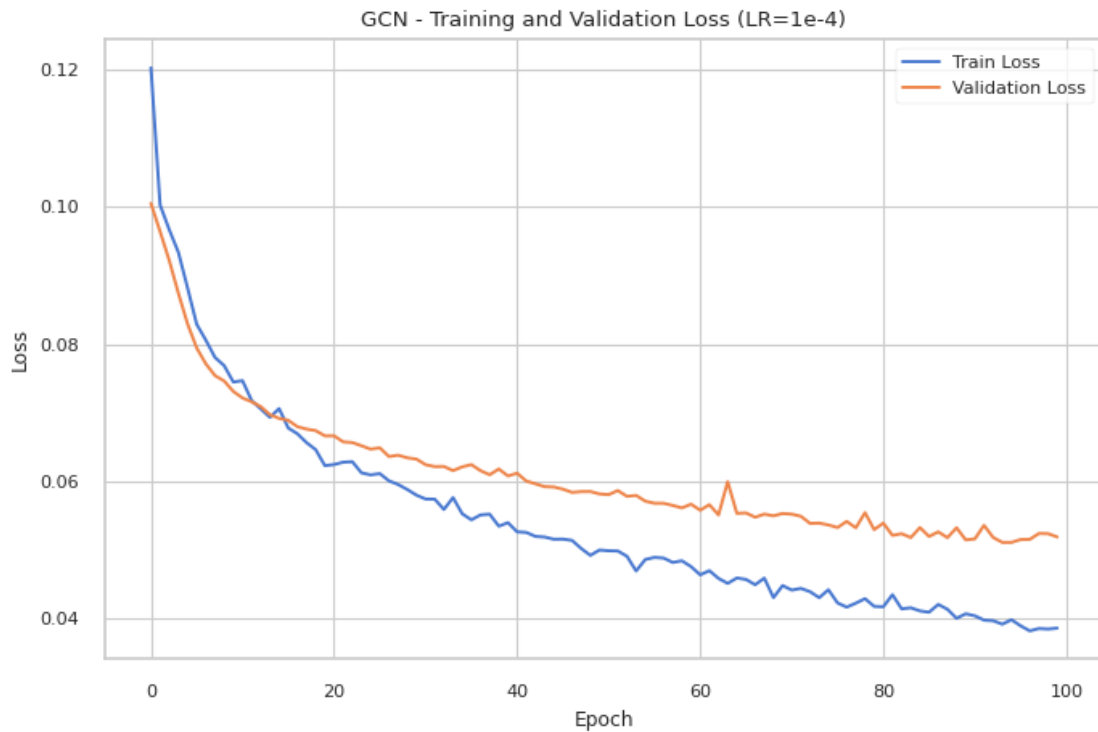
```
[127]: # =====
# Plot Training/Validation Loss per OGNI learning rate (GCN)
# =====

for lr_label, history in gcn_histories.items():
    plt.figure(figsize=(8, 5))
    plt.plot(history['train_loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Validation Loss')
    plt.title(f'GCN - Training and Validation Loss ({lr_label})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
```

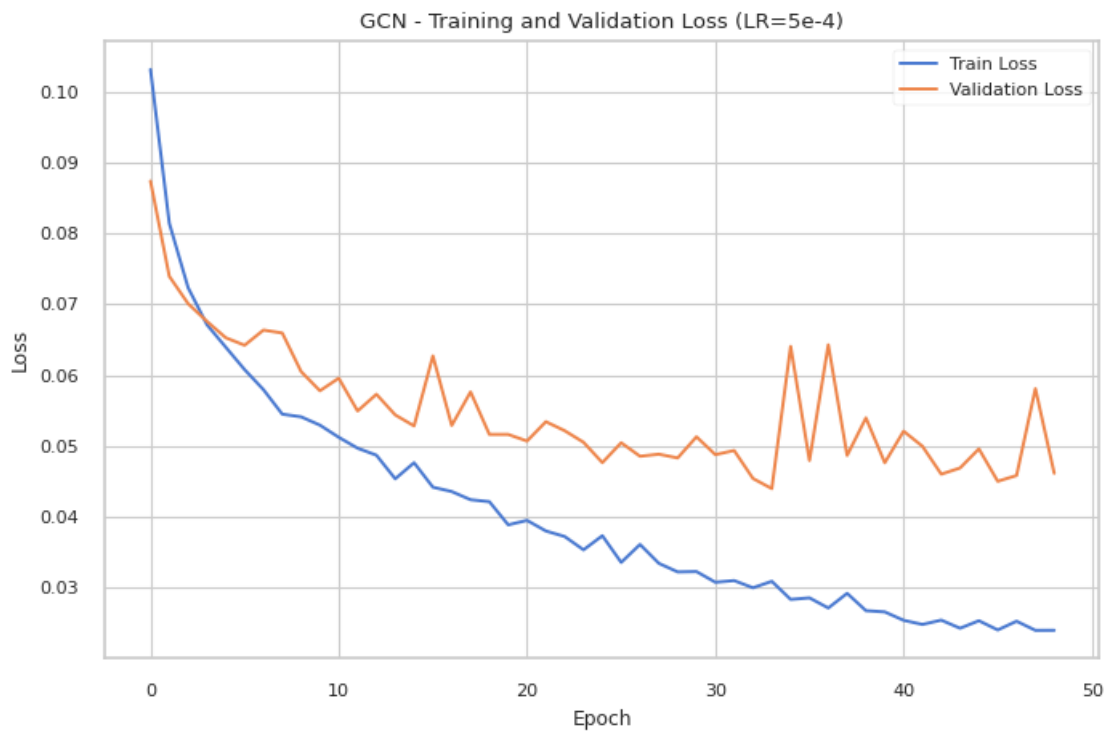
```
plt.grid(True)

# Nome file unico per ogni lr
save_plot(plt.gcf(), f'task4_gcn_loss_curves_{lr_label.replace("=", "")}',
save_dir)
plt.show()
```

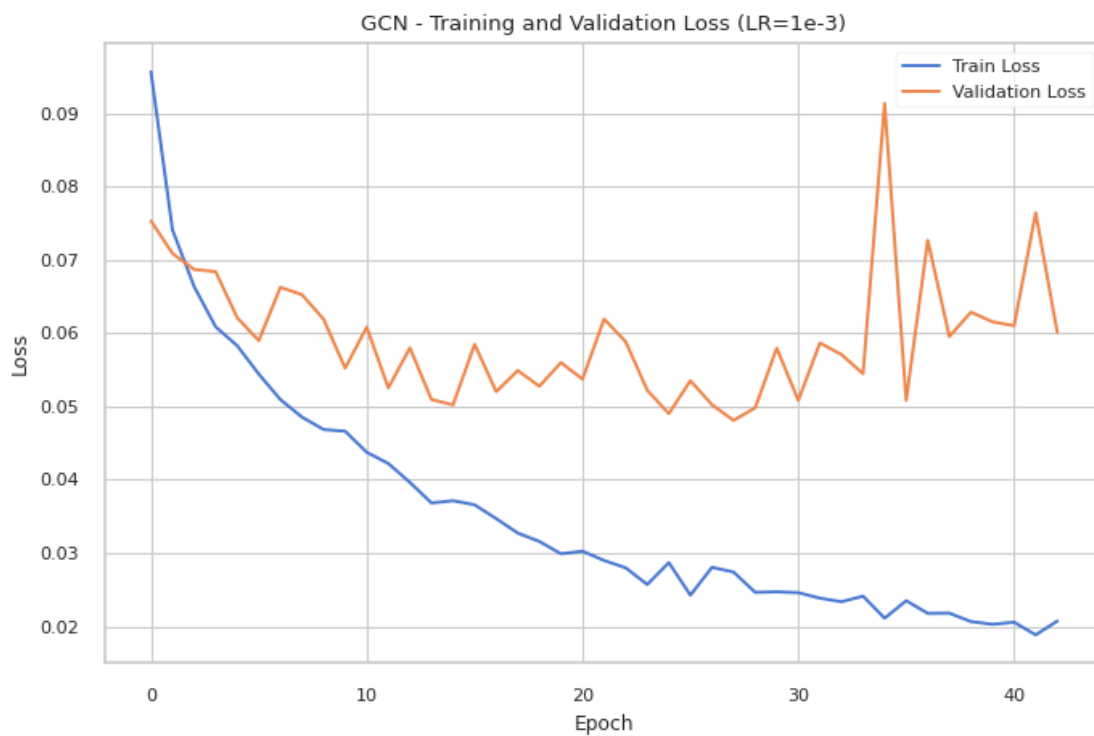
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR1e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR5e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR1e-3.png



```
[128]: # =====
# GCN: Plot Learning Rate Comparison (Validation Loss + Macro F1)
# =====

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#1b9e77', '#d95f02', '#7570b3']

# Plot Validation Loss
for (lr_label, history), color in zip(gcn_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label, linewidth=2,
            color=color)

ax1.set_title('GCN - Validation Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.legend()
ax1.grid(alpha=0.3)

# Plot Validation Macro F1
for (lr_label, history), color in zip(gcn_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,
            linewidth=2, color=color)

ax2.set_title('GCN - Validation Macro F1 Score', fontsize=14, fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Macro F1 Score', fontsize=12)
ax2.legend()
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gcn_lr_comparison", save_dir)
plt.show()
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_lr_comparison.png



```
[129]: # --- Evaluate GCN ---
for lr_label, model_gcn in gcn_models.items():
    print(f"\n{'='*60}")
    print(f"Evaluation for GCN - {lr_label}")
    print('='*60)

    y_val_true_gcn, y_val_pred_gcn = predict_loader_gnn(model_gcn, val_loader)
    print('\nValidation report (GCN):')
    print(classification_report(y_val_true_gcn, y_val_pred_gcn, digits=4,
↪zero_division=0))

    y_test_true_gcn, y_test_pred_gcn = predict_loader_gnn(model_gcn,
↪test_loader)
    print('Test report (GCN):')
    print(classification_report(y_test_true_gcn, y_test_pred_gcn, digits=4,
↪zero_division=0))
```

```
=====
Evaluation for GCN - LR=1e-4
=====
```

Validation report (GCN):

	precision	recall	f1-score	support
0	0.3929	0.7213	0.5087	122
1	0.9888	0.9567	0.9725	3143
accuracy			0.9479	3265
macro avg	0.6908	0.8390	0.7406	3265
weighted avg	0.9666	0.9479	0.9552	3265

Test report (GCN):

	precision	recall	f1-score	support
0	0.3897	0.6543	0.4885	243
1	0.9862	0.9602	0.9731	6262
accuracy			0.9488	6505
macro avg	0.6880	0.8073	0.7308	6505
weighted avg	0.9639	0.9488	0.9550	6505

=====

Evaluation for GCN - LR=5e-4

=====

Validation report (GCN):

	precision	recall	f1-score	support
0	0.4645	0.8033	0.5886	122
1	0.9921	0.9640	0.9779	3143
accuracy			0.9580	3265
macro avg	0.7283	0.8837	0.7832	3265
weighted avg	0.9724	0.9580	0.9633	3265

Test report (GCN):

	precision	recall	f1-score	support
0	0.4583	0.7243	0.5614	243
1	0.9891	0.9668	0.9778	6262
accuracy			0.9577	6505
macro avg	0.7237	0.8455	0.7696	6505
weighted avg	0.9692	0.9577	0.9622	6505

=====

Evaluation for GCN - LR=1e-3

=====

Validation report (GCN):

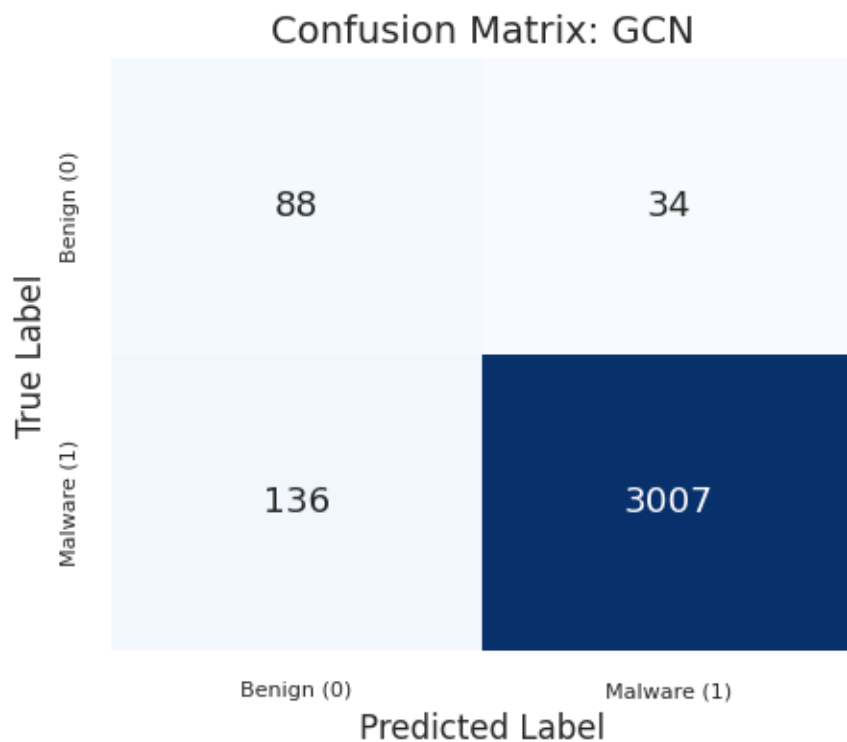
	precision	recall	f1-score	support
0	0.5077	0.8115	0.6246	122
1	0.9925	0.9695	0.9808	3143
accuracy			0.9636	3265
macro avg	0.7501	0.8905	0.8027	3265
weighted avg	0.9744	0.9636	0.9675	3265

Test report (GCN):

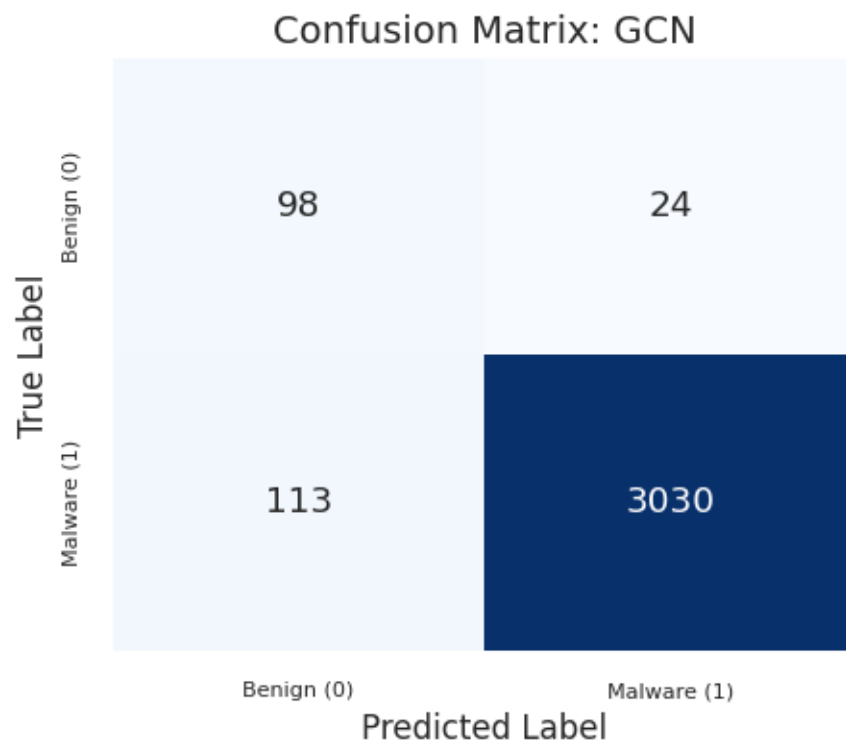
	precision	recall	f1-score	support
0	0.4674	0.7078	0.5630	243
1	0.9884	0.9687	0.9785	6262
accuracy			0.9590	6505
macro avg	0.7279	0.8383	0.7707	6505
weighted avg	0.9690	0.9590	0.9629	6505

```
[130]: for lr_label, model_gcn in gcn_models.items():
        plot_cm_task4(model=model_gcn,
                       loader=val_loader,
                       model_name="GCN",
                       save_dir=results_path + 'images/task4_plots/')
```

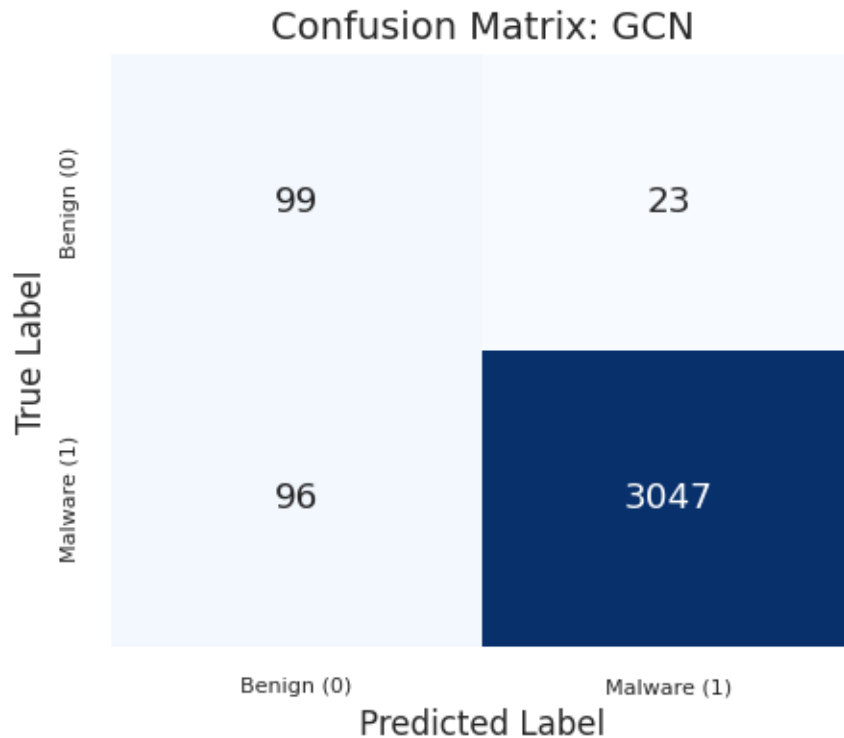
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



```
[131]: # =====
# GCN: Select Best Model and Evaluate
# =====

# Select best model based on final validation F1;
# change this label if another LR performs better
best_gcn_lr = 'LR=5e-4'
model_gcn = gcn_models[best_gcn_lr]
history_gcn = gcn_histories[best_gcn_lr]
elapsed_gcn = gcn_elapsed_times[best_gcn_lr]

print(f"Selected best GCN model: {best_gcn_lr}")
print(f"Final Val F1 Macro: {history_gcn['val_f1_macro'][-1]:.4f}")

# Evaluate
y_val_true_gcn, y_val_pred_gcn = predict_loader_gnn(model_gcn, val_loader)
print('\nValidation report (GCN):')
print(classification_report(y_val_true_gcn, y_val_pred_gcn, digits=4,
    ↪ zero_division=0))

y_test_true_gcn, y_test_pred_gcn = predict_loader_gnn(model_gcn, test_loader)
print('Test report (GCN):')
```

```
print(classification_report(y_test_true_gcn, y_test_pred_gcn, digits=4,
↪zero_division=0))
```

Selected best GCN model: LR=5e-4

Final Val F1 Macro: 0.7667

Validation report (GCN):

	precision	recall	f1-score	support
0	0.4645	0.8033	0.5886	122
1	0.9921	0.9640	0.9779	3143
accuracy			0.9580	3265
macro avg	0.7283	0.8837	0.7832	3265
weighted avg	0.9724	0.9580	0.9633	3265

Test report (GCN):

	precision	recall	f1-score	support
0	0.4583	0.7243	0.5614	243
1	0.9891	0.9668	0.9778	6262
accuracy			0.9577	6505
macro avg	0.7237	0.8455	0.7696	6505
weighted avg	0.9692	0.9577	0.9622	6505

2. GraphSAGE This model uses the GraphSAGE architecture

```
[132]: # --- 2. GraphSAGE Model Definition ---

class GraphSAGEmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(GraphSAGEmodel, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # SAGEConv layers
        self.conv1 = SAGEConv(embed_dim, hidden_dim)
        self.conv2 = SAGEConv(hidden_dim, hidden_dim)
        self.conv3 = SAGEConv(hidden_dim, hidden_dim)

        self.pool = global_mean_pool # or global_max_pool

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
```

```

        nn.Dropout(0.5),
        nn.Linear(hidden_dim, output_dim)
    )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        x = self.embedding(x.squeeze(1))

        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv3(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.pool(x, batch)
        out = self.classifier(x)
        return out

```

```

[133]: # =====
# GraphSAGE: Training with Multiple Learning Rates
# =====

sage_learning_rates = [1e-4, 5e-4, 1e-3]
sage_lr_labels = ['LR=1e-4', 'LR=5e-4', 'LR=1e-3']

sage_histories = {}
sage_models = {}
sage_elapsed_times = {}

for lr, lr_label in zip(sage_learning_rates, sage_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GraphSAGE with {lr_label}")
    print('='*60)

    torch.manual_seed(42)
    np.random.seed(42)

    model_sage_temp = GraphSAGEmodel(
        vocab_size=VOCAB_SIZE_GNN,
        embed_dim=EMBED_DIM_GNN,

```

```

        hidden_dim=HIDDEN_DIM_GNN,
        output_dim=OUTPUT_DIM_GNN
    ).to(device)

    optimizer_temp = optim.AdamW(model_sage_temp.parameters(), lr=lr)

    model_sage_temp, history_temp, elapsed_temp = train_gnn_common(
        model_sage_temp,
        train_loader,
        val_loader,
        optimizer_temp,
        criterion_gnn,
        epochs=EPOCHS_GNN,
        min_delta=MIN_DELTA_GNN,
        patience=PATIENCE_GNN
    )

    sage_models[lr_label] = model_sage_temp
    sage_histories[lr_label] = history_temp
    sage_elapsed_times[lr_label] = elapsed_temp

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
    print(f'\n{lr_label} - Training time: {elapsed_str}')

```

```

=====
Training GraphSAGE with LR=1e-4
=====

```

```

Epoch 1/100 - Train Loss: 0.1102 - Val Loss: 0.0954 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0657 - Val Loss: 0.0667 - Val F1: 0.6469
Epoch 20/100 - Train Loss: 0.0536 - Val Loss: 0.0607 - Val F1: 0.6645
Epoch 30/100 - Train Loss: 0.0473 - Val Loss: 0.0558 - Val F1: 0.7122
Epoch 40/100 - Train Loss: 0.0430 - Val Loss: 0.0543 - Val F1: 0.7385
Epoch 50/100 - Train Loss: 0.0386 - Val Loss: 0.0555 - Val F1: 0.6857
Epoch 60/100 - Train Loss: 0.0356 - Val Loss: 0.0524 - Val F1: 0.7156
Epoch 70/100 - Train Loss: 0.0322 - Val Loss: 0.0507 - Val F1: 0.7414
Epoch 80/100 - Train Loss: 0.0297 - Val Loss: 0.0540 - Val F1: 0.7025
Epoch 90/100 - Train Loss: 0.0267 - Val Loss: 0.0498 - Val F1: 0.7535
Epoch 100/100 - Train Loss: 0.0269 - Val Loss: 0.0513 - Val F1: 0.7934

```

```

LR=1e-4 - Training time: 00:04:51

```

```

=====
Training GraphSAGE with LR=5e-4
=====

```

```

Epoch 1/100 - Train Loss: 0.0974 - Val Loss: 0.0829 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0416 - Val Loss: 0.0534 - Val F1: 0.7157
Epoch 20/100 - Train Loss: 0.0292 - Val Loss: 0.0524 - Val F1: 0.7902

```

Epoch 30/100 - Train Loss: 0.0233 - Val Loss: 0.0536 - Val F1: 0.8315
 Epoch 40/100 - Train Loss: 0.0201 - Val Loss: 0.0608 - Val F1: 0.8290
 Early stopping at epoch 42 (best val loss 0.045433)

LR=5e-4 - Training time: 00:02:03

```
=====
Training GraphSAGE with LR=1e-3
=====
```

Epoch 1/100 - Train Loss: 0.0913 - Val Loss: 0.0794 - Val F1: 0.5415
 Epoch 10/100 - Train Loss: 0.0331 - Val Loss: 0.0476 - Val F1: 0.7455
 Epoch 20/100 - Train Loss: 0.0224 - Val Loss: 0.0533 - Val F1: 0.8184
 Epoch 30/100 - Train Loss: 0.0186 - Val Loss: 0.0504 - Val F1: 0.8449
 Early stopping at epoch 34 (best val loss 0.044953)

LR=1e-3 - Training time: 00:01:40

```
[134]: # =====
# GraphSAGE: Plot Learning Rate Comparison
# =====

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#377eb8', '#e41a1c', '#4daf4a']

for (lr_label, history), color in zip(sage_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label, linewidth=2,
    ↪color=color)

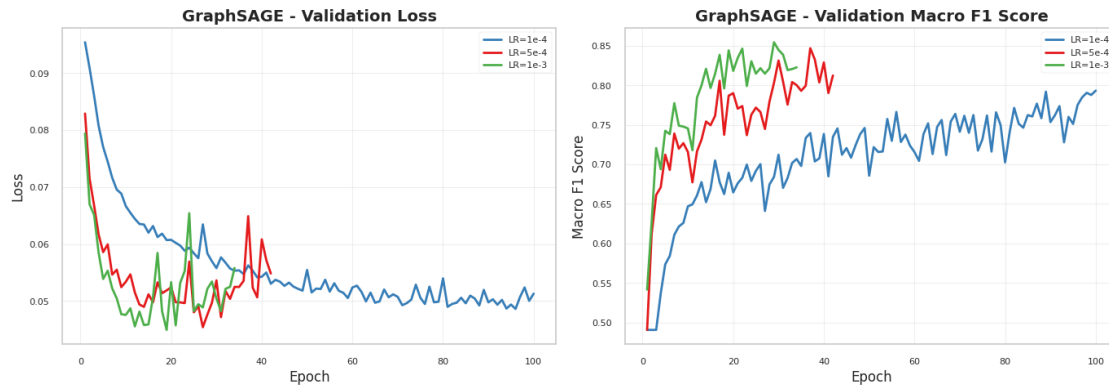
ax1.set_title('GraphSAGE - Validation Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.legend()
ax1.grid(alpha=0.3)

for (lr_label, history), color in zip(sage_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,
    ↪linewidth=2, color=color)

ax2.set_title('GraphSAGE - Validation Macro F1 Score', fontsize=14,
    ↪fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Macro F1 Score', fontsize=12)
ax2.legend()
ax2.grid(alpha=0.3)
```

```
plt.tight_layout()
save_plot(plt.gcf(), f"task4_sage_lr_comparison.png", save_dir)
plt.show()
```

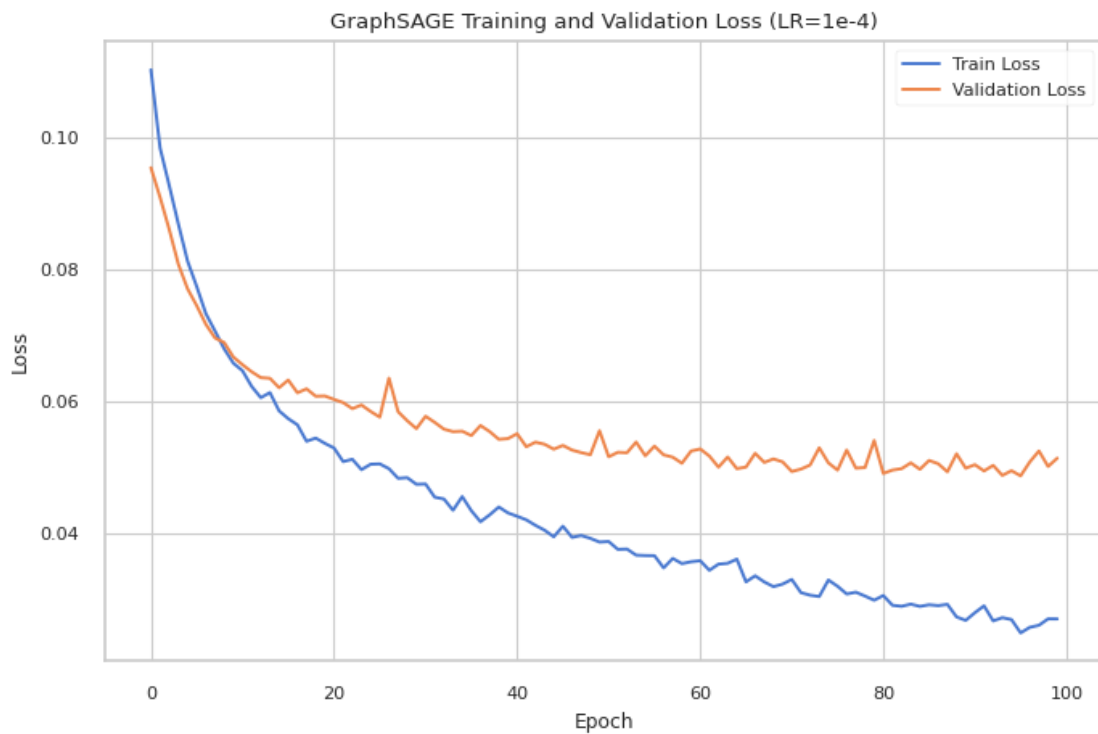
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_sage_lr_comparison.png.png



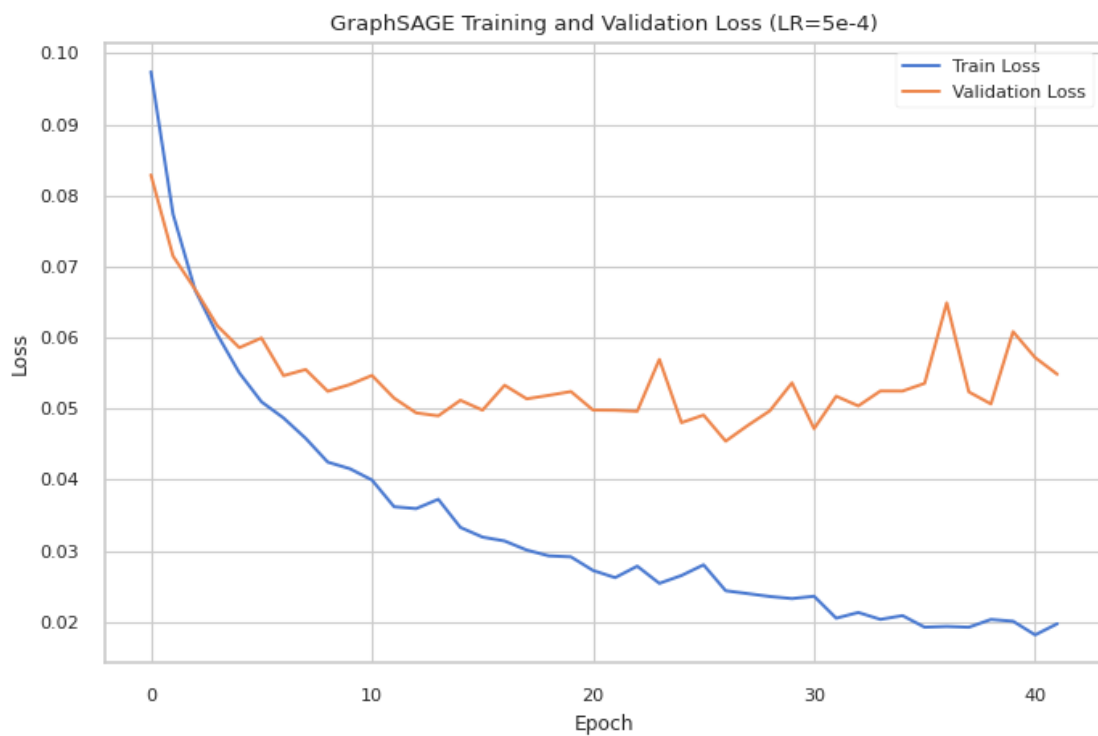
```
[135]: for lr_label, model_sage in sage_models.items():
        history = sage_histories[lr_label] # <-- Prendi la history per questo LR

        plt.figure(figsize=(8, 5))
        plt.plot(history['train_loss'], label='Train Loss')
        plt.plot(history['val_loss'], label='Validation Loss')
        plt.title(f'GraphSAGE Training and Validation Loss ({lr_label})')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()
        plt.grid(True)
        save_plot(plt.gcf(), f'graphsage_loss_curves_{lr_label.replace("=", "")}',
        ↪save_dir)
        plt.show()
```

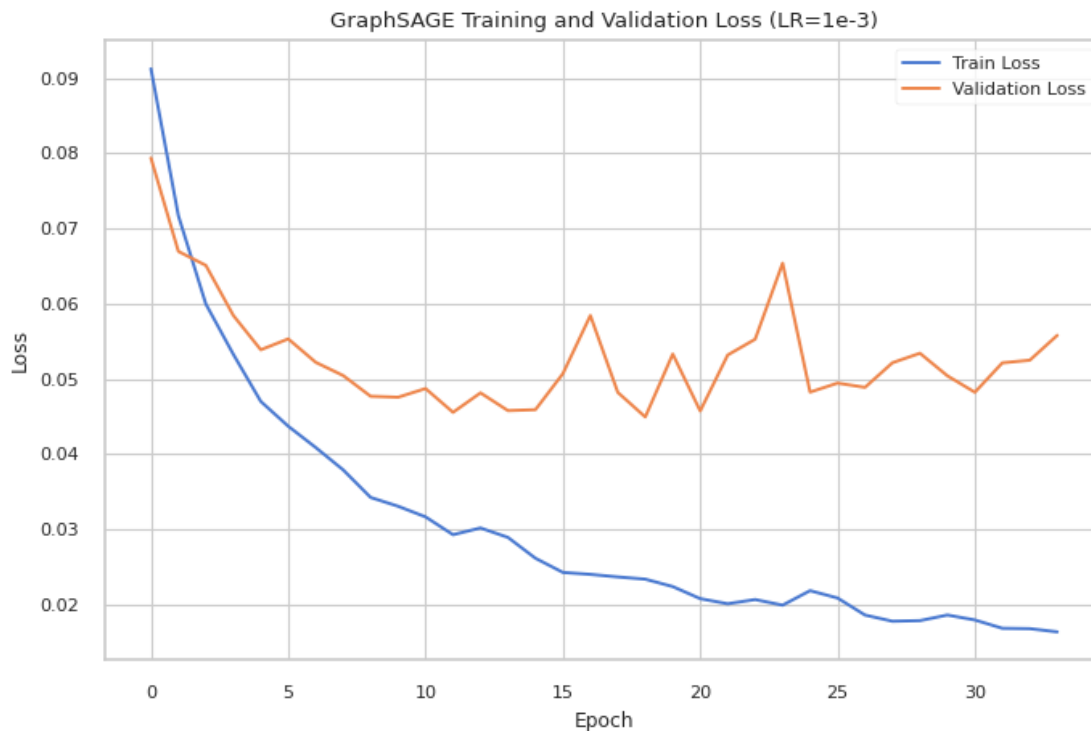
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR1e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR5e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR1e-3.png



```
[136]: # --- Evaluate GraphSAGE ---
for lr_label, model_sage in sage_models.items():
    y_val_true_sage, y_val_pred_sage = predict_loader_gnn(model_sage, val_loader)
    print('\nValidation report (GraphSAGE):')
    print(classification_report(y_val_true_sage, y_val_pred_sage, digits=4,
    ↪zero_division=0))

    y_test_true_sage, y_test_pred_sage = predict_loader_gnn(model_sage,
    ↪test_loader)
    print('Test report (GraphSAGE):')
    print(classification_report(y_test_true_sage, y_test_pred_sage, digits=4,
    ↪zero_division=0))
```

Validation report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4125	0.8115	0.5470	122

1	0.9924	0.9551	0.9734	3143
accuracy			0.9498	3265
macro avg	0.7024	0.8833	0.7602	3265
weighted avg	0.9707	0.9498	0.9575	3265

Test report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4311	0.7984	0.5599	243
1	0.9919	0.9591	0.9752	6262
accuracy			0.9531	6505
macro avg	0.7115	0.8787	0.7676	6505
weighted avg	0.9710	0.9531	0.9597	6505

Validation report (GraphSAGE):

	precision	recall	f1-score	support
0	0.3778	0.8361	0.5204	122
1	0.9933	0.9465	0.9694	3143
accuracy			0.9424	3265
macro avg	0.6855	0.8913	0.7449	3265
weighted avg	0.9703	0.9424	0.9526	3265

Test report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4148	0.8313	0.5534	243
1	0.9932	0.9545	0.9735	6262
accuracy			0.9499	6505
macro avg	0.7040	0.8929	0.7634	6505
weighted avg	0.9716	0.9499	0.9578	6505

Validation report (GraphSAGE):

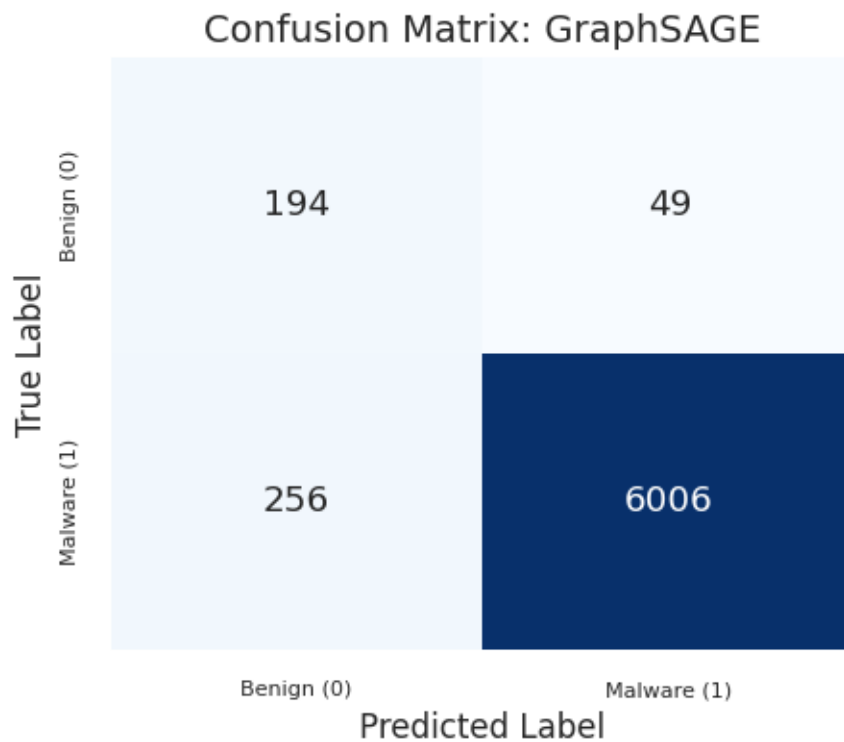
	precision	recall	f1-score	support
0	0.6742	0.7295	0.7008	122
1	0.9895	0.9863	0.9879	3143
accuracy			0.9767	3265
macro avg	0.8319	0.8579	0.8443	3265
weighted avg	0.9777	0.9767	0.9772	3265

Test report (GraphSAGE):

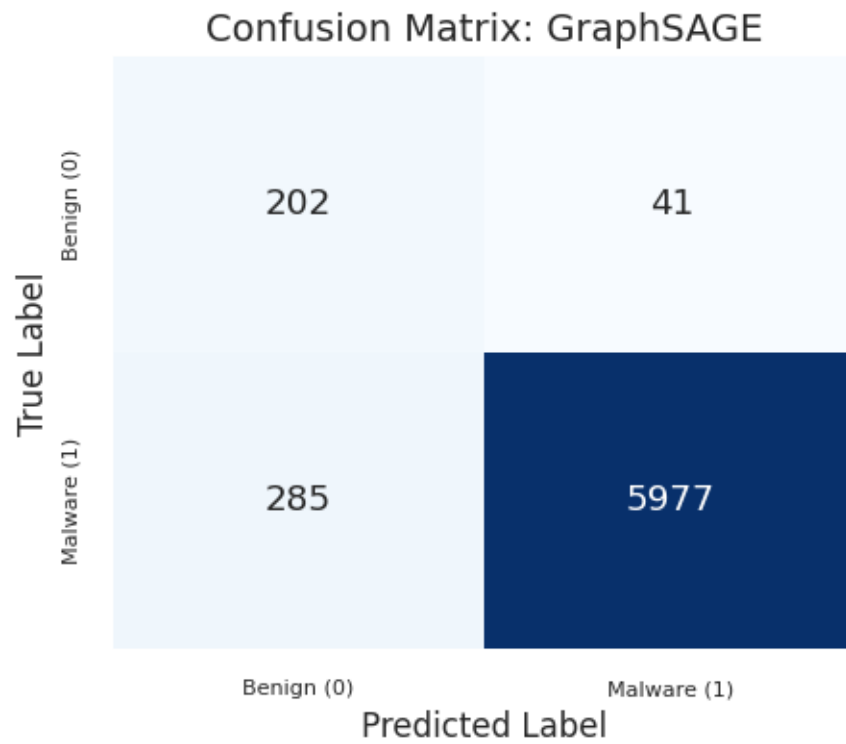
	precision	recall	f1-score	support
0	0.6446	0.6420	0.6433	243
1	0.9861	0.9863	0.9862	6262
accuracy			0.9734	6505
macro avg	0.8154	0.8141	0.8147	6505
weighted avg	0.9734	0.9734	0.9734	6505

```
[137]: for lr_label, model_sage in sage_models.items():
        plot_cm_task4(model=model_sage,
                       loader=test_loader,
                       model_name="GraphSAGE",
                       save_dir=results_path + 'images/task4_plots/')
```

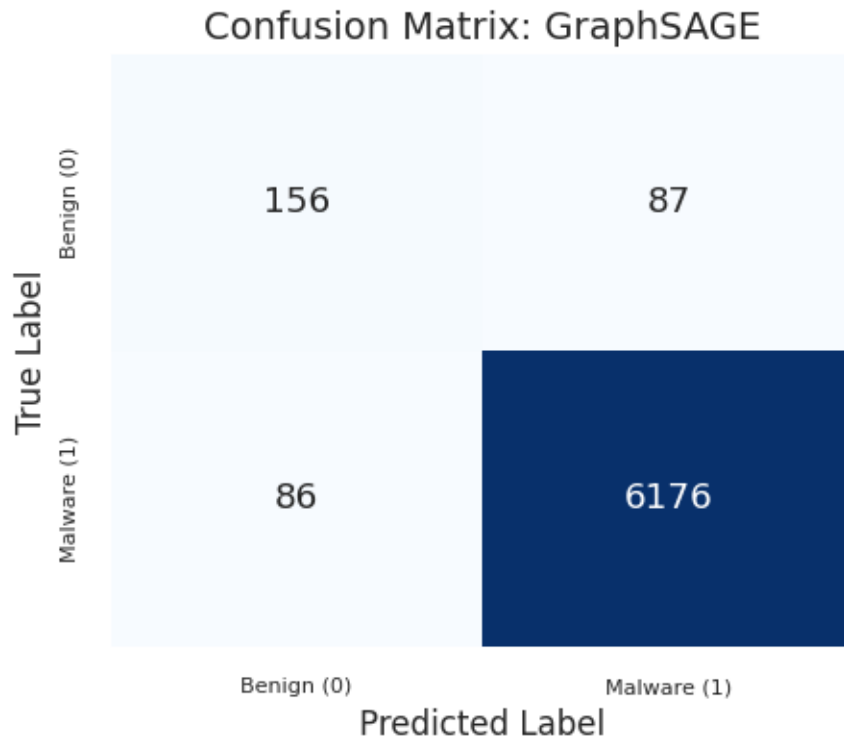
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_graphsage.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_graphsage.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_graphsage.png



```
[138]: # =====
# GraphSAGE: Select Best Model and Evaluate
# =====

best_sage_lr = 'LR=5e-4'
model_sage = sage_models[best_sage_lr]
history_sage = sage_histories[best_sage_lr]
elapsed_sage = sage_elapsed_times[best_sage_lr]

print(f"Selected best GraphSAGE model: {best_sage_lr}")
print(f"Final Val F1 Macro: {history_sage['val_f1_macro'][-1]:.4f}")

y_val_true_sage, y_val_pred_sage = predict_loader_gnn(model_sage, val_loader)
print('\nValidation report (GraphSAGE):')
print(classification_report(y_val_true_sage, y_val_pred_sage, digits=4,
    ↪zero_division=0))

y_test_true_sage, y_test_pred_sage = predict_loader_gnn(model_sage, test_loader)
print('Test report (GraphSAGE):')
print(classification_report(y_test_true_sage, y_test_pred_sage, digits=4,
    ↪zero_division=0))
```

Selected best GraphSAGE model: LR=5e-4

Final Val F1 Macro: 0.8125

Validation report (GraphSAGE):

	precision	recall	f1-score	support
0	0.3778	0.8361	0.5204	122
1	0.9933	0.9465	0.9694	3143
accuracy			0.9424	3265
macro avg	0.6855	0.8913	0.7449	3265
weighted avg	0.9703	0.9424	0.9526	3265

Test report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4148	0.8313	0.5534	243
1	0.9932	0.9545	0.9735	6262
accuracy			0.9499	6505
macro avg	0.7040	0.8929	0.7634	6505
weighted avg	0.9716	0.9499	0.9578	6505

3. GAT This model uses the Graph Attention Network (GAT) architecture.

```
[139]: class GATmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim, heads=4):
        super(GATmodel, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # GAT Layers
        # Layer 1: Output is hidden_dim * heads
        self.conv1 = GATConv(embed_dim, hidden_dim, heads=heads, dropout=0.2)

        # Layer 2: Input is hidden_dim * heads. We set heads=1 for the final
        ↪ graph embedding before pooling.
        # concat=False means we average the heads or just output one head.
        self.conv2 = GATConv(hidden_dim * heads, hidden_dim, heads=1,
        ↪ concat=False, dropout=0.2)

        # Layer 3: Input should now be hidden_dim (output of conv2)
        self.conv3 = GATConv(hidden_dim, hidden_dim, heads=1, concat=False,
        ↪ dropout=0.2)

        self.pool = global_mean_pool
```

```

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        x = self.embedding(x.squeeze(1))

        # Conv 1
        x = self.conv1(x, edge_index)
        x = F.elu(x)

        # Conv 2
        x = self.conv2(x, edge_index)

        # Conv 3
        x = self.conv3(x, edge_index)

        # Pooling
        x = self.pool(x, batch)

        # Classifier
        out = self.classifier(x)
        return out

```

```

[140]: # =====
# GAT: Training with Multiple Learning Rates
# =====

gat_learning_rates = [5e-5, 1e-4, 5e-4]
gat_lr_labels = ['LR=5e-5', 'LR=1e-4', 'LR=5e-4']

gat_histories = {}
gat_models = {}
gat_elapsed_times = {}

for lr, lr_label in zip(gat_learning_rates, gat_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GAT with {lr_label}")
    print(f"{'='*60}")

    torch.manual_seed(42)
    np.random.seed(42)

```

```

model_gat_temp = GATmodel(
    vocab_size=VOCAB_SIZE_GNN,
    embed_dim=EMBED_DIM_GNN,
    hidden_dim=HIDDEN_DIM_GNN,
    output_dim=OUTPUT_DIM_GNN,
    heads=4
).to(device)

optimizer_temp = optim.AdamW(model_gat_temp.parameters(), lr=lr)

model_gat_temp, history_temp, elapsed_temp = train_gnn_common(
    model_gat_temp,
    train_loader,
    val_loader,
    optimizer_temp,
    criterion_gnn,
    epochs=EPOCHS_GNN,
    min_delta=MIN_DELTA_GNN,
    patience=PATIENCE_GNN
)

gat_models[lr_label] = model_gat_temp
gat_histories[lr_label] = history_temp
gat_elapsed_times[lr_label] = elapsed_temp

elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
print(f'\n{lr_label} - Training time: {elapsed_str}')

```

```

=====
Training GAT with LR=5e-5
=====

```

```

Epoch 1/100 - Train Loss: 0.1187 - Val Loss: 0.0990 - Val F1: 0.4902
Epoch 10/100 - Train Loss: 0.0725 - Val Loss: 0.0689 - Val F1: 0.6144
Epoch 20/100 - Train Loss: 0.0610 - Val Loss: 0.0597 - Val F1: 0.6953
Epoch 30/100 - Train Loss: 0.0557 - Val Loss: 0.0562 - Val F1: 0.7096
Epoch 40/100 - Train Loss: 0.0509 - Val Loss: 0.0540 - Val F1: 0.7248
Epoch 50/100 - Train Loss: 0.0485 - Val Loss: 0.0532 - Val F1: 0.7274
Epoch 60/100 - Train Loss: 0.0435 - Val Loss: 0.0491 - Val F1: 0.7633
Epoch 70/100 - Train Loss: 0.0432 - Val Loss: 0.0480 - Val F1: 0.7570
Epoch 80/100 - Train Loss: 0.0420 - Val Loss: 0.0485 - Val F1: 0.7560
Epoch 90/100 - Train Loss: 0.0413 - Val Loss: 0.0469 - Val F1: 0.7604
Epoch 100/100 - Train Loss: 0.0392 - Val Loss: 0.0453 - Val F1: 0.7795

```

```

LR=5e-5 - Training time: 00:07:29

```

```

=====
Training GAT with LR=1e-4

```

```

=====
Epoch 1/100 - Train Loss: 0.1088 - Val Loss: 0.0926 - Val F1: 0.5266
Epoch 10/100 - Train Loss: 0.0629 - Val Loss: 0.0612 - Val F1: 0.6938
Epoch 20/100 - Train Loss: 0.0523 - Val Loss: 0.0528 - Val F1: 0.7326
Epoch 30/100 - Train Loss: 0.0466 - Val Loss: 0.0506 - Val F1: 0.7486
Epoch 40/100 - Train Loss: 0.0423 - Val Loss: 0.0463 - Val F1: 0.7680
Epoch 50/100 - Train Loss: 0.0381 - Val Loss: 0.0458 - Val F1: 0.7440
Epoch 60/100 - Train Loss: 0.0343 - Val Loss: 0.0433 - Val F1: 0.7948
Epoch 70/100 - Train Loss: 0.0366 - Val Loss: 0.0442 - Val F1: 0.7525
Epoch 80/100 - Train Loss: 0.0326 - Val Loss: 0.0421 - Val F1: 0.7849
Epoch 90/100 - Train Loss: 0.0325 - Val Loss: 0.0403 - Val F1: 0.7898
Epoch 100/100 - Train Loss: 0.0300 - Val Loss: 0.0399 - Val F1: 0.8014

```

LR=1e-4 - Training time: 00:07:26

```

=====
Training GAT with LR=5e-4
=====
Epoch 1/100 - Train Loss: 0.0913 - Val Loss: 0.0726 - Val F1: 0.6441
Epoch 10/100 - Train Loss: 0.0486 - Val Loss: 0.0553 - Val F1: 0.7479
Epoch 20/100 - Train Loss: 0.0375 - Val Loss: 0.0433 - Val F1: 0.7685
Epoch 30/100 - Train Loss: 0.0331 - Val Loss: 0.0462 - Val F1: 0.7710
Epoch 40/100 - Train Loss: 0.0302 - Val Loss: 0.0399 - Val F1: 0.8133
Epoch 50/100 - Train Loss: 0.0291 - Val Loss: 0.0397 - Val F1: 0.7744
Epoch 60/100 - Train Loss: 0.0251 - Val Loss: 0.0442 - Val F1: 0.8000
Early stopping at epoch 62 (best val loss 0.037055)

```

LR=5e-4 - Training time: 00:04:38

```

[141]: # =====
# GAT: Plot Learning Rate Comparison
# =====

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#984ea3', '#17becf', '#ff7f00']

for (lr_label, history), color in zip(gat_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label, linewidth=2,
            color=color)

ax1.set_title('GAT - Validation Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.legend()
ax1.grid(alpha=0.3)

```

```

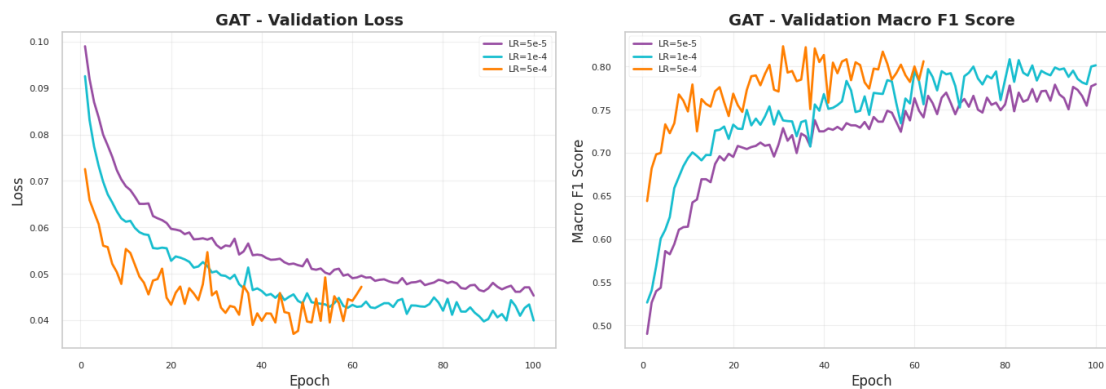
for (lr_label, history), color in zip(gat_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,
             linewidth=2, color=color)

ax2.set_title('GAT - Validation Macro F1 Score', fontsize=14, fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Macro F1 Score', fontsize=12)
ax2.legend()
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gat_lr_comparison.png", save_dir)
plt.show()

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gat_lr_comparison.png.png



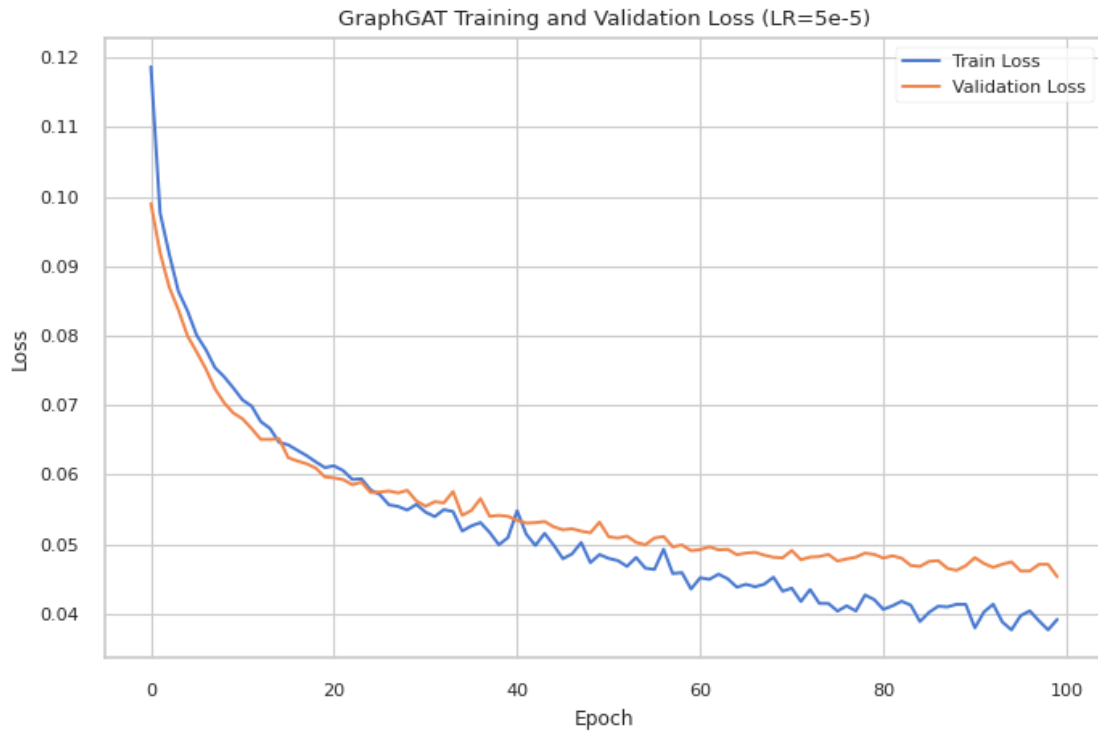
```

[142]: for lr_label, model_gat in gat_models.items():
        history = gat_histories[lr_label] # <-- Prendi la history per questo LR

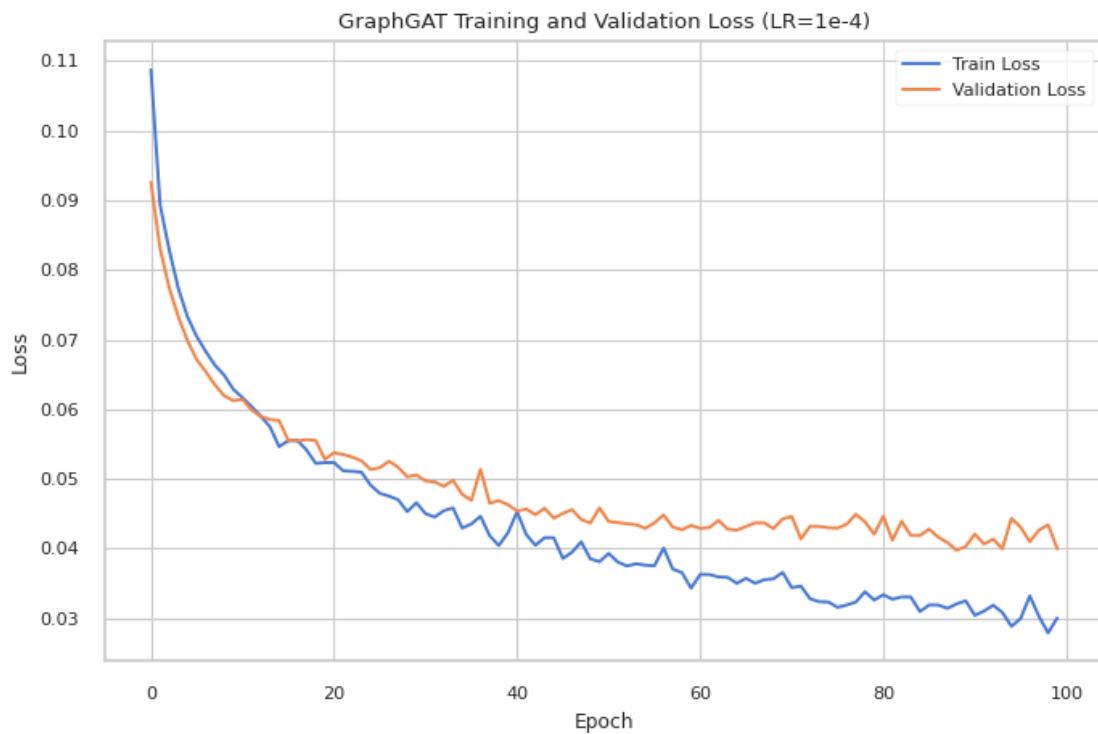
        plt.figure(figsize=(8, 5))
        plt.plot(history['train_loss'], label='Train Loss')
        plt.plot(history['val_loss'], label='Validation Loss')
        plt.title(f'GraphGAT Training and Validation Loss ({lr_label})')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()
        plt.grid(True)
        save_plot(plt.gcf(), f'graphgat_loss_curves_{lr_label.replace("=", "")}',
                 save_dir)
        plt.show()

```

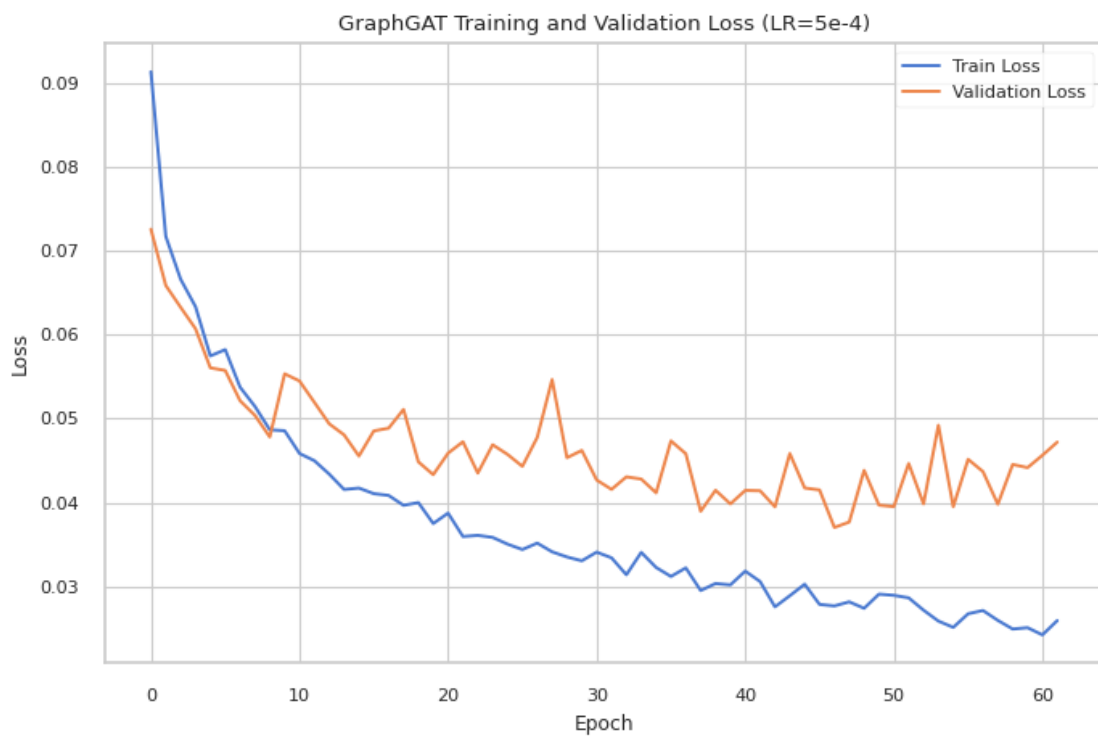
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR5e-5.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR1e-4.png



Saved plot: `/content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR5e-4.png`



```
[143]: # --- Evaluate GAT ---
for lr_label, model_gat in gat_models.items():
    y_val_true_gat, y_val_pred_gat = predict_loader_gnn(model_gat, val_loader)
    print('\nValidation report (GAT):')
    print(classification_report(y_val_true_gat, y_val_pred_gat, digits=4,
    ↪zero_division=0))

    y_test_true_gat, y_test_pred_gat = predict_loader_gnn(model_gat, test_loader)
    print('Test report (GAT):')
    print(classification_report(y_test_true_gat, y_test_pred_gat, digits=4,
    ↪zero_division=0))
```

Validation report (GAT):

	precision	recall	f1-score	support
0	0.5030	0.6803	0.5784	122
1	0.9874	0.9739	0.9806	3143
accuracy			0.9629	3265
macro avg	0.7452	0.8271	0.7795	3265
weighted avg	0.9693	0.9629	0.9656	3265

Test report (GAT):

	precision	recall	f1-score	support
0	0.5111	0.6626	0.5771	243
1	0.9868	0.9754	0.9810	6262
accuracy			0.9637	6505
macro avg	0.7489	0.8190	0.7791	6505
weighted avg	0.9690	0.9637	0.9660	6505

Validation report (GAT):

	precision	recall	f1-score	support
0	0.5027	0.7541	0.6033	122
1	0.9903	0.9710	0.9806	3143
accuracy			0.9629	3265
macro avg	0.7465	0.8626	0.7919	3265
weighted avg	0.9720	0.9629	0.9665	3265

Test report (GAT):

	precision	recall	f1-score	support
0	0.5113	0.7449	0.6064	243
1	0.9899	0.9724	0.9811	6262
accuracy			0.9639	6505
macro avg	0.7506	0.8586	0.7937	6505
weighted avg	0.9720	0.9639	0.9671	6505

Validation report (GAT):

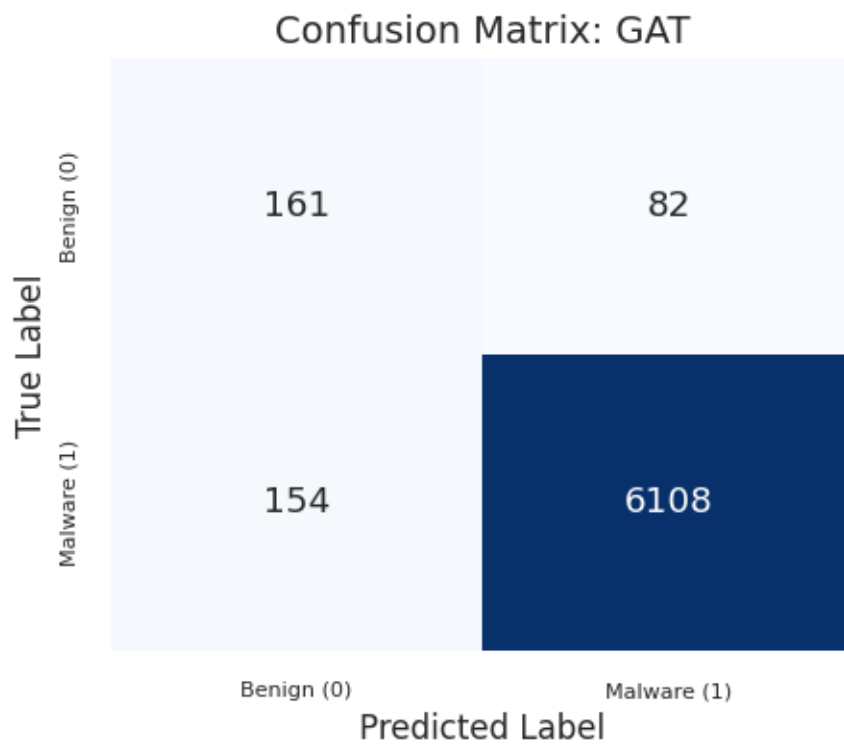
	precision	recall	f1-score	support
0	0.5130	0.8115	0.6286	122
1	0.9925	0.9701	0.9812	3143
accuracy			0.9642	3265
macro avg	0.7527	0.8908	0.8049	3265
weighted avg	0.9746	0.9642	0.9680	3265

Test report (GAT):

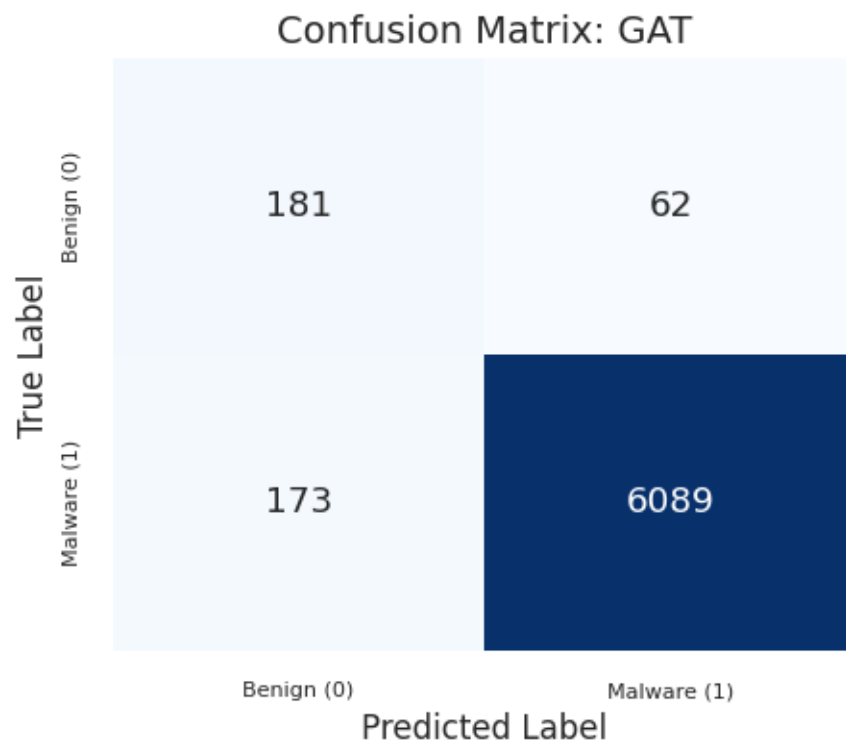
	precision	recall	f1-score	support
0	0.4672	0.7901	0.5872	243
1	0.9916	0.9650	0.9781	6262
accuracy			0.9585	6505
macro avg	0.7294	0.8776	0.7827	6505
weighted avg	0.9720	0.9585	0.9635	6505

```
[144]: for lr_label, model_gat in gat_models.items():
        plot_cm_task4(model=model_gat,
                      loader=test_loader,
                      model_name="GAT",
                      save_dir=results_path + 'images/task4_plots/')
```

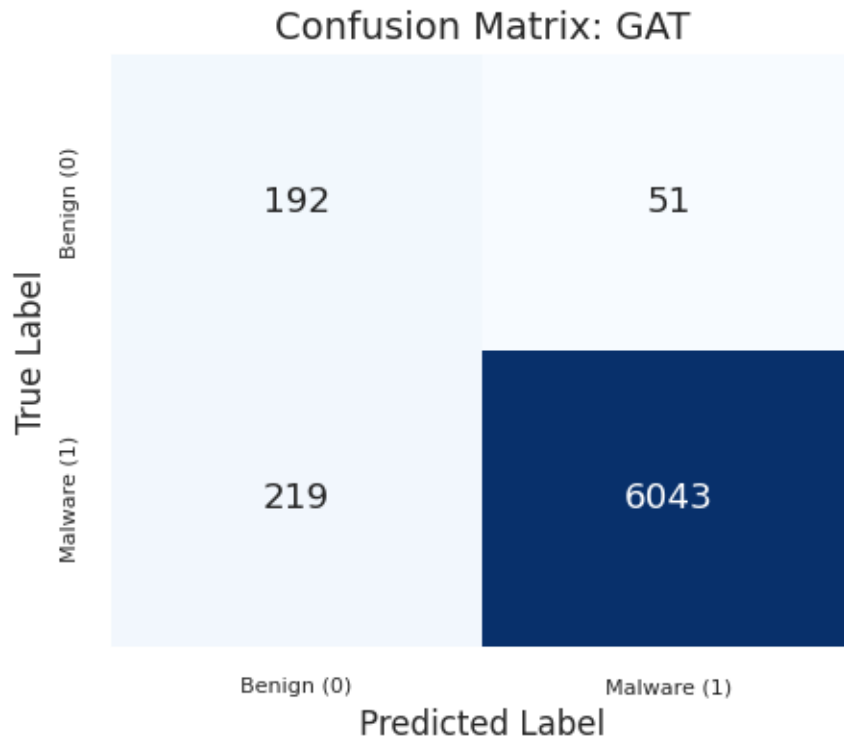
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



```
[145]: # =====
# GAT: Select Best Model and Evaluate
# =====

best_gat_lr = 'LR=5e-4'
model_gat = gat_models[best_gat_lr]
history_gat = gat_histories[best_gat_lr]
elapsed_gat = gat_elapsed_times[best_gat_lr]

print(f"Selected best GAT model: {best_gat_lr}")
print(f"Final Val F1 Macro: {history_gat['val_f1_macro'][-1]:.4f}")

y_val_true_gat, y_val_pred_gat = predict_loader_gnn(model_gat, val_loader)
print('\nValidation report (GAT):')
print(classification_report(y_val_true_gat, y_val_pred_gat, digits=4,
    ↪zero_division=0))

y_test_true_gat, y_test_pred_gat = predict_loader_gnn(model_gat, test_loader)
print('Test report (GAT):')
print(classification_report(y_test_true_gat, y_test_pred_gat, digits=4,
    ↪zero_division=0))
```

Selected best GAT model: LR=5e-4

Final Val F1 Macro: 0.8061

Validation report (GAT):

	precision	recall	f1-score	support
0	0.5130	0.8115	0.6286	122
1	0.9925	0.9701	0.9812	3143
accuracy			0.9642	3265
macro avg	0.7527	0.8908	0.8049	3265
weighted avg	0.9746	0.9642	0.9680	3265

Test report (GAT):

	precision	recall	f1-score	support
0	0.4672	0.7901	0.5872	243
1	0.9916	0.9650	0.9781	6262
accuracy			0.9585	6505
macro avg	0.7294	0.8776	0.7827	6505
weighted avg	0.9720	0.9585	0.9635	6505

```
[146]: # =====
# GNN Comparison: Validation Macro F1 Score (Best LR for each model)
# =====

fig, ax = plt.subplots(figsize=(10, 6))

# Plot best model for each architecture
best_models_gnn = {
    'GCN': history_gcn,
    'GraphSAGE': history_sage,
    'GAT': history_gat
}

colors_gnn = ['#1b9e77', '#377eb8', '#984ea3']

for (name, history), color in zip(best_models_gnn.items(), colors_gnn):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax.plot(epochs_range, history['val_f1_macro'], label=name, linewidth=2.5,
            color=color)

ax.set_title('GNN Architectures - Validation Macro F1 Score Comparison',
            fontweight='bold',
            fontsize=14)
ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Macro F1 Score', fontsize=12)
```

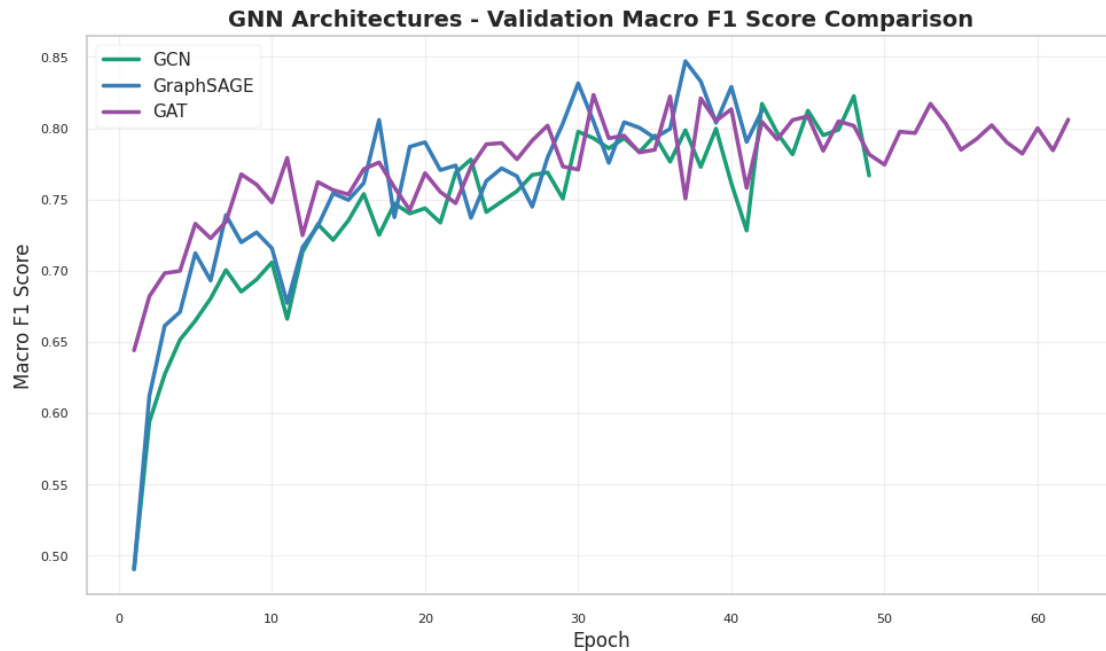
```

ax.legend(fontsize=11)
ax.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gnn_comparison_f1_macro", save_dir)
plt.show()

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gnn_comparison_f1_macro.png



```

[147]: # =====
# FINAL REPORT: All Architectures Comparison (FFNN, RNN, GNN)
# =====

def get_test_metrics(y_true, y_pred):
    """Extract test metrics from predictions."""
    report = classification_report(y_true, y_pred, output_dict=True,
    ↪ zero_division=0)
    return {
        'precision': report['macro avg']['precision'],
        'recall': report['macro avg']['recall'],
        'f1_score': report['macro avg']['f1-score'],
        'accuracy': report['accuracy']
    }

# Collect metrics for all models

```

```

# Note: Adjust variable names based on your Task 2 and Task 3 results

# --- FFNN (from Task 2) ---
# Assuming you have: y_test_ffnn_true, y_test_ffnn_pred, t_seq (time)
# If variable names differ, adjust accordingly

# --- RNN (from Task 3) ---
# Assuming you have: y_test_true_lstm, y_test_pred_lstm, t_lstm (time)
# If variable names differ, adjust accordingly

# --- GNN (from Task 4) ---
metrics_gcn = get_test_metrics(y_test_true_gcn, y_test_pred_gcn)
metrics_sage = get_test_metrics(y_test_true_sage, y_test_pred_sage)
metrics_gat = get_test_metrics(y_test_true_gat, y_test_pred_gat)

# Build summary table
print("\n" + "="*100)
print("FINAL REPORT: Model Architecture Comparison (Test Set)")
print("="*100)
print(f"{'Architecture':<20} {'Precision':<12} {'Recall':<12} {'F1-Score':<12}␣
    ↳{'Accuracy':<12} {'Time':<15}")
print("-"*100)

# Print GNN results
print(f"{'GCN':<20} {metrics_gcn['precision']:.4f}           {metrics_gcn['recall']:
    ↳.4f}           {metrics_gcn['f1_score']:.4f}           {metrics_gcn['accuracy']:.4f}␣
    ↳           {elapsed_gcn:.2f}s")
print(f"{'GraphSAGE':<20} {metrics_sage['precision']:.4f}           ␣
    ↳{metrics_sage['recall']:.4f}           {metrics_sage['f1_score']:.4f}           ␣
    ↳{metrics_sage['accuracy']:.4f}           {elapsed_sage:.2f}s")
print(f"{'GAT':<20} {metrics_gat['precision']:.4f}           {metrics_gat['recall']:
    ↳.4f}           {metrics_gat['f1_score']:.4f}           {metrics_gat['accuracy']:.4f}␣
    ↳           {elapsed_gat:.2f}s")

print("="*100)

```

```

=====
=====
FINAL REPORT: Model Architecture Comparison (Test Set)
=====
=====

```

Architecture	Precision	Recall	F1-Score	Accuracy	Time
GCN	0.7237	0.8455	0.7696	0.9577	162.56s
GraphSAGE	0.7040	0.8929	0.7634	0.9499	123.70s

GAT	0.7294	0.8776	0.7827	0.9585	278.72s
=====					
=====					

Q: How does each model perform compared to the previous architectures? Can you beat the baseline? Our experimental results confirm that while the frequency-based baseline is competitive in accuracy, deep learning architectures are essential for a balanced and reliable classification.

1. **Beating the Baseline** We successfully outperformed the baseline’s Macro F1-score (0.7223). Although the baseline achieved high accuracy (96.97%), it suffered from a severe bias toward the majority class, failing to identify ~65% of benign samples (Recall: 34.57%). In contrast, our best models—LSTM and GAT—dramatically improved benign detection (Recall > 74%), resulting in significantly higher Macro F1-scores (0.8346 and 0.7903, respectively).
2. **Architecture Comparison** FFNN (Task 2): These models were the weakest (Macro F1: 0.56–0.66), performing below the baseline. Their static nature prevents them from capturing the crucial temporal relationships between API calls, even when using learnable embeddings.

RNNs & LSTM (Task 3): Moving to sequential models provided the largest performance boost. By processing the chronological order of calls, the LSTM achieved the best overall results (Macro F1: 0.8346). Its gating mechanism effectively manages long-term dependencies that simpler RNNs might miss.

GNNs (Task 4): Graph-based models (GCN, GraphSAGE, GAT) proved highly robust. GAT emerged as the top GNN performer (Macro F1: 0.7903). While slightly behind the LSTM in F1-score, GNNs excels at modeling the structural “behavioral signature” of the software, capturing complex non-linear transitions between system calls.

[147]: